



Department of Computer Science

Final Year Project Proposal

Bursary-Bridge: A Web-Based Bursary Matching and Application Management System for South African Students and Funding Organisations

Supervisor

Ms Zulu

Submitted by

Student Name	Student Number
1. CPP Mchunu	202242486
2. A Nsibande	240079346
3. KN Bikile	202356797
4. SP Tshazi	240103222
5.	
6.	
7.	
8.	

September 2026

Table of Contents

1. Introduction and Background.....	3
2. Problem Statement.....	5
3. Goal and Objectives.....	6
3.1. Goal.....	6
3.2. Objectives.....	6
4. Methodology.....	7
5. Equipment, Tools, and Technologies Required for the Project.....	10
6. Project Scope.....	11
7. Project Beneficiaries.....	13
8. Project Assumptions and Constraints.....	14
9. Project Risks.....	15
10. Project Work Plan.....	17
10.1. Expertise of the Team members.....	17
10.2. Work plan.....	17
11. References.....	19
12. Appendices (if applicable).....	20

1. Introduction and Background

Post-school education in South Africa is large, expanding and expensive. The Department of Higher Education and Training records a headcount enrolment of 1 071 715 students at public higher education institutions in 2023, with a further 286 454 at private institutions, giving 1 358 169 students in the higher education sector in that year alone [1]. For most of these students the decisive question is not academic but financial: whether the tuition, accommodation, transport and materials for the coming year can be paid for. Where the answer is no, a qualified student does not register, or registers and then withdraws.

The funding that answers that question comes from two very different places, and they are organised very differently. The first is the state. The National Student Financial Aid Scheme administers the Department of Higher Education and Training bursary scheme, which is means-tested: an applicant qualifies where combined annual household income is not more than R350 000, or not more than R600 000 where the applicant has a disability [2]. Students whose household income falls above the bursary threshold but below R600 000 a year, commonly described as the missing middle, are directed instead to a loan facility rather than to a bursary [2]. The scheme is a single, well-signposted door: one applicant, one application, one published set of rules.

The second source is everything else. Corporations, foundations, state-owned enterprises, professional bodies, government departments and universities themselves fund education through bursaries, scholarships and grants that each carry their own criteria, their own closing dates and their own application form. There is no register of these opportunities and no shared application process. A student looking for them is searching a scattered and largely unstructured field, and it is this second source that this project addresses.

The way that field is published today is worth describing precisely, because it determines what any system can do with it. Bursary information reaches students principally through three channels. Aggregator websites collect opportunities and publish them as editorial articles grouped by field of study; the largest South African example, which this project crawled with permission of its robots directives, organises its listings across thirteen such category pages [3]. Funder websites publish the same information again on a careers or corporate-social-investment page, usually as prose and often as a downloadable PDF. Institutional financial aid offices circulate what they have been sent, by notice board, email list or campus portal. In all three channels the unit of publication is a page of text written for a human reader. Closing dates appear inside sentences, eligibility rules appear as bullet points in varying wording, and the same opportunity is frequently republished in several places with different details.

The consequences of that publishing model are visible on both sides of the transaction. For the student, discovery is a matter of searching rather than of being matched: the tools available return lists of opportunities that contain a keyword, not lists of opportunities the student is actually eligible for.

Eligibility must then be established by reading each page in full, and because the rules are prose rather than data, the reading is easy to get wrong. Students consequently spend effort on applications they cannot win and miss ones they could. Each application that is attempted requires the same personal, academic and financial details to be written out again, most often onto a PDF form that is downloaded, printed or filled by hand, and returned by email.

For the funding organisation the mirror image holds. Applications arrive as email attachments in whatever shape the applicant sent them, and are sifted by hand or transcribed into a spreadsheet before they can be compared. Screening an applicant against the organisation's own criteria is manual, so it is slow and inconsistent, and it does not scale: an organisation that advertises widely is punished for it with a larger pile of unstructured documents. Because the process leaves no structured record of why an applicant was set aside, the organisation cannot report on its own selection, and the unsuccessful applicant receives no explanation.

Software already exists in adjacent parts of this problem. University student information systems manage registration and academic records but do not source external funding. Applicant tracking systems used in recruitment manage a pipeline of candidates but are built around employment, are licensed commercially, and do not model academic eligibility such as minimum averages, fields of study or subject requirements. Aggregator websites solve discovery in part but hold their content as articles rather than as data, and offer no application handling at all. What is absent is a system that holds funding opportunities as structured, machine-readable criteria, holds a student's circumstances once, and uses the two together to decide, explain and administer the fit between them.

That absence affects identifiable people. It affects students, and most sharply those who are first in their family to study, who attend schools or live in areas where no one is on hand to explain where to look, and who have the least capacity to absorb the cost of an application that was never viable. It affects funding organisations, whose administrative burden limits how many applicants they can fairly consider and therefore how widely they can advertise. It affects institutional financial aid offices, which mediate between the two without a common record to work from.

This project proposes Bursary-Bridge, a web-based system that addresses both sides of the gap. A student states their circumstances once. A funding organisation states its criteria once, as typed, structured data rather than as prose. The system then scores every opportunity against every student profile, ranks the results, and states for each criterion whether it was met, could not be evaluated, or was not met, so that a score is explained rather than merely asserted. Opportunities gathered from published sources are held in the same structured form alongside those entered by funders, each carrying the source it came from. On the organisation's side the system receives applications, scores and screens them against that organisation's own published criteria, and carries an applicant through review, shortlisting and selection with an audit record of each step.

The contribution of the project is therefore threefold. It contributes a data model that expresses education-funding eligibility as evaluable criteria rather than prose, and a transparent weighted matching algorithm over that model which reports its own reasoning. It contributes a working system that reduces the repeated manual re-entry of the same information across many application forms. And it contributes, to the funding organisation, a structured and auditable alternative to screening applications by hand in an email inbox.

2. Problem Statement

South African students who need funding beyond the state scheme must currently discover, assess and apply for bursaries through a process that is entirely manual and largely unstructured. Opportunities are published as pages of prose on aggregator sites, funder websites and notice boards, with eligibility rules written as sentences and closing dates buried in them. A student must find each page, read it, judge their own eligibility from the wording, and then complete a separate application form, re-entering the same identity, academic and household details every time. Funding organisations receive the resulting applications as email attachments in inconsistent formats and screen them by hand.

Evidence of how unstructured the published field is was obtained directly. An automated crawl of the thirteen category pages of the largest South African bursary aggregator, conducted for this project under the crawl delay its robots directives request, yielded 189 opportunities that could be resolved to a named funder, a closing date and a source page [3]. A substantial number of further listings were rejected by the same pipeline because those three facts could not be established from the published page without guessing at them. A student reading the same pages faces exactly that difficulty, without a pipeline to reject the ambiguous cases on their behalf.

Three causes underlie this. First, eligibility criteria exist only as prose, so nothing can evaluate them and every assessment is a human reading. Second, there is no shared record of a student's circumstances, so each funder asks for the same facts again on a form of its own design. Third, there is no structured intake on the funder's side, so applications arrive as documents rather than as records and must be transcribed before they can be compared.

The people affected are students seeking funding, the organisations that fund education, and the institutional financial aid offices that stand between them. The consequences differ by party. Students expend scarce time and money on applications for which they were never eligible, and miss opportunities they would have won, with the burden falling heaviest on first-generation and rural applicants who have no one to direct their search. Funders carry an administrative load that limits the number of applicants they can consider fairly, which in turn discourages them from advertising widely. Unsuccessful applicants are told nothing about why they were unsuccessful, so the same mistakes are repeated in the next cycle.

The gap is specific. No available system holds education-funding eligibility as structured, evaluable data, holds a student's circumstances once, matches the two automatically with a stated reason for every criterion, and then carries the resulting application through to a funder's selection decision. A new solution is needed because the components that exist solve only fragments of this: aggregators solve discovery without data or application handling, university systems manage enrolled students rather than external funding, and commercial applicant tracking systems model employment rather than academic eligibility. Closing this gap requires structured criteria, a single student profile and an explainable matching mechanism to be designed together, which is what this project undertakes.

3. Goal and Objectives

3.1. Goal

The goal of this project is to design, develop and evaluate a web-based system that automatically matches South African students to the education-funding opportunities for which they are eligible, explains the reason behind every match, and provides funding organisations with a single structured platform through which to publish funding programmes and administer the applications they receive.

3.2. Objectives

Table 1 states the measurable steps through which the goal will be achieved. Each objective names a deliverable that can be inspected, and the second column states the evidence by which its completion is judged. Together they span the project lifecycle from investigation to documentation.

Table 1 Objectives and the evidence that each has been met

No.	Objective	How completion is measured
1	Investigate how education-funding opportunities are currently published, discovered and applied for in South Africa, and how funding organisations currently receive and screen applications.	Completion is measured by a written review of the publishing channels and the existing systems adjacent to the problem, together with a crawl of a named published source that quantifies how much of the information is structured and how much is prose.
2	Analyse the findings of the investigation to derive the functional and non-functional requirements of the proposed system, and specify them in a Software Requirements Specification.	Completion is measured by an approved Software Requirements Specification containing use cases for every actor, each with a numbered identifier that later design and test documents can cite.
3	Design a data model that expresses funding eligibility as typed, machine-evaluable criteria rather than as free text, and that holds a student's identity, academic and household circumstances once.	Completion is measured by a documented schema in which every eligibility criterion is a typed column or related record, and by the absence of any free-text field that the matching mechanism must interpret.
4	Design a weighted matching algorithm that scores a student profile against a funding programme and returns, for each criterion, whether it was met, not met, or could not be evaluated.	Completion is measured by a specified algorithm with stated weights, a defined treatment of criteria that cannot be evaluated, and a score-with-reasons output structure.
5	Develop the student-facing components of the system: profile capture, a ranked list of matched opportunities with the reasons behind each score,	Completion is measured by working screens for each of these, operating against the real database rather than against sample data.

No.	Objective	How completion is measured
	a browsable directory of all opportunities held, and an application workflow.	
6	Develop the organisation-facing components of the system: funding programme publication with structured criteria, receipt of applications, applicant review against the organisation's own criteria, shortlisting and selection.	Completion is measured by working screens for each of these, with every query scoped so that one organisation cannot read another organisation's applicants.
7	Implement an ingestion pipeline that collects funding opportunities from named published sources, maps them onto the structured data model, and records the source of each.	Completion is measured by a populated directory in which every opportunity carries the source page it was taken from, and by a rejection record for every listing the pipeline declined to import.
8	Test the system at unit, integration and end-to-end level, and validate the accuracy of the matching and document-extraction components against prepared test data.	Completion is measured by an executable test suite, a documented Test Plan, and a recorded accuracy figure for the components that extract data from documents.
9	Evaluate the completed system against the requirements specified in objective two, and against the problem the project set out to address.	Completion is measured by a traceability check from each specified requirement to the implementation that satisfies it, and by an audit confirming that no user-visible content is invented rather than sourced.
10	Document the project in the Software Requirements Specification, the Software Design Description, the Test Plan and the final project report.	Completion is measured by the delivery of each document in the prescribed departmental template.

4. Methodology

Selected methodology

This project follows an Agile approach using the Scrum framework, as defined by Schwaber and Sutherland [4] and grounded in the principles of the Agile Manifesto [5]. Development proceeds in fixed-length sprints, each of which produces a working increment of the system; the requirements are held as a product backlog that is re-ordered at the start of each sprint; and the supervisor acts in the role of the product owner, reviewing each increment and directing what is built next.

Scrum was selected in preference to the alternatives for reasons that follow from the problem rather than from convention. The eligibility criteria that funders use are not known in advance and can only be established by examining what is actually published; the data model therefore has to change as that examination proceeds. Waterfall assumes a requirements set that is complete before design begins, which this project does not have, and its sequential phases would defer the discovery of a wrong assumption until integration. Rapid Application Development shares Agile's iterative character but emphasises rapid prototyping over the sustained quality practices this system needs, given that it processes personal and financial information. Design Science Research, in the form set out by Peffers and colleagues [6], is a sound fit for the artefact-building character of the project and its phases are reflected in the evaluation stage described below, but as a research methodology it governs the demonstration and evaluation of an artefact rather than the day-to-day organisation of a four-person

development team working to a fixed academic deadline. Scrum governs that, and the rigour Design Science Research asks for is retained in how the completed system is evaluated.

Phases of the project

The work is organised into the seven phases below. The phases are sequential in emphasis rather than in execution: sprints cut across them, so that design, development and testing of one part of the system proceed while another part is still being investigated.

1. Investigation and background research. The publishing channels for funding opportunities are examined, the systems adjacent to the problem are reviewed, and a named published source is crawled to establish how much of the information exists as data and how much as prose.
2. Requirements engineering. The findings are turned into functional and non-functional requirements and use cases, recorded in a Software Requirements Specification structured in accordance with ISO/IEC/IEEE 29148 [7].
3. System analysis and design. The data model, the architecture, the matching algorithm and the user interface are designed and recorded in a Software Design Description structured in accordance with IEEE 1016 [8].
4. Implementation. The system is built in sprints, each ending in a working increment reviewed by the supervisor.
5. Data ingestion. The pipeline that collects opportunities from published sources is built, run against a sample for review, and then run in full.
6. Testing and validation. Unit, integration and end-to-end tests are written and executed, and the accuracy of the extraction components is measured against prepared test data.
7. Evaluation and documentation. The completed system is evaluated against the specified requirements and the stated problem, and the project documentation is finalised.

Requirements gathering

Requirements are derived from documentary analysis rather than from survey or interview. The primary material is the published record itself: the eligibility conditions, closing dates and required supporting documents that funders state on their own pages, and the application forms they distribute. This material is authoritative, it is publicly available, and it is what any system in this space must in fact consume. The state scheme's published eligibility criteria [2] provide a documented reference point for the structure that means-tested criteria take. Requirements derived this way are recorded as use cases and reviewed with the supervisor at the sprint review that follows their capture.

Design approach

The system is designed as a layered web application. Presentation is separated from a service layer that holds the read and write logic, which in turn is separated from the pure domain modules that carry the matching and eligibility rules. Authorisation is established at the top of every server-side entry point by a guard that returns the identifiers used to scope every subsequent database query, so that an authorisation failure results in a missing record rather than in a leaked one. The domain modules are kept free of database access so that the matching and eligibility logic can be tested directly.

Techniques and algorithms

Four technical mechanisms carry most of the system's behaviour.

- **Weighted criterion matching.** A student profile is scored against a funding programme across a set of weighted criteria covering field of study, institution, academic performance, subject requirements, household circumstances and demographic conditions. A criterion that is met contributes its full weight; a criterion that cannot be evaluated, because the profile does not yet carry the fact it needs, contributes a defined partial weight and is reported as requiring verification rather than being silently treated as a failure. The contributions are summed and expressed as a percentage, and the per-criterion verdicts are returned with it so that the score can be explained.
- **Rule-based eligibility evaluation.** Eligibility is evaluated separately from the match score, against the typed criteria the funder published, and returns one of three verdicts: eligible, not eligible, or pending verification. Keeping eligibility separate from the score prevents a high score from implying a qualification the student does not hold.
- **Structured extraction from documents.** Application forms distributed as PDF documents are read, their field labels are normalised and matched against a canonical set of profile fields, and a confidence level is recorded for each value extracted. Where confidence is insufficient the field is left empty and flagged for the user rather than filled with a guess.
- **Identity resolution and duplicate detection.** Applications received outside the system are matched to existing records on verified identifiers, and suspected duplicates are flagged for a human decision rather than merged or deleted automatically.

Data collection

The system's own data is collected by an ingestion pipeline that fetches published listing pages from named sources, parses them, and maps each opportunity onto the structured model. Only sources explicitly agreed with the supervisor are fetched. Before any source is crawled its robots directives and terms of use are checked, and the crawl delay those directives request is observed; the principal source used in this project requests a delay of thirty seconds between requests, and the pipeline honours it [3].

Every imported opportunity records the page it was taken from. A listing that cannot be resolved to a funder, a closing date and a source page is recorded as rejected with the reason, and is not imported with assumed values.

Development process

Work is tracked as a product backlog of user stories derived from the use cases. Each sprint begins with sprint planning, in which items are selected into a sprint backlog, and ends with a sprint review at which the increment is demonstrated to the supervisor and a sprint retrospective at which the team adjusts its own practice. Version control is used throughout, with each change committed against the objective it serves. Database changes are made as versioned migrations so that any environment can be brought to the current schema reproducibly.

Testing and evaluation

Testing operates at four levels. Unit tests exercise the pure domain modules, in particular the matching and eligibility logic, against constructed profiles for which the correct verdict is known. Integration tests exercise the service layer against a real database to confirm that queries are correctly scoped and that one organisation cannot reach another's records. End-to-end tests drive the running application through a browser to confirm that complete journeys, such as registration through to submitted application, work as specified. Accuracy testing measures the document-extraction components against prepared forms whose correct values are known in advance, and reports the proportion of fields extracted correctly.

Evaluation is conducted against two references. The first is the Software Requirements Specification: each specified requirement is traced to the implementation that satisfies it, and any requirement not satisfied is recorded as such. The second is the problem statement: the system is examined for whether a student can in fact obtain a ranked, explained list of opportunities from a single profile, and whether a funding organisation can in fact screen and shortlist applicants against its own published criteria without manual transcription. An automated audit additionally confirms that no funding opportunity visible to a user is invented rather than traceable to a recorded source.

5. Equipment, Tools, and Technologies Required for the Project

Tables 2 and 3 list the hardware and the software the system is built on. No hardware beyond ordinary development machines is required, because the system is a web application with no sensor, device or embedded component.

Hardware

Table 2 Hardware required for the project

Item	Purpose in this project
------	-------------------------

Development workstations	Laptop or desktop computers, one per team member, used for development, testing and documentation.
Server or hosting environment	A machine capable of running the application process and the database, used for the deployed instance against which end-to-end testing and demonstration are carried out.
Mobile handsets and tablets	Used to verify that the interface is usable at narrow viewport widths, since many students will reach the system on a telephone.

Software and technologies

Table 3 Software and technologies required for the project

Category	Technology and the reason for it
Programming language	TypeScript 5.7, used for both server and browser code so that one set of types describes the data on both sides.
Application framework	Next.js 15.5 with the App Router, and React 19 with server components, providing the pages, the routing and the HTTP route handlers in one codebase.
Presentation	Tailwind CSS 3.4, with the colour palette and type scale expressed as design tokens so that a visual change is made once and applies everywhere.
Database	PostgreSQL 16, chosen for its typed columns, enumerated types and referential integrity, all of which the eligibility model depends on.
Data access	Prisma 6.19, providing a typed schema, a generated client and versioned migrations.
Validation	Zod 3.25, used to validate every request body on the server, with the same schemas reused in the browser for immediate feedback.
Document processing	pdf-lib for writing filled application forms, pdfjs-dist for reading submitted PDF documents, and JSZip for expanding archives of applications received in bulk.
Ingestion	Cheerio, used to parse the HTML of published listing pages during ingestion.
Authentication	bcryptjs for password hashing, with server-side sessions addressed by an opaque token held in an httpOnly cookie.
Testing	Playwright for end-to-end browser testing, together with purpose-written test harnesses executed under the TypeScript runtime for unit, integration and accuracy testing.
Version control	Git, with the repository hosted remotely so that all four team members work against one history.
Documentation	Microsoft Word, for the departmental document templates in which the specification, design and test documents are delivered.

6. Project Scope

The boundaries below were set so that the project is completable within the four months of a single semester by a team of four while still addressing both sides of the problem stated in Section 2.

In scope

- A student account with a profile holding identity, contact, academic, study-preference, subject-result and household-circumstance information, captured once and reused throughout the system.
- A weighted matching engine that scores every funding programme against a student profile and returns the per-criterion reasons behind the score.
- A rule-based eligibility evaluation that returns a separate verdict of eligible, not eligible, or pending verification against the criteria the funder published.
- A browsable and searchable directory of all funding opportunities held by the system, whether matched to the student or not, each showing the source it came from.
- An ingestion pipeline that imports opportunities from published sources agreed with the supervisor, records the provenance of each, and rejects rather than guesses at listings it cannot resolve.
- An application workflow in which a student applies to a programme using the profile already captured, attaches supporting documents, and tracks the status of each application.
- Assisted completion of PDF application forms from the stored profile, with a confidence level recorded for every extracted field and no field filled on a low-confidence guess.
- A composer that assembles a motivational letter from facts already held on the profile and the student's own answers, without generating claims the student did not make.
- An organisation account through which funding programmes are published with structured eligibility criteria, and through which applications are received, reviewed, shortlisted and selected.
- Bulk intake of applications that an organisation received outside the system, including extraction, duplicate detection and a review step before anything is imported.
- An administrative role for managing the catalogue of institutions, qualifications and subjects, and for suspending or restoring accounts.
- In-application notifications recording the events that concern a user.
- A responsive interface usable at desktop, tablet and mobile widths.

Out of scope

- Disbursement of funds. The system records selection decisions; it does not make, hold or transfer payments.

- Verification of submitted documents against any external authority. The system records what was submitted and flags what needs checking, but it does not confirm a certificate or an identity document with its issuer.
- Integration with the student information systems of universities or with government systems, including the state funding scheme. No such interface is specified or built.
- A native mobile application. The interface is delivered as a responsive web application only.
- Automated generation of prose by a language model. The motivational letter composer assembles text deterministically from stated facts; it does not write on the student's behalf.
- Payment processing, subscription billing or any commercial function.
- Languages other than English in the user interface.
- Exhaustive coverage of every published bursary in South Africa. Only sources explicitly agreed with the supervisor are ingested.

7. Project Beneficiaries

The project delivers value to two groups directly, through their own use of the system, and to three further groups indirectly, through the effect the system has on the process they take part in. Table 4 sets out the direct beneficiaries and Table 5 the indirect ones, in each case naming how the group meets the system and what it gains.

Direct beneficiaries

Table 4 Direct beneficiaries of the project

Beneficiary	How they interact with the system	Benefit
Students seeking education funding	They register, complete one profile, and receive a ranked list of opportunities with the reasons behind each score. They browse the full directory, apply through the system using the profile already captured, attach supporting documents once, and track each application.	Search is replaced by matching, so effort is directed at opportunities they are actually eligible for. The reasons attached to each score show what would improve a match, which is information the current process never returns. The same details are entered once rather than on every form, and the status of every application is visible in one place.
Funding organisations	They register an organisation, publish funding programmes with structured eligibility criteria, receive applications into a single pipeline, review applicants against their own criteria, and shortlist and select from that pipeline. Applications received outside the system can be imported in bulk into the same pipeline.	Screening becomes structured rather than manual, so a larger field of applicants can be considered consistently and the organisation is no longer penalised for advertising widely. Every decision leaves an auditable record, which makes reporting on the organisation's own selection possible for the first time.

Indirect beneficiaries

Table 5 Indirect beneficiaries of the project

Beneficiary	Relationship to the system	Benefit
Institutional financial aid offices	They are not users of the system in this release, but they advise the students who are.	A student who arrives with a ranked and explained list of opportunities, rather than with a general question about where to look, can be advised in less time and more precisely.
Families and communities of funded students	They do not interact with the system.	A student who secures funding they would otherwise have missed is a student who registers, remains registered and completes a qualification. The benefit of that outcome extends beyond the individual.
The Department of Computer Science and future project teams	The project produces a specification, a design description, a test plan and a working system, all in the departmental templates.	The data model for expressing funding eligibility as evaluable criteria, and the explainable matching mechanism built over it, are reusable contributions that a later project can extend to further sources or further funding types.

8. Project Assumptions and Constraints

Assumptions

- Funding opportunities published on the sources agreed with the supervisor remain publicly accessible for the duration of the project, and their page structure does not change so fundamentally that the ingestion pipeline must be rewritten.
- The eligibility conditions that funders publish can be expressed as typed criteria. Conditions that genuinely cannot be typed are handled as requiring verification rather than being forced into the model.
- Students are able to supply the profile information the matching engine needs, and will supply it where the system explains why it is being asked.
- Team members have continuous access to development machines and to a reliable internet connection for the duration of the project.
- The software on which the system is built remains freely available under its current open-source licensing for the duration of the project.
- The supervisor is available at the agreed intervals to review each increment and to approve the sources that may be ingested.
- A database server and a hosting environment sufficient to run the application for demonstration and testing are available to the team.

Constraints

- Time. The project must be completed within a single semester of four months, alongside the team's other coursework. This is the binding constraint on the project: it fixes the total effort available, it is why the work plan overlaps its phases rather than running them in sequence, and it makes scope control the principal planning discipline.
- Team size. The team consists of four members, which limits the amount of work that can proceed in parallel in any sprint.
- Budget. The project has no funding, so every tool and service used must be free or open-source. This excludes commercial hosting tiers, paid application programming interfaces and licensed libraries.
- No access to real funder data. No funding organisation has provided the project with its application records, so organisation-facing features are developed and demonstrated against data created for that purpose, which is clearly identified as such wherever it appears.
- No model-backed text generation. No language-model service is available within the project's means, so any feature that would otherwise generate prose is implemented deterministically from stated facts.
- No electronic mail delivery service. Notifications are written to the database and shown in the application; outbound email is designed for but not dispatched.
- Legal constraints on data. The system processes personal information and is therefore bound by the Protection of Personal Information Act [9], which constrains what may be collected, how long it may be kept and who may see it.
- Constraints on data collection. Only sources whose robots directives and terms of use permit automated access may be ingested, and only at the request rate those directives specify, which limits how quickly the directory can be populated.

9. Project Risks

Table 6 sets out the risks that could prevent the project from reaching the objectives stated in Section 3.2. Likelihood and impact are the team's own assessment; each risk carries the mitigation already planned for it rather than a general intention to be careful.

Table 6 Project risks, their assessment and their mitigation

Risk	Likelihood	Impact	Mitigation
Exposure of students' personal	Medium	High	Authorisation is established at every server-side

Risk	Likelihood	Impact	Mitigation
information. The system holds identity numbers, contact details, academic records and household income, which are exactly the categories the Protection of Personal Information Act protects [9].			entry point by a guard that returns the identifiers used to scope every database query, so a record belonging to another user cannot be reached. Passwords are hashed rather than stored. Sessions are addressed by an opaque token in an httpOnly cookie. Integration tests specifically assert that one organisation cannot read another organisation's applicants.
A published source changes its page structure, or becomes unavailable, and the ingestion pipeline stops producing correct results.	High	Medium	The pipeline records the source page for every opportunity and rejects any listing it cannot fully resolve, so a structural change produces rejections rather than corrupted records. A sample is reviewed before every full ingestion run. Previously imported opportunities are retained with their recorded provenance.
Eligibility criteria published as prose cannot be expressed in the typed data model, so opportunities are imported with the wrong criteria attached.	High	High	Criteria that cannot be established are left unset and reported as requiring verification rather than being assumed. The matching engine treats an unevaluable criterion explicitly, and the interface labels it as such, so an uncertain criterion never silently becomes a pass or a fail.
Scope creep. The problem admits many additional features, and adding them during development would put the core deliverables at risk.	High	High	Section 6 states what is out of scope explicitly. New ideas enter the product backlog rather than the current sprint, and are admitted only at sprint planning against the objectives in Section 3.2.
Integration failures between components developed in parallel by different team members.	Medium	Medium	All four members work against one version-controlled repository with a shared type definition generated from the database schema, so an incompatible change fails to compile rather than failing at run time. Each sprint ends in an integrated working increment.
Insufficient test data. The document-extraction and bulk-intake components cannot be validated without realistic application forms, and no funder has supplied any.	Medium	Medium	Test forms are prepared by the team with known correct values so that extraction accuracy can be measured objectively. Data created for testing is clearly identified as such and is removed from the database before delivery.
Loss of work through hardware failure or an unrecoverable local error.	Medium	High	All code and documentation are committed to a remote repository, and database schema changes are held as versioned migrations so that any environment can be rebuilt from the repository alone.
Development delay caused by the team's concurrent academic commitments.	High	Medium	The work plan in Section 10.2 schedules the highest-risk work early. Sprint retrospectives surface slippage while there is still time to re-order the backlog, and out-of-scope items are not started.
A team member becomes unavailable for an extended period.	Low	High	Responsibilities in Section 10.1 are allocated so that no single member is the only person who has worked in a given area, and all work is visible in the shared repository rather than held locally.
The matching results are technically correct but not useful to students, because the weights chosen do not reflect	Medium	Medium	Weights are held as configuration rather than embedded in code, and can be overridden per funding programme. Every score is returned with its per-criterion reasons, so a weighting

Risk	Likelihood	Impact	Mitigation
what matters in practice.			that produces an unexpected result can be identified and corrected rather than merely observed.

10. Project Work Plan

10.1. Expertise of the Team members

The team consists of the four members listed on the cover page. Responsibilities are allocated below by area of the system, so that every area has a member accountable for it and no area is the work of one person alone. The level of knowledge recorded for each member is stated in bracketed form because the project has no record from which prior experience could be established, and it is not assumed here.

Table 7 allocates each area of the system to the member accountable for it.

Table 7 Team members, their responsibilities and their level of knowledge

Member	Responsibilities	Level of knowledge
CPP Mchunu	Requirements engineering and the Software Requirements Specification; the student-facing profile, matching and opportunity screens; coordination of the sprint reviews with the supervisor.	High.
A Nsibande	Data model and database migrations; the matching and eligibility domain modules; the ingestion pipeline and the provenance of imported opportunities.	High.
KN Bikile	The organisation-facing portal: funding programme publication, applicant review, shortlisting and selection; bulk application intake and duplicate detection.	High.
SP Tshazi	Document processing, including extraction from submitted PDF forms and assisted form completion; the test suites at unit, integration and end-to-end level; the Test Plan.	High.

The team leader is Asanda Nsibande, listed on the cover page as A Nsibande. The team leader is responsible for the product backlog, for scheduling and chairing the sprint events, and for the delivery of each document to the supervisor. All four members contribute to design decisions, to code review and to the final project report; the allocation above records accountability for an area, not exclusive ownership of it.

10.2. Work plan

The project runs over a single semester of four months. The plan below organises it into ten tasks across the sixteen weeks of that semester; weeks rather than months are used because at four columns a chart cannot show which tasks overlap, and the overlap is the point. The semester runs from July 2026 to

October 2026: week 1 is the first week of July and week 16 falls in October, the month in which the project is presented.

Tasks overlap deliberately. Development begins before the whole specification is settled and testing begins before development ends, which is what the chosen methodology requires and what four months make unavoidable. The three tasks that carry the most risk, the background investigation, the requirements and the data ingestion, are scheduled early, so that a problem found in any of them is found while there is still time to act on it.

Table 8 lists the tasks and the weeks each occupies, and Figure 1 presents the same plan as a Gantt chart so that the overlap between tasks can be seen at a glance.

Table 8 Project work plan by task and week

Task	Weeks	Description
Project initiation and proposal	1–3	Topic definition, supervisor consultation, and this proposal.
Background research and investigation	1–5	Review of the publishing channels and of adjacent systems; the exploratory crawl of an agreed source.
Requirements gathering and specification	3–7	Use cases and requirements; delivery of the Software Requirements Specification.
System analysis and design	5–8	Data model, architecture, matching algorithm and interface design; delivery of the Software Design Description.
Implementation: student portal	6–13	Profile, matching, directory, applications and documents.
Data ingestion and provenance	8–11	The ingestion pipeline; sample review, then the full run.
Implementation: organisation portal	8–14	Programme publication, applicant review, shortlisting, selection and bulk intake.
Testing and validation	10–16	Unit, integration, end-to-end and accuracy testing; delivery of the Test Plan.
Evaluation	14–16	Traceability of requirements to implementation; audit of the data visible to users.
Documentation and final presentation	1–16	Maintained throughout; final report and demonstration in the closing week.

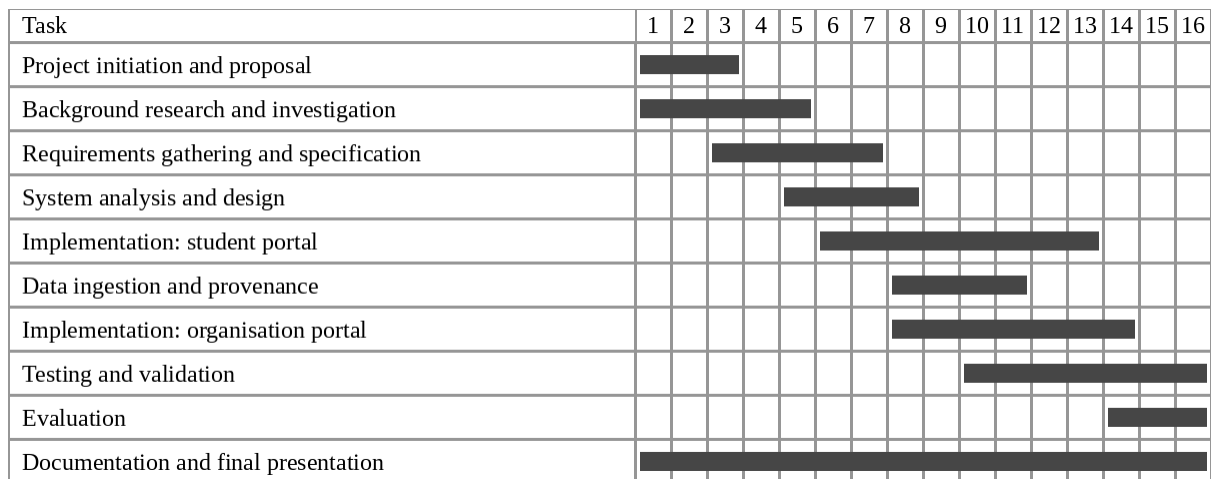


Figure 1 Gantt chart of the project work plan, weeks 1 to 16 of the semester

11. References

References are numbered in the order of their first citation in the text and are cited by bracketed number throughout.

- [1] Department of Higher Education and Training, Statistics on Post-School Education and Training in South Africa: 2023. Pretoria: Department of Higher Education and Training, released June 2025. ISBN 978-1-77997-960-5. [Online]. Available: <https://www.dhet.gov.za/Information%20Systems%20Management/Statistics%20on%20Post-School%20Education%20and%20Training%20in%20South%20Africa,%202023.pdf>. [Accessed: 24 September 2026].
- [2] National Student Financial Aid Scheme, “The DHET Bursary Scheme,” NSFAS. [Online]. Available: <https://www.nsfas.org.za/content/bursary-scheme.html>. [Accessed: 24 September 2026].
- [3] ZA Bursaries, “Bursaries in South Africa.” [Online]. Available: <https://www.zabursaries.co.za/>. Robots directives available at <https://www.zabursaries.co.za/robots.txt>. [Accessed: 24 September 2026].
- [4] K. Schwaber and J. Sutherland, The 2020 Scrum Guide. November 2020. [Online]. Available: <https://scrumguides.org/scrum-guide.html>. [Accessed: 24 September 2026].
- [5] K. Beck et al., “Manifesto for Agile Software Development,” 2001. [Online]. Available: <https://agilemanifesto.org/>. [Accessed: 24 September 2026].
- [6] K. Peffers, T. Tuunanen, M. A. Rothenberger and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” Journal of Management Information Systems, vol. 24, no. 3, pp. 45–77, 2007. doi: 10.2753/MIS0742-1222240302.

- [7] ISO/IEC/IEEE 29148:2018, Systems and Software Engineering — Life Cycle Processes — Requirements Engineering. Geneva: International Organization for Standardization, 2018.
- [8] IEEE Std 1016-2009, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions. New York: Institute of Electrical and Electronics Engineers, 2009.
- [9] Republic of South Africa, Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer, 2013. [Online]. Available: <https://www.justice.gov.za/legislation/acts/2013-004.pdf>. [Accessed: 24 September 2026].

12. Appendices (if applicable)

Two appendices are attached. Minutes of supervision meetings and questionnaire instruments are not reproduced here: the project gathers its requirements from documentary analysis of published material rather than by survey or interview, as stated in Section 4, so no questionnaire was administered.

Appendix A: Funding sources the ingestion pipeline is configured to read

The listing pages below are the sources the ingestion pipeline is configured to read. Each was checked for its robots directives and terms of use before any request was made, and each is agreed with the supervisor. No source outside this list is fetched.

- <https://www.zabursaries.co.za/bursary-news/>
- <https://www.zabursaries.co.za/accounting-bursaries-south-africa/>
- <https://www.zabursaries.co.za/commerce-bursaries-south-africa/>
- <https://www.zabursaries.co.za/computer-science-it-bursaries-south-africa/>
- <https://www.zabursaries.co.za/construction-and-built-environment-bursaries-south-africa/>
- <https://www.zabursaries.co.za/education-bursaries-south-africa/>
- <https://www.zabursaries.co.za/engineering-bursaries-south-africa/>
- <https://www.zabursaries.co.za/general-bursaries-south-africa/>
- <https://www.zabursaries.co.za/government-bursaries-south-africa/>
- <https://www.zabursaries.co.za/international-scholarships-bursaries-south-africa/>
- <https://www.zabursaries.co.za/law-bursaries-south-africa/>
- <https://www.zabursaries.co.za/medical-bursaries-south-africa/>
- <https://www.zabursaries.co.za/music-and-performing-arts-bursaries-south-africa/>

- <https://www.zabursaries.co.za/science-bursaries-south-africa/>

Appendix B: Project documents

The documents below are produced by the project in the departmental templates, and are the deliverables against which objective ten in Section 3.2 is measured.

Table 9 lists the documents the project produces and the structure each follows.

Table 9 Project documents and the structure each follows

Document	Structure it follows	Status
Final Year Project Proposal	Departmental proposal template	This document.
Software Requirements Specification	Departmental template; structured in accordance with ISO/IEC/IEEE 29148	Delivered.
Software Design Description	Departmental template; structured in accordance with IEEE 1016	Delivered.
Test Plan	Departmental template	Delivered.
Final project report and demonstration	Departmental requirements	Scheduled for the closing week of the work plan.



**UNIVERSITY OF
ZULULAND**

A NODE FOR AFRICAN THOUGHT

Department of Computer Science

Software Requirements Specification

Bursary-Bridge
A Bursary and Scholarship Matching Platform

Supervisor
Ms Zulu

Submitted by

Student Name	Student Number
1. CPP Mchunu	202242486
2. A Nsibande	240079346
3. KN Bikile	202356797
4. SP Tshazi	240103222
5.	
6.	

September 2026

Table of Contents

1.0. Introduction.....	1
1.1. Purpose.....	1
1.2. Scope of Project.....	1
1.3. Glossary.....	2
1.4. References.....	4
1.5. Overview of Document.....	4
2.0. Overall Description.....	6
2.1 System Environment.....	6
2.2 Functional Requirements Specification.....	7
2.2.1 Common Use Cases.....	7
Use case: Log In.....	8
Use case: Log Out.....	9
Use case: Manage Account Settings.....	9
Use case: View Notifications.....	10
2.2.2 Student Use Cases.....	10
Use case: Register Student Account.....	13
Use case: Verify Email Address.....	14
Use case: Complete Student Profile.....	14
Use case: Manage Study Preferences.....	15
Use case: View Matched Opportunities.....	16
Use case: Search and Filter Opportunities.....	17
Use case: View Opportunity Details.....	17
Use case: Apply for Funding.....	18
Use case: Track Application.....	19
Use case: Manage Documents.....	19
Use case: Record Subject Results.....	20
Use case: Browse All Bursaries.....	21
Use case: Generate Motivational Letter.....	21
Use case: Auto-Fill Application Forms.....	22
Use case: Respond to Information Request.....	22
2.2.3 Corporate User Use Cases.....	23
Use case: Register Corporate Account.....	26
Use case: Complete Organisation Profile.....	27
Use case: Create Funding Programme.....	27
Use case: Define Eligibility Criteria.....	28
Use case: Publish Funding Programme.....	29
Use case: Import Applications.....	30
Use case: Review Imported Applications.....	30
Use case: View Applications.....	31
Use case: Review Applicant.....	32
Use case: Shortlist Applicant.....	32
Use case: Request Information from Applicant.....	33
Use case: Select Beneficiary.....	33
Use case: View Reports.....	34
2.2.4 Administrator Use Cases.....	35
Use case: Manage Institution and Course Catalogue.....	35
Use case: Manage User Accounts.....	36
Use case: Moderate Organisations and Programmes.....	37
Use case: Manage Bursary Data Sources.....	37
Use case: View Audit Trail.....	38
2.2.5 Complete Use Case Diagram.....	38
2.3 User Characteristics.....	41
2.4 Non-Functional Requirements.....	41
3.0. Requirements Specification.....	43

3.1 External Interface Requirements.....	43
3.2 Functional Requirements.....	43
3.2.1 Log In.....	44
3.2.2 Log Out.....	44
3.2.3 Manage Account Settings.....	44
3.2.4 View Notifications.....	45
3.2.5 Register Student Account.....	45
3.2.6 Verify Email Address.....	46
3.2.7 Complete Student Profile.....	47
3.2.8 Manage Study Preferences.....	48
3.2.9 View Matched Opportunities.....	48
3.2.10 Search and Filter Opportunities.....	49
3.2.11 View Opportunity Details.....	49
3.2.12 Apply for Funding.....	50
3.2.13 Track Application.....	51
3.2.14 Manage Documents.....	51
3.2.15 Record Subject Results.....	52
3.2.16 Browse All Bursaries.....	53
3.2.17 Generate Motivational Letter.....	53
3.2.18 Auto-Fill Application Forms.....	54
3.2.19 Respond to Information Request.....	55
3.2.20 Register Corporate Account.....	56
3.2.21 Complete Organisation Profile.....	56
3.2.22 Create Funding Programme.....	57
3.2.23 Define Eligibility Criteria.....	58
3.2.24 Publish Funding Programme.....	58
3.2.25 Import Applications.....	59
3.2.26 Review Imported Applications.....	60
3.2.27 View Applications.....	61
3.2.28 Review Applicant.....	61
3.2.29 Shortlist Applicant.....	62
3.2.30 Request Information from Applicant.....	62
3.2.31 Select Beneficiary.....	63
3.2.32 View Reports.....	63
3.2.33 Manage Institution and Course Catalogue.....	64
3.2.34 Manage User Accounts.....	64
3.2.35 Moderate Organisations and Programmes.....	65
3.2.36 Manage Bursary Data Sources.....	66
3.2.37 View Audit Trail.....	66
3.3 Detailed Non-Functional Requirements.....	67
3.3.1 Logical Structure of the Data.....	67
3.3.2 Security.....	73

List of Figures

Figure 1 - System Environment.....	6
Figure 2 - Application Status Lifecycle.....	7
Figure 3 - Common Use Cases.....	8
Figure 4 - Log In Use Case.....	8
Figure 5 - Log Out Use Case.....	9
Figure 6 - Manage Account Settings Use Case.....	9
Figure 7 - View Notifications Use Case.....	10
Figure 8 - Student Use Cases (1 of 3).....	11
Figure 9 - Student Use Cases (2 of 3).....	12
Figure 10 - Student Use Cases (3 of 3).....	13
Figure 11 - Register Student Account Use Case.....	13
Figure 12 - Verify Email Address Use Case.....	14
Figure 13 - Complete Student Profile Use Case.....	15
Figure 14 - Manage Study Preferences Use Case.....	15
Figure 15 - View Matched Opportunities Use Case.....	16
Figure 16 - Search and Filter Opportunities Use Case.....	17
Figure 17 - View Opportunity Details Use Case.....	17
Figure 18 - Apply for Funding Use Case.....	18
Figure 19 - Track Application Use Case.....	19
Figure 20 - Manage Documents Use Case.....	19
Figure 21 - Record Subject Results Use Case.....	20
Figure 22 - Browse All Bursaries Use Case.....	21
Figure 23 - Generate Motivational Letter Use Case.....	21
Figure 24 - Auto-Fill Application Forms Use Case.....	22
Figure 25 - Respond to Information Request Use Case.....	23
Figure 26 - Corporate User Use Cases (1 of 3).....	24
Figure 27 - Corporate User Use Cases (2 of 3).....	25
Figure 28 - Corporate User Use Cases (3 of 3).....	26
Figure 29 - Register Corporate Account Use Case.....	26
Figure 30 - Complete Organisation Profile Use Case.....	27
Figure 31 - Create Funding Programme Use Case.....	28
Figure 32 - Define Eligibility Criteria Use Case.....	28
Figure 33 - Publish Funding Programme Use Case.....	29
Figure 34 - Import Applications Use Case.....	30
Figure 35 - Review Imported Applications Use Case.....	30
Figure 36 - View Applications Use Case.....	31
Figure 37 - Review Applicant Use Case.....	32
Figure 38 - Shortlist Applicant Use Case.....	32
Figure 39 - Request Information from Applicant Use Case.....	33
Figure 40 - Select Beneficiary Use Case.....	34
Figure 41 - View Reports Use Case.....	34
Figure 42 - Administrator Use Cases.....	35
Figure 43 - Manage Institution and Course Catalogue Use Case.....	36
Figure 44 - Manage User Accounts Use Case.....	36
Figure 45 - Moderate Organisations and Programmes Use Case.....	37
Figure 46 - Manage Bursary Data Sources Use Case.....	37
Figure 47 - View Audit Trail Use Case.....	38
Figure 48 - Complete Use Case Diagram.....	40
Figure 49 - Logical Structure of the Bursary-Bridge Data.....	67

List of Tables

Table 1 - Glossary of Terms.....	2
Table 2 - User Data Entity.....	67
Table 3 - Student Profile Data Entity.....	68
Table 4 - Study Preference Data Entity.....	68
Table 5 - Organisation Data Entity.....	68
Table 6 - Funding Programme Data Entity.....	69
Table 7 - Eligibility Rule Data Entity.....	69
Table 8 - Application Data Entity.....	69
Table 9 - Subject Result Data Entity.....	70
Table 10 - Motivational Letter Data Entity.....	70
Table 11 - Information Request Data Entity.....	70
Table 12 - Application Import Batch Data Entity.....	71
Table 13 - Application Import File Data Entity.....	71
Table 14 - External Applicant Data Entity.....	72
Table 15 - Opportunity Source Data Entity.....	72
Table 16 - Shortlist Data Entity.....	72
Table 17 - Document Data Entity.....	72

1.0. Introduction

1.1. Purpose

The purpose of this document is to present a detailed description of Bursary-Bridge. It explains the purpose and the features of the system, what the system does, the constraints under which it must operate and how it reacts to the people who use it. This document is intended for the stakeholders who commissioned the system and for the developers who will build it.

1.2. Scope of Project

Bursary-Bridge is a web application that connects South African students with bursaries, scholarships and other education funding, and gives the organisations that fund education a place to publish those opportunities and administer the applications they attract.

Students who need funding face two related problems. Opportunities are scattered across many websites and are hard to find, and each one asks for the same information in a different form. Organisations that offer funding face the mirror of those problems. They receive large numbers of applications, many from students who do not meet the criteria, and they screen them by hand using email and spreadsheets.

Bursary-Bridge addresses both. A student builds one profile, which records what they want to study and where, what results they hold and what funding they need. The system compares that profile against every published funding programme and presents the opportunities that fit, each with a match score and, importantly, the reasons behind that score. The student applies using the profile they have already built rather than entering the same information again.

An organisation states its criteria once, as structured data rather than as prose. The same criteria then drive the matches shown to students and the eligibility verdict shown to the reviewer, so the two views cannot disagree. The organisation receives applications that are already scored and assessed, and moves applicants through review, shortlisting and selection. The system serves three kinds of user: the Student, the Corporate User and the Administrator. The Administrator maintains the catalogue of institutions, qualifications and subjects that the other two select from, and suspends or restores accounts; the Administrator does not take part in matching or in selection. The system does not attempt to disburse funds, to verify

documents against any external authority, or to integrate with the administration systems of universities or of government. Those lie outside the scope of this release.

1.3. Glossary

Table 1 defines the terms used throughout this specification. Each term carries the same meaning wherever it appears, in this document and in the Software Design Description and Test Plan that accompany it.

Table 1 - Glossary of Terms

Term	Definition
Administrator	A member of the Bursary-Bridge operating team who maintains the catalogue, the data sources and the accounts on the platform. An Administrator does not apply for funding and does not decide who receives it.
All Bursaries	The directory of every published Funding Programme held by the system, presented without regard to any Student's profile. Distinct from My Matches.
Applicant	A Student who has submitted an application to a Funding Programme, or an applicant whose application an Organisation imported.
Application	A Student's formal request for funding under a specific Funding Programme. An application carries a status, the profile the Student applied with, answers to programme questions and any attached documents.
Auto-Fill	The transfer of information from one application form a Student has already completed onto other, blank forms.
Beneficiary	An Applicant who has been selected to receive funding under a Funding Programme.
Corporate User	A representative of an Organisation who administers that Organisation's Funding Programmes on the system. Also referred to as the Funder.
Database	The collection of all information monitored by this system.
Eligibility Rule	The machine-readable set of conditions attached to a Funding Programme, covering supported institutions and courses, qualification level, minimum academic average, year of study, citizenship, household income and province.
Eligibility Outcome	The verdict returned when an Applicant is evaluated against a programme's Eligibility Rule: Eligible, Not Eligible, or Pending Verification.
Funding Programme	A bursary, scholarship or grant published by an Organisation, to which Students may apply.
Import Batch	One upload of application forms received outside Bursary-Bridge, processed together against a single Funding

	Programme.
Information Request	A Funder's request to an Applicant for specific documents or information in support of a submitted application.
Institution	A recognised South African university, university of technology or TVET college held in the standardised catalogue.
Match Score	A percentage from 0 to 100 expressing how closely a Student's profile fits a Funding Programme, produced by the Matching Engine.
Matching Engine	The component that scores a Student profile against a Funding Programme across six weighted criteria and returns both the score and the reasons behind it.
Motivational Letter	A letter a Student produces for a Funding Programme, composed from facts on their profile and sentences they wrote themselves.
My Matches	The Funding Programmes the Matching Engine has scored against a particular Student's profile. Distinct from All Bursaries.
Organisation	A corporation, foundation, NGO or other body that funds education through the system.
Programme	A standardised course of study, such as Computer Science, held in the catalogue. Distinct from a Funding Programme.
Profile Strength	A percentage indicating how complete a Student's profile is, used to prompt the Student to supply information that would improve their matches.
Provenance	The record of where an externally collected Funding Programme was read from: the address of the page, the name of the source, and the date it was last confirmed.
Shortlist	The set of Applicants an Organisation is actively considering for a Funding Programme.
Software Requirements Specification	A document that completely describes all of the functions of a proposed system and the constraints under which it must operate.
Stakeholder	Any person with an interest in the project who is not a developer.
Student	A learner seeking funding for tertiary study, who holds a profile on the system.
Subject Result	A subject or module a Student is taking, with the year and, where the Student knows it, the mark.
Study Preference	A course paired with the institution at which the Student intends to study it. A Student may hold a maximum of six.
User	A Student, a Corporate User or an Administrator.

1.4. References

The sources below are cited by bracketed number throughout this document. Each is cited at least once in the text, and every bracketed number in the text appears here.

- [1] IEEE, IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications. Los Alamitos, CA: IEEE Computer Society, 1998.
- [2] Object Management Group, OMG Unified Modeling Language (OMG UML), Version 2.5.1. Needham, MA: Object Management Group, 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1/>.
- [3] Republic of South Africa, Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer, 2013. [Online]. Available: <https://www.justice.gov.za/legislation/acts/2013-004.pdf>.

1.5. Overview of Document

The next chapter, the Overall Description section, gives an overview of the functionality of the product. It describes the environment the system sits in, the actors who use it and the use cases each actor performs, in language intended for the stakeholders who commissioned it.

The third chapter, the Requirements Specification section, is written primarily for the developers. It describes each use case formally, with its trigger, its preconditions, the basic path through it, the alternative and exception paths and its postconditions, together with the logical structure of the data and the security requirements.

Both chapters describe the same software product in its entirety, but are intended for different audiences and so use different language. The division of the document between them, and the content of each section, follow the recommended practice for a software requirements specification set out in [1].

Four conventions are used throughout. Figures are numbered consecutively and listed in the List of Figures; every use case is accompanied by its own numbered diagram. Tables that present data in their own right are numbered and listed in the List of Tables; the formal requirement tables of Section 3.2 are not separately numbered, because each sits directly beneath the numbered heading of the use case it describes. Sources are cited by bracketed number against the list in Section 1.4. Cross references name the section, figure or table by its number rather than by its position on the page.

2.0. Overall Description

2.1 System Environment

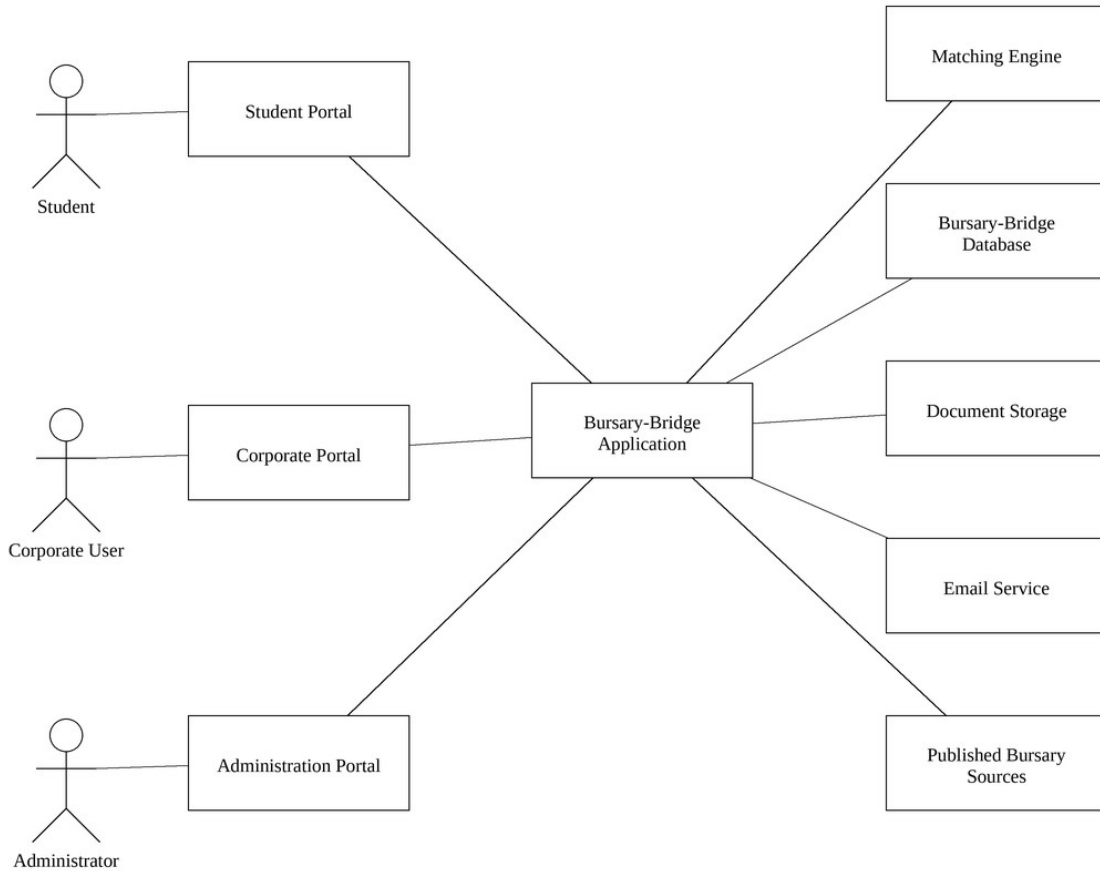


Figure 1 - System Environment

Figure 1 shows the environment in which Bursary-Bridge operates. The system has three active actors, the Student, the Corporate User and the Administrator, each reaching it through a portal of their own, and five cooperating components: the Matching Engine, the database, document storage, an email service and the published bursary sources from which the directory is collected.

The Student and the Corporate User both reach the system through a web browser over the Internet. Each is presented with a different portal, but both are served by one application and one database, so a funding programme published by an organisation is visible to a matching student immediately.

The Matching Engine is a component of the application rather than a separate system. It is drawn separately here because it is the part of the system that gives Bursary-Bridge its

purpose: it compares a student profile against a funding programme and produces both a score and the reasons for that score.

Document Storage holds the files students upload. It is drawn separately because those files are held outside the web root and are released only through an authorised request, and because in a production deployment it would be an object store rather than part of the application server.

The Email Service is the one cooperating system. It delivers address verification messages and the notifications raised when an application changes status. The system does not depend on it for any decision; if a message cannot be delivered the user can still complete every journey through the interface.

2.2 Functional Requirements Specification

This section sets out the use cases for each actor. Use cases common to both actors are given first, followed by those belonging to the Student and then those belonging to the Corporate User. Each use case is presented with a diagram, a brief description, the steps that make it up and a cross reference to its formal description in Section 3.2.

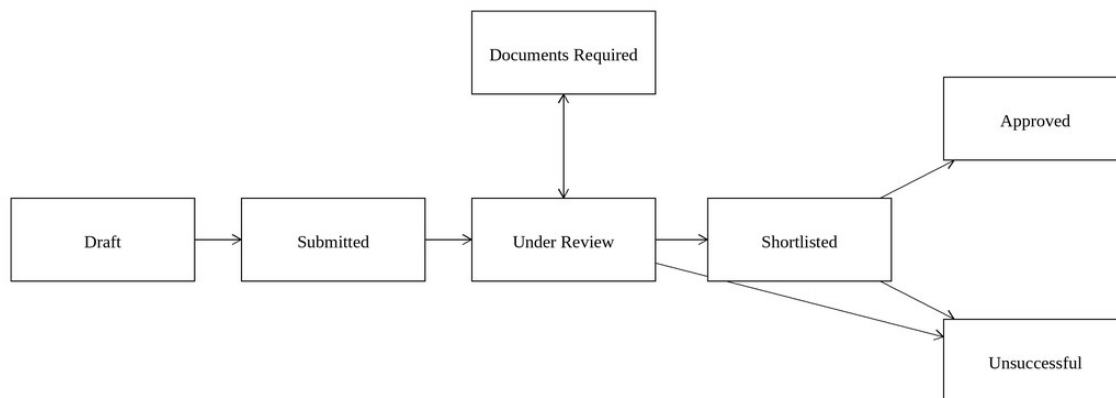


Figure 2 - Application Status Lifecycle

Figure 2 summarises the use cases that follow. A Student prepares an application as a draft and submits it. The Corporate User reviews it, may ask for further information, and either shortlists the applicant or records the application as unsuccessful. A shortlisted applicant is either selected as a beneficiary or, ultimately, unsuccessful.

2.2.1 Common Use Cases

Both kinds of user share a small number of use cases. They are described once here and are not repeated under each actor.

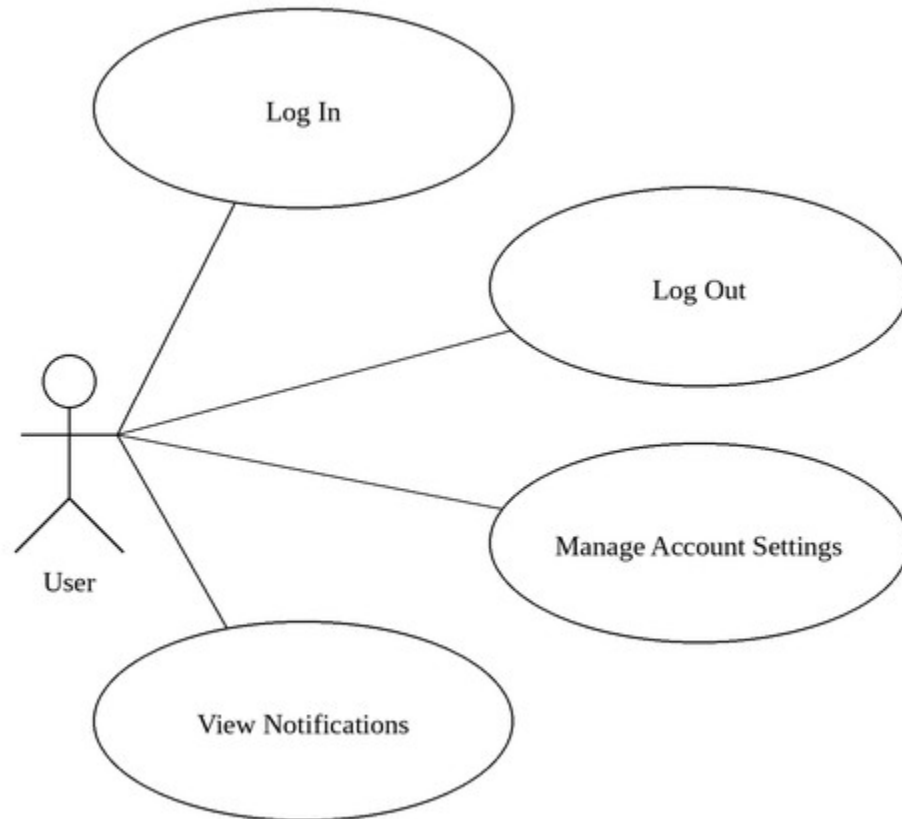


Figure 3 - Common Use Cases

Use case: **Log In**

Diagram:

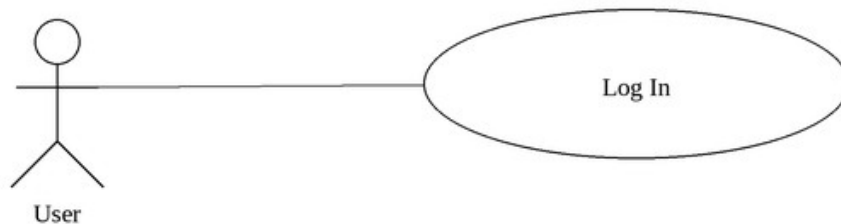


Figure 4 - Log In Use Case

Brief Description

A registered User authenticates and is taken to the part of the system that matches their role.

Initial Step-By-Step Description

Before this use case can be initiated, the User has already registered an account and has accessed the Bursary-Bridge login page.

1. The User enters their email address and password.
2. The System verifies the credentials against the stored password hash.

3. The System creates a session and issues a session cookie.
4. The System directs a Student to the Student Dashboard and a Corporate User to the Corporate Dashboard.
5. If the User has not finished their profile, the System returns them to the step at which they stopped.

Xref: Section 3.2.1, Log In

Use case: **Log Out**

Diagram:

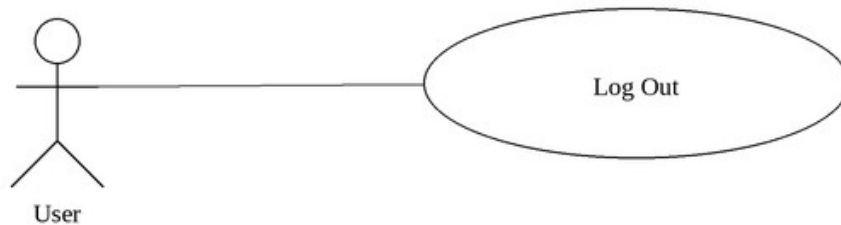


Figure 5 - Log Out Use Case

Brief Description

A User ends their session.

Initial Step-By-Step Description

Before this use case can be initiated, the User is signed in.

1. The User selects Log out from the navigation sidebar.
2. The System deletes the session record.
3. The System clears the session cookie and returns the User to the login page.

Xref: Section 3.2.2, Log Out

Use case: **Manage Account Settings**

Diagram:

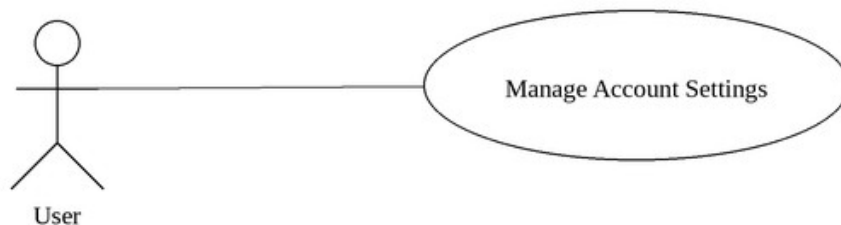


Figure 6 - Manage Account Settings Use Case

Brief Description

A User updates their contact details, notification preference or password.

Initial Step-By-Step Description

Before this use case can be initiated, the User has signed in and accessed the Settings page.

1. The User edits their first name, last name, email address, mobile number or email notification preference.
2. The System validates the submitted values and saves them.
3. If the email address changed, the System marks the address unverified and sends a new verification email.
4. The User may instead choose to change their password by supplying the current password and a new one twice.
5. The System verifies the current password, stores the new hash and ends every other session.

Xref: Section 3.2.3, Manage Account Settings

Use case: **View Notifications**

Diagram:

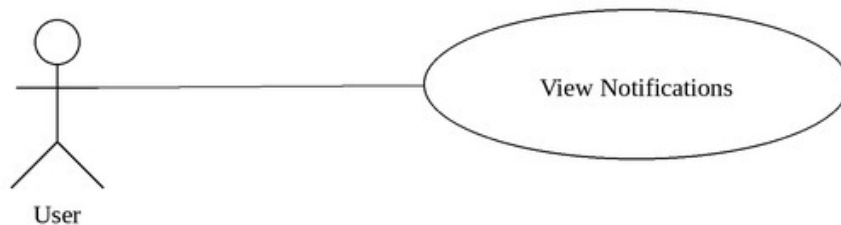


Figure 7 - View Notifications Use Case

Brief Description

A User reads the notifications the System has raised for them and marks them as read.

Initial Step-By-Step Description

Before this use case can be initiated, the User has signed in.

1. The User selects Notifications from the sidebar.
2. The System presents the User's notifications, newest first, with unread items highlighted.
3. The User selects a notification.
4. The System marks that notification read and follows its link.
5. The User may instead choose to mark every notification as read.

Xref: Section 3.2.4, View Notifications

2.2.2 Student Use Cases

The Student is the actor the system exists to serve. The use cases below carry the Student from registration through to tracking a submitted application.

This actor has more than one diagram. A fan of use cases grows wider with every use case on it, and every figure is set in the same text column, so beyond about five the labels become too

small to read. The 3 diagrams below therefore divide this actor's use cases between them; together they show the complete set, and each use case also appears on its own diagram with its description.

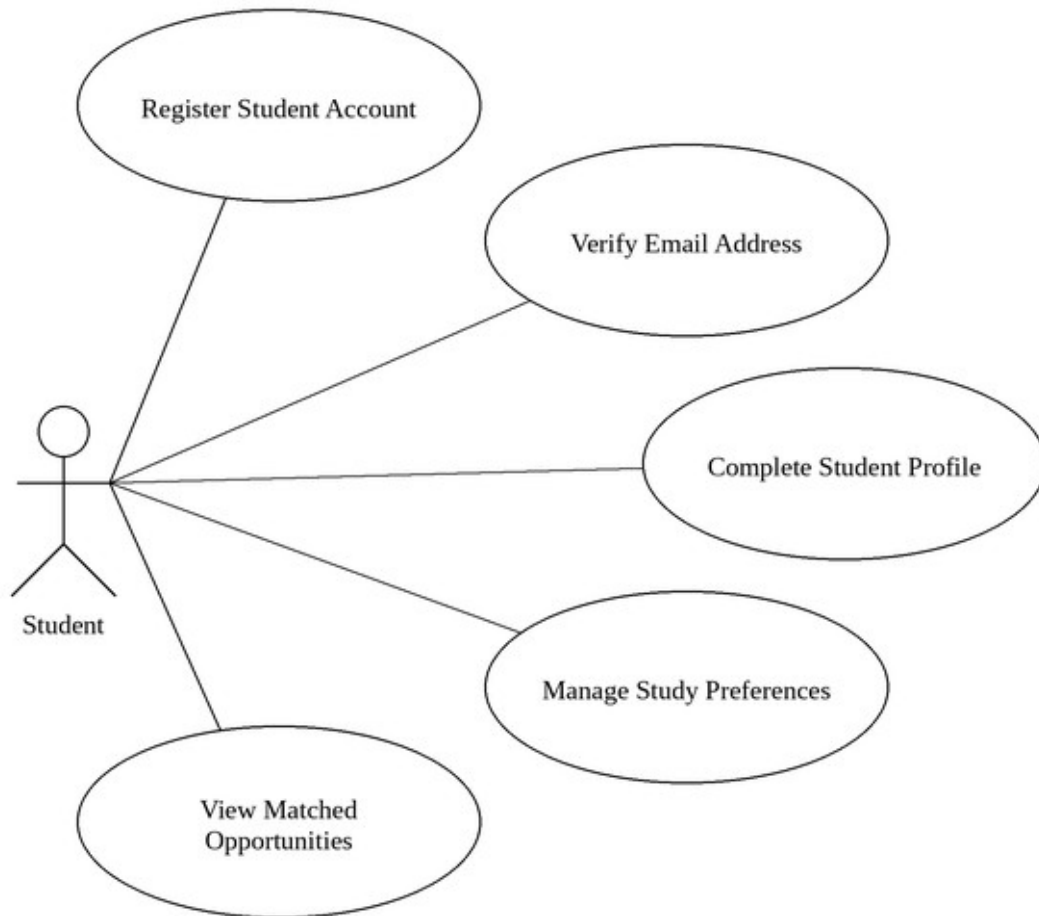


Figure 8 - Student Use Cases (1 of 3)

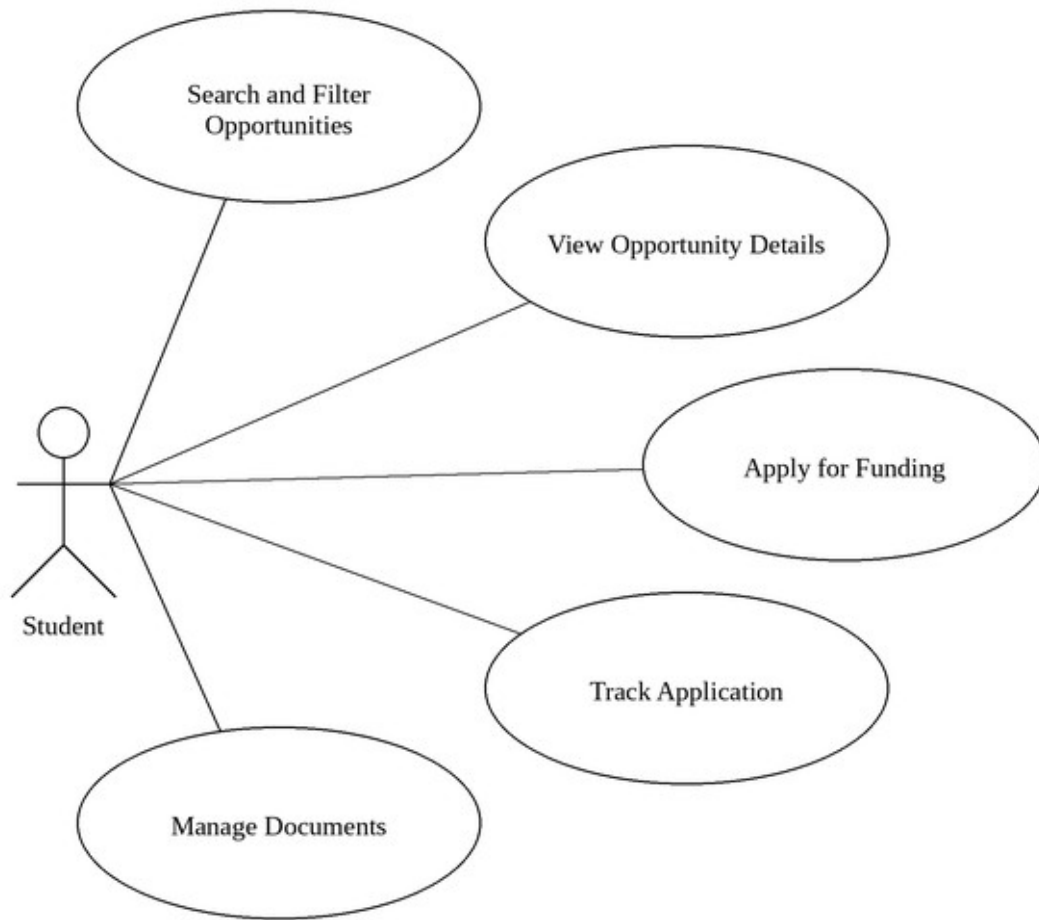


Figure 9 - Student Use Cases (2 of 3)

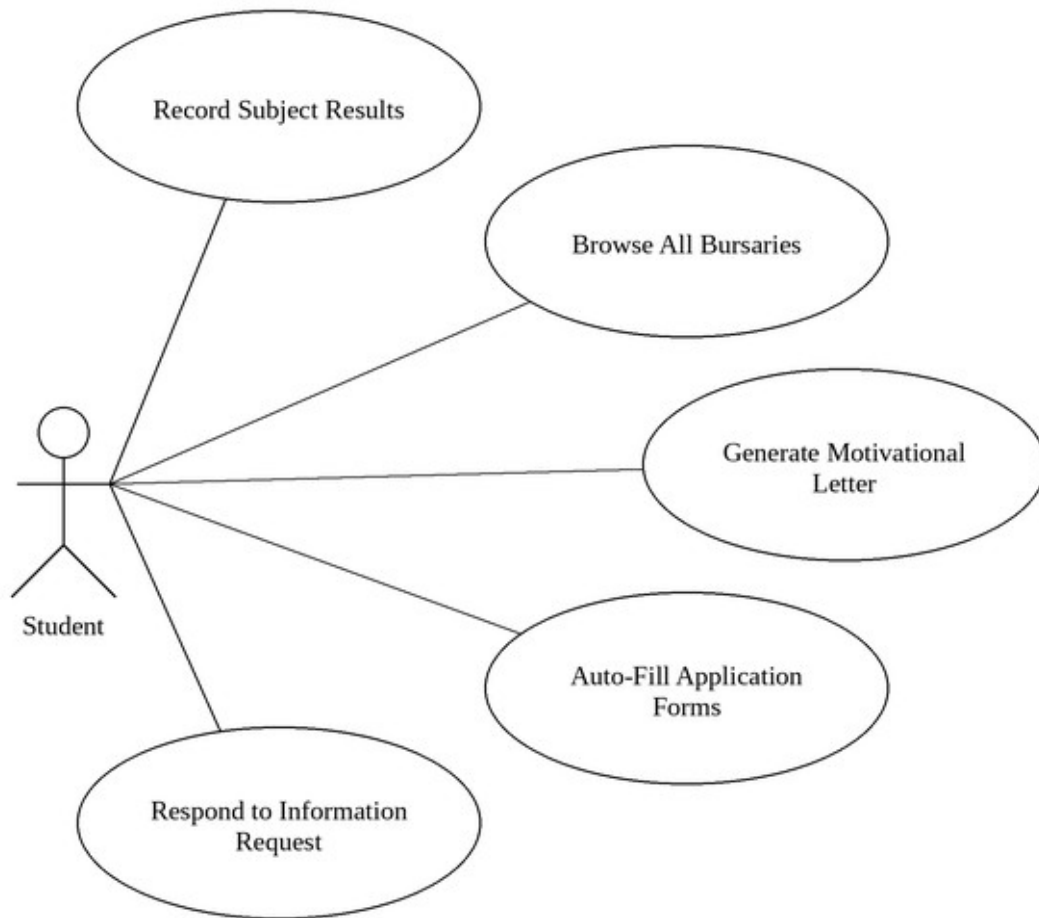


Figure 10 - Student Use Cases (3 of 3)

Use case: **Register Student Account**

Diagram:

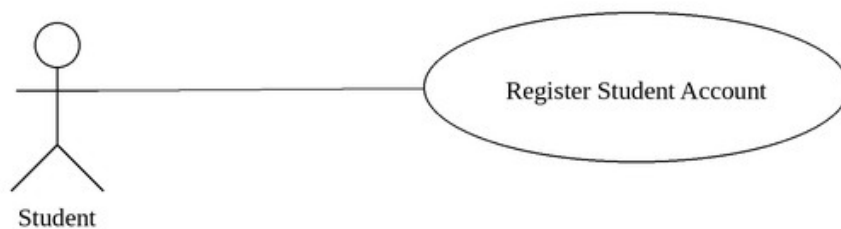


Figure 11 - Register Student Account Use Case

Brief Description

A prospective Student creates an account so that they can build a funding profile.

Initial Step-By-Step Description

Before this use case can be initiated, the visitor has accessed the Bursary-Bridge website and chosen to sign up as a Student.

1. The Student supplies a first name, last name, email address, mobile number and password, and confirms the password.
2. The Student indicates whether to receive funding opportunity notifications by email.
3. The Student accepts the Terms of Service and Privacy Notice.
4. The System validates every field and confirms that the email address is not already registered.
5. The System creates the account, an empty Student profile and a session, and sends a verification email.
6. The System presents the Check your email screen.

Xref: Section 3.2.5, Register Student Account

Use case: **Verify Email Address**

Diagram:

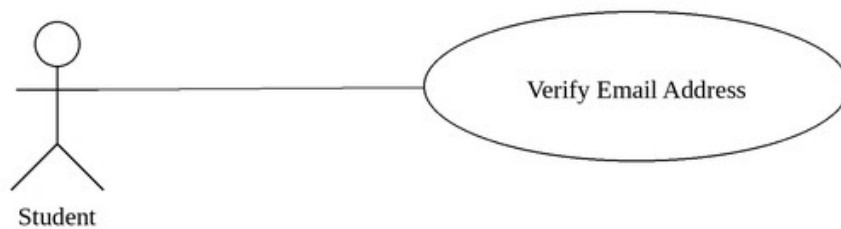


Figure 12 - Verify Email Address Use Case

Brief Description

The Student confirms ownership of the email address supplied at registration.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has registered and the System has sent a verification email.

1. The Student opens the verification email and follows the link.
2. The System reads the token from the link and compares its hash against the stored token.
3. The System confirms that the token has not expired.
4. The System marks the email address verified and deletes the token.
5. The System takes the Student to the first step of the profile.

Xref: Section 3.2.6, Verify Email Address

Use case: **Complete Student Profile**

Diagram:

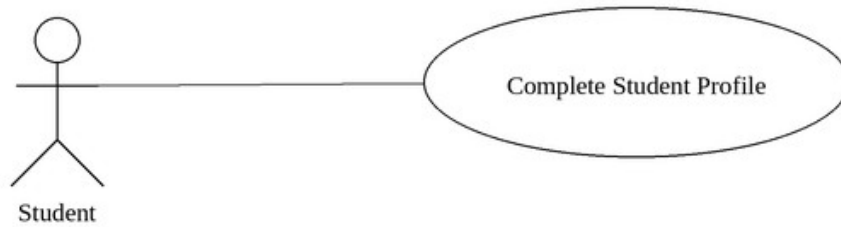


Figure 13 - Complete Student Profile Use Case

Brief Description

The Student supplies the information the Matching Engine needs, one step at a time.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has registered and is signed in.

1. The Student states their education stage, the qualification level they intend to study and their study status. If they are already enrolled, the System also asks for their current institution, current programme and year of study.
2. The Student records their study preferences, as described in Section 2.2.2, Manage Study Preferences.
3. The Student gives their latest academic average, or indicates that they do not know it, and selects the results and achievements they hold.
4. The Student selects what they need funding for and describes their current funding situation.
5. The Student selects a household income band and, optionally, date of birth, citizenship and further eligibility information.
6. The Student selects their province, city or town, expected study location and up to five career interests.
7. The System presents the whole profile for review, with a link to edit any section.
8. The Student confirms, and the System opens the Student Dashboard.

Xref: Section 3.2.7, Complete Student Profile

Use case: **Manage Study Preferences**

Diagram:

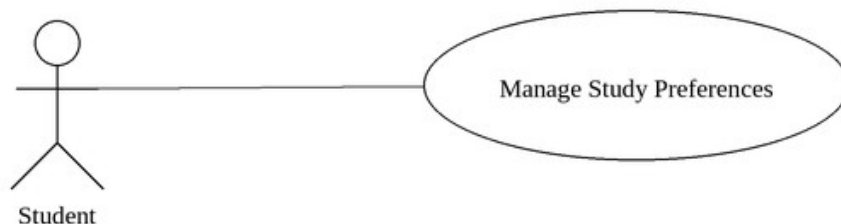


Figure 14 - Manage Study Preferences Use Case

Brief Description

The Student records up to six courses, each paired with the institution at which they intend to study it.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has signed in and reached the Study Preferences step or opened it from their profile.

1. The System presents the Student's existing preferences, numbered in order.
2. The Student selects a course from the standardised catalogue.
3. The Student selects the institution at which they intend to study that course.
4. The Student repeats steps 2 and 3 for each combination they are considering, up to six.
5. The Student may remove a preference or change either half of a pair.
6. The System saves the list, renumbering the preferences so that they remain in order.

Xref: Section 3.2.8, Manage Study Preferences

Use case: **View Matched Opportunities**

Diagram:

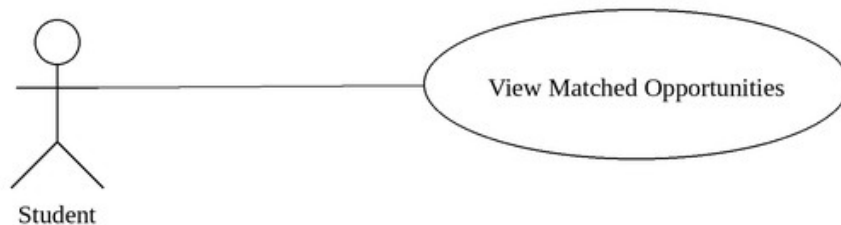


Figure 15 - View Matched Opportunities Use Case

Brief Description

The Student sees the funding programmes that best fit their profile, ranked, with the reasons behind each score.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has completed their profile and accessed the Student Dashboard.

1. The System retrieves every published funding programme that is still open.
2. The System scores each programme against the Student's profile.
3. The System presents the number of matches, active applications and approaching deadlines.
4. The System presents the best matching programmes, each with its match percentage and the criteria that were met.

5. The System presents profile strength and the sections that would most improve the Student's matches.

Xref: Section 3.2.9, View Matched Opportunities

Use case: **Search and Filter Opportunities**

Diagram:

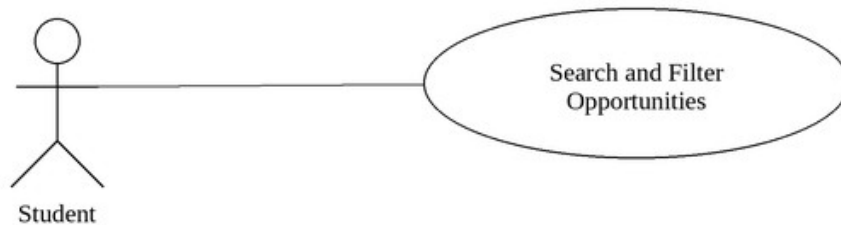


Figure 16 - Search and Filter Opportunities Use Case

Brief Description

The Student narrows the list of funding opportunities to those of interest.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has completed their profile and accessed the Opportunities page.

1. The System presents every open opportunity, scored and ranked by match.
2. The Student enters search terms, or selects filters for course, institution, qualification, funding type, what the programme covers, province, minimum match score or closing date.
3. The Student may choose to hide opportunities already applied for.
4. The System applies the filters and presents the matching opportunities in pages.
5. The Student may reorder the results by match, closing date or name.

Xref: Section 3.2.10, Search and Filter Opportunities

Use case: **View Opportunity Details**

Diagram:

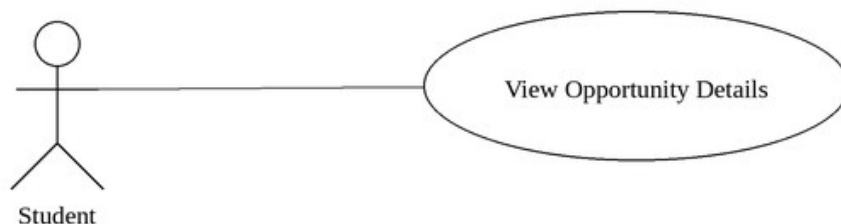


Figure 17 - View Opportunity Details Use Case

Brief Description

The Student reads the full terms of a funding programme and why they do or do not match it.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has selected an opportunity from the Dashboard or the Opportunities page.

1. The System presents the programme name, the funding Organisation, the closing date and what the programme covers.
2. The System presents the full description, the supported courses and the supported institutions.
3. The System presents the eligibility requirements, including academic, qualification, year of study, citizenship, financial and geographic conditions.
4. The System presents the documents the programme requires and how to apply.
5. The System presents the Student's match score together with every criterion, showing which were met, which were not, and which could not be evaluated.

Xref: Section 3.2.11, View Opportunity Details

Use case: **Apply for Funding**

Diagram:

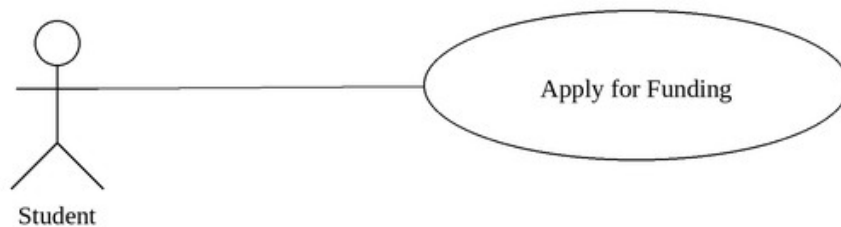


Figure 18 - Apply for Funding Use Case

Brief Description

The Student submits an application to a funding programme, reusing the profile they have already built.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has completed their profile and has opened a published funding programme that is still open.

1. The System presents the profile that will be sent with the application, for review only, with links to edit any section.
2. The Student answers any questions specific to this programme.
3. The Student attaches the documents the programme requires from those already uploaded.
4. The System presents the application for review, together with the match score.
5. The Student confirms that the information is accurate and submits.

6. The System records the application, snapshots the match score and the eligibility verdict, and notifies the Student.

Xref: Section 3.2.12, Apply for Funding

Use case: **Track Application**

Diagram:

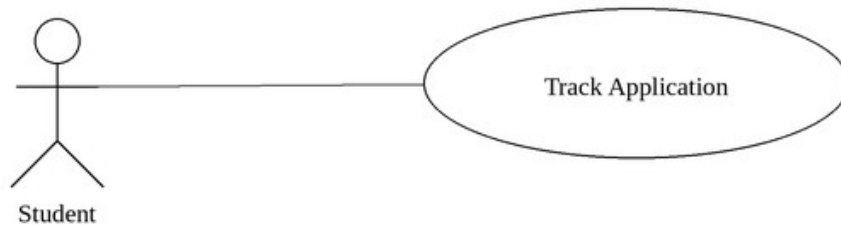


Figure 19 - Track Application Use Case

Brief Description

The Student follows the progress of an application through to a decision.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has at least one application and has accessed My Applications.

1. The System presents the Student's applications grouped by status.
2. The Student selects an application.
3. The System presents the programme, the funder, the date applied, the closing date and the date of the last change.
4. The System presents a progress timeline from started through submitted, under review, shortlisted and decided.
5. The System presents the answers given, the documents attached and the match as it stood at submission.

Xref: Section 3.2.13, Track Application

Use case: **Manage Documents**

Diagram:

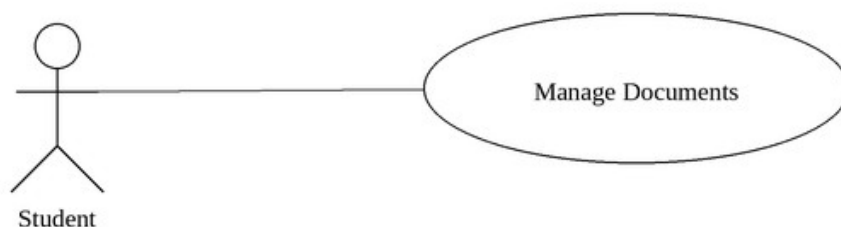


Figure 20 - Manage Documents Use Case

Brief Description

The Student uploads and maintains the supporting documents funders commonly ask for.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has signed in and accessed the Documents page.

1. The System presents the documents the Student has already uploaded.
2. The Student selects the kind of document and chooses a file.
3. The System checks the file type and size, stores the file and records it against the Student.
4. The System recalculates profile strength.
5. The Student may view or permanently remove any document.

Xref: Section 3.2.14, Manage Documents

Use case: **Record Subject Results**

Diagram:

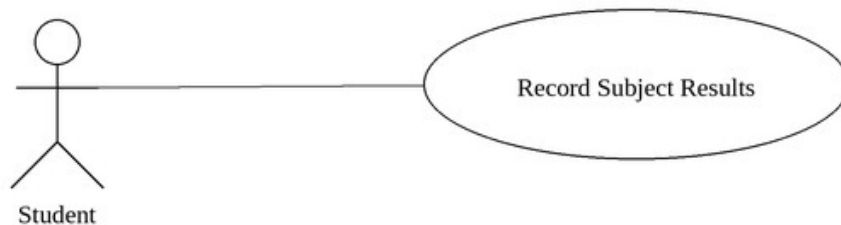


Figure 21 - Record Subject Results Use Case

Brief Description

A Student records the subjects or modules they are taking and the marks they have, so that funding programmes stating a subject minimum can be matched against them.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has signed in and has accessed the My Results page.

1. The Student selects Add a result.
2. The Student chooses a subject from the standardised subject list, or types a subject the list does not carry.
3. The Student enters the year, and the mark if they have it.
4. The System stores the result against the Student's profile.
5. The System shows any funding programme requirement that the result now satisfies.

Xref: Section 3.2.15, Record Subject Results

Use case: **Browse All Bursaries**

Diagram:

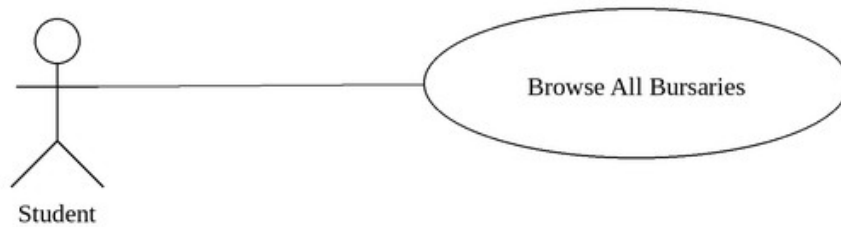


Figure 22 - Browse All Bursaries Use Case

Brief Description

A Student browses the complete directory of funding opportunities held by the system, whether or not they match the Student's own profile.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has signed in and has accessed the All Bursaries page.

1. The Student opens All Bursaries.
2. The System lists every published opportunity, open ones first.
3. The Student searches by name, organisation, course or institution, or filters by status, funding type, field of study, institution, province or qualification.
4. The System shows the opportunities that satisfy the search and filters, with the status of each.
5. The Student opens an opportunity to read its full details and the source it was read from.

Xref: Section 3.2.16, Browse All Bursaries

Use case: **Generate Motivational Letter**

Diagram:

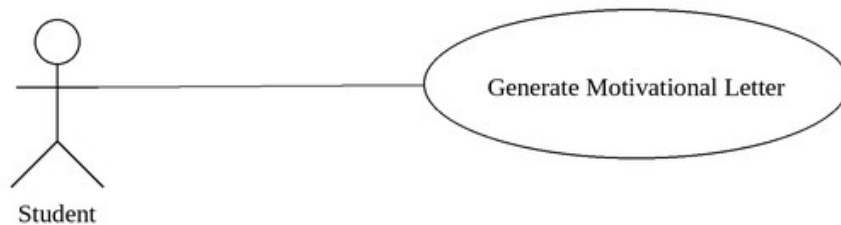


Figure 23 - Generate Motivational Letter Use Case

Brief Description

A Student produces a draft motivational letter for a chosen funding programme, assembled from their profile and from answers they write themselves.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has signed in and has accessed the Motivational Letters page.

1. The Student selects the funding programme the letter is for.
2. The Student answers, in their own words, as many of the seven questions as they wish.
3. The System shows which facts from the Student's profile it will use.
4. The System composes a draft from those facts and those answers.
5. The Student edits the draft, saves it, and downloads it as a PDF.

Xref: Section 3.2.17, Generate Motivational Letter

Use case: **Auto-Fill Application Forms**

Diagram:

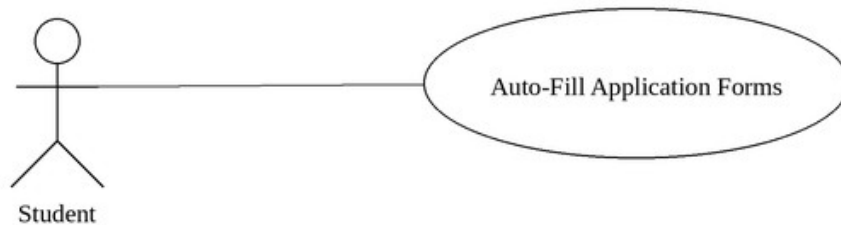


Figure 24 - Auto-Fill Application Forms Use Case

Brief Description

A Student supplies one completed application form and a number of blank ones, and the System transfers the information from the first onto the others.

Initial Step-By-Step Description

Before this use case can be initiated, the Student has signed in and has accessed the Auto-Fill Forms page.

1. The Student uploads one application form they have already completed.
2. The Student uploads one or more blank forms.
3. The System reads the completed form and identifies the fields on each blank form.
4. The System fills the fields it can match with confidence and leaves the rest blank for the Student.
5. The Student reviews each field, corrects anything wrong, and downloads the completed forms.

Xref: Section 3.2.18, Auto-Fill Application Forms

Use case: **Respond to Information Request**

Diagram:

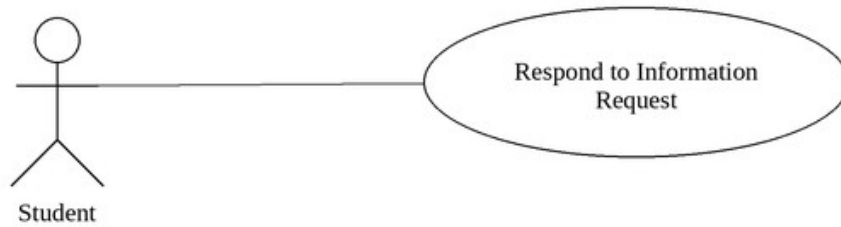


Figure 25 - Respond to Information Request Use Case

Brief Description

A Student supplies documents or information that a Funder has asked for in support of a submitted application.

Initial Step-By-Step Description

Before this use case can be initiated, the Funder has requested information and the Student has opened the application.

1. The Student opens the application the request belongs to.
2. The System shows exactly what the Funder asked for, in the Funder's own words.
3. The Student attaches a document to each item requested.
4. The Student submits the response.
5. The System notifies the Funder and records the response against the application.

Xref: Section 3.2.19, Respond to Information Request

2.2.3 Corporate User Use Cases

The Corporate User administers an Organisation's funding programmes and decides who receives funding.

This actor has more than one diagram. A fan of use cases grows wider with every use case on it, and every figure is set in the same text column, so beyond about five the labels become too small to read. The 3 diagrams below therefore divide this actor's use cases between them; together they show the complete set, and each use case also appears on its own diagram with its description.

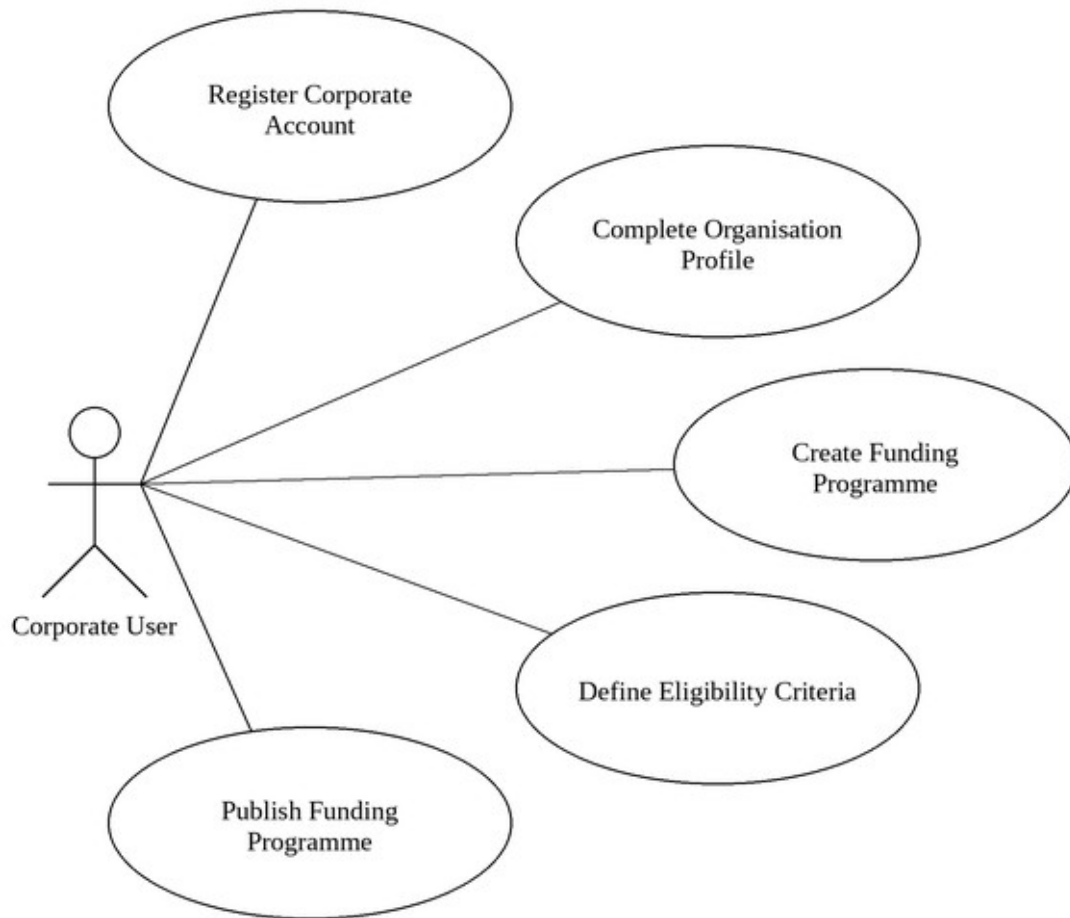


Figure 26 - Corporate User Use Cases (1 of 3)

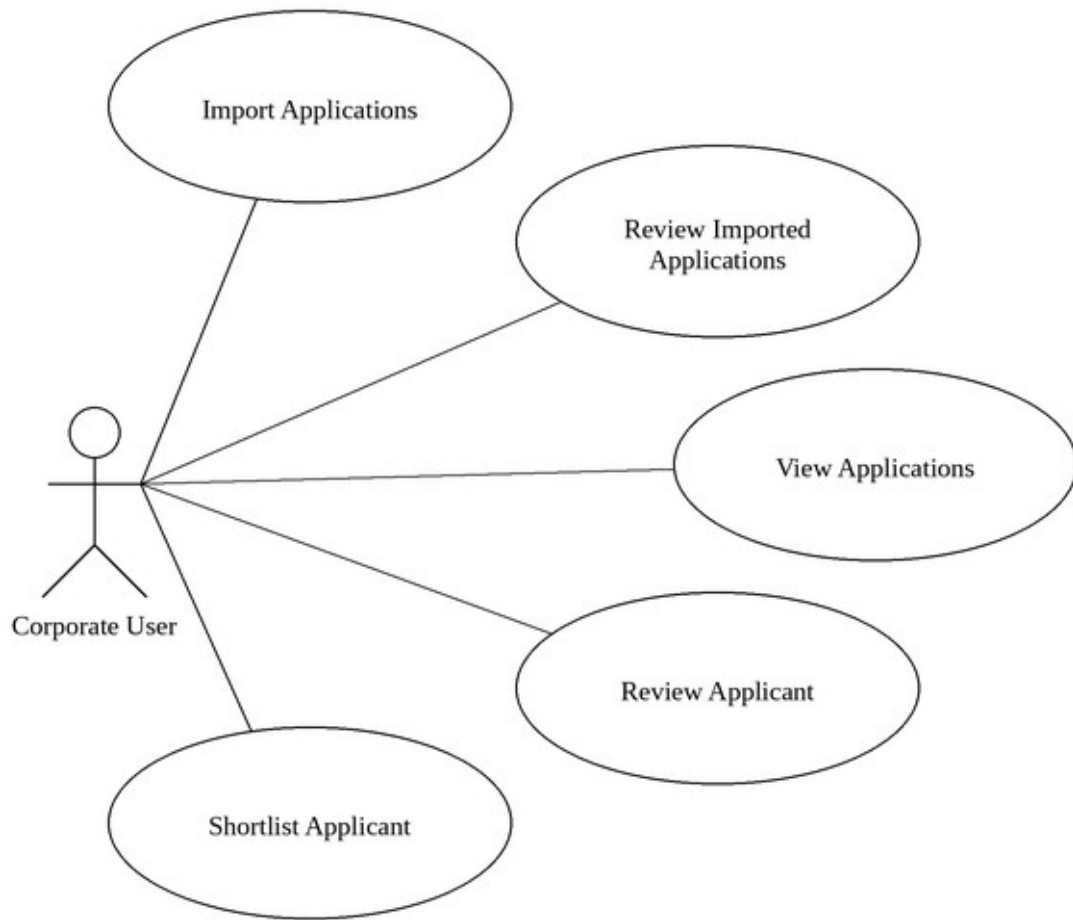


Figure 27 - Corporate User Use Cases (2 of 3)

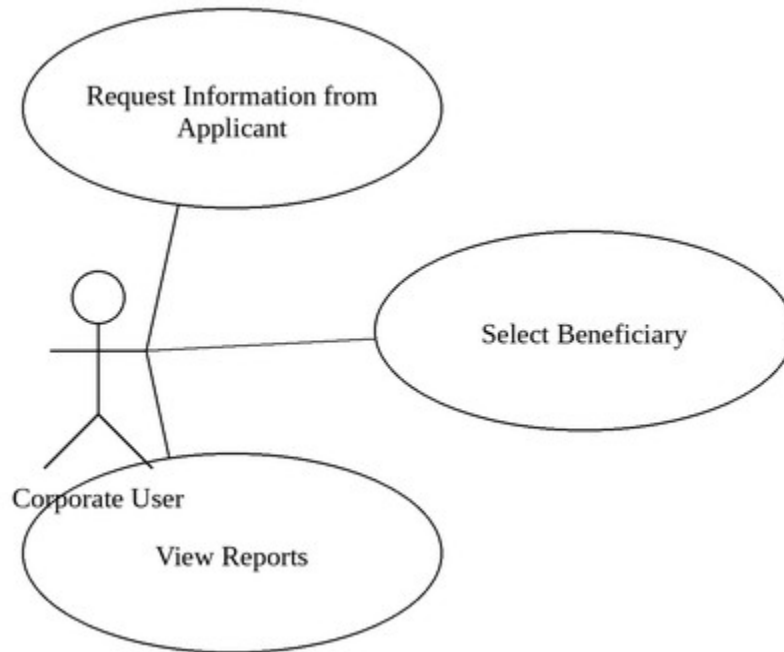


Figure 28 - Corporate User Use Cases (3 of 3)

Use case: **Register Corporate Account**

Diagram:

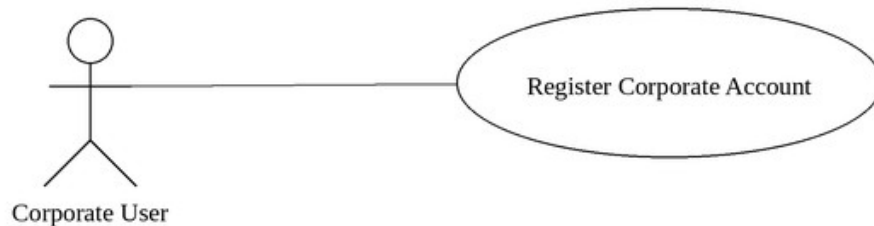


Figure 29 - Register Corporate Account Use Case

Brief Description

A representative of a funding Organisation creates an account so that the Organisation can publish programmes.

Initial Step-By-Step Description

Before this use case can be initiated, the visitor has accessed the Bursary-Bridge website and chosen to sign up as an Organisation.

1. The Corporate User supplies a first name, last name, work email address, mobile number and password, and confirms the password.
2. The Corporate User indicates whether to receive notifications by email.
3. The Corporate User accepts the Terms of Service and Privacy Notice.

4. The System validates every field and confirms the email address is not already registered.
5. The System creates the account, an Organisation record and a session, and sends a verification email.
6. The System opens the first step of the Organisation profile.

Xref: Section 3.2.20, Register Corporate Account

Use case: **Complete Organisation Profile**

Diagram:

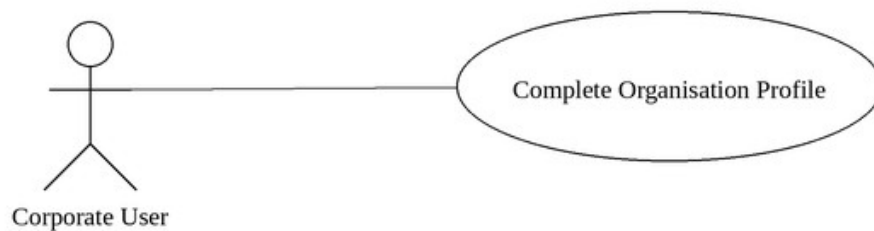


Figure 30 - Complete Organisation Profile Use Case

Brief Description

The Corporate User describes the Organisation, their role within it, and how it funds education.

Initial Step-By-Step Description

Before this use case can be initiated, the Corporate User has registered and is signed in.

1. The Corporate User gives the Organisation's name, type, industry, website and country.
2. The Corporate User states their own role, the size of the Organisation and their department.
3. The Corporate User states whether the Organisation currently funds education, which kinds of programme it runs and roughly how many applications it receives each year.
4. The Corporate User describes, optionally, how applications are managed today and the three biggest challenges faced.
5. The System presents the whole profile for review, with a link to edit any section.
6. The Corporate User confirms, and the System opens the Corporate Dashboard.

Xref: Section 3.2.21, Complete Organisation Profile

Use case: **Create Funding Programme**

Diagram:

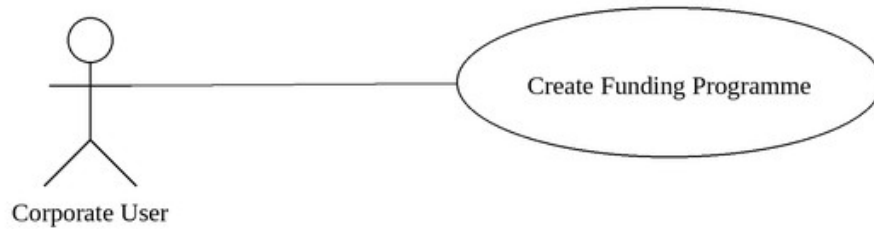


Figure 31 - Create Funding Programme Use Case

Brief Description

The Corporate User sets up a bursary, scholarship or grant for students to apply to.

Initial Step-By-Step Description

Before this use case can be initiated, the Corporate User has completed the Organisation profile and accessed the Programmes page.

1. The Corporate User gives the programme name, a short description, a full description and the programme type.
2. The Corporate User gives the date applications open and the date they close, and optionally the number of awards available.
3. The Corporate User defines the eligibility criteria, as described in Section 2.2.3, Define Eligibility Criteria.
4. The Corporate User selects what the programme pays for.
5. The Corporate User selects the documents applicants must provide and adds any questions specific to this programme.
6. The System presents the programme for review.
7. The Corporate User saves the programme as a draft, or publishes it.

Xref: Section 3.2.22, Create Funding Programme

Use case: **Define Eligibility Criteria**

This use case is part of the Create Funding Programme use case.

Diagram:

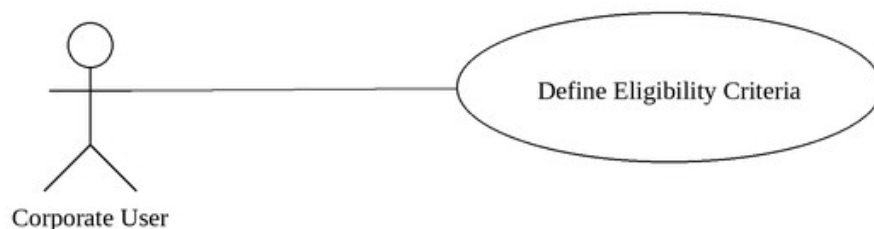


Figure 32 - Define Eligibility Criteria Use Case

Brief Description

The Corporate User states precisely who may apply to a funding programme.

Initial Step-By-Step Description

Before it can be initiated, the Corporate User has reached the Eligibility Criteria step of the programme builder.

1. The Corporate User selects the institutions the programme supports, or leaves the list empty to accept any institution.
2. The Corporate User selects the courses the programme supports, or leaves the list empty to accept any course.
3. The Corporate User selects the qualification levels and years of study that qualify.
4. The Corporate User sets a minimum academic average, if one applies.
5. The Corporate User sets the citizenship, household income and provincial conditions, if any apply.
6. The Corporate User records any further requirements in their own words.

Xref: Section 3.2.23, Define Eligibility Criteria

Use case: **Publish Funding Programme**

This use case extends the Create Funding Programme use case.

Diagram:

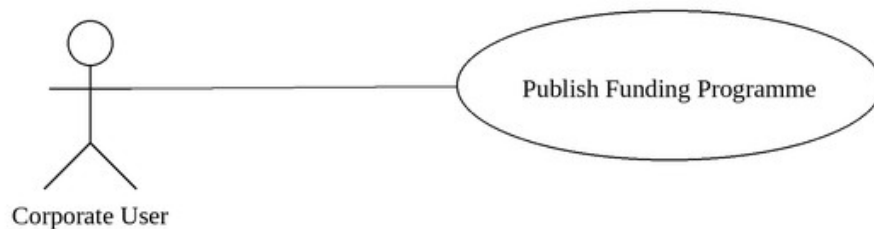


Figure 33 - Publish Funding Programme Use Case

Brief Description

The Corporate User makes a programme visible to students, or closes it again.

Initial Step-By-Step Description

Before it can be initiated, the Corporate User has opened a programme belonging to their Organisation.

1. The Corporate User selects to publish the programme.
2. The System confirms that the closing date has not already passed.
3. The System sets the programme status to published.
4. The programme immediately appears among the matches of every student whose profile fits it.
5. The Corporate User may later close the programme, or reopen a closed programme.

Xref: Section 3.2.24, Publish Funding Programme

Use case: **Import Applications**

Diagram:

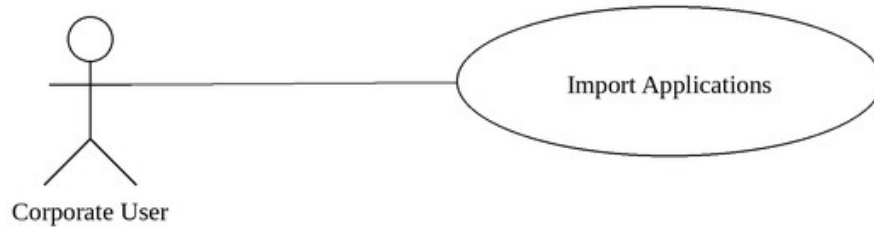


Figure 34 - Import Applications Use Case

Brief Description

A Corporate User uploads application forms the Organisation received outside Bursary-Bridge and has the System turn them into applicant records against one of its funding programmes.

Initial Step-By-Step Description

Before this use case can be initiated, the Corporate User has signed in and has opened a funding programme belonging to their Organisation.

1. The Corporate User selects Import Applications against a funding programme.
2. The Corporate User uploads the application forms, individually or as an archive.
3. The System reads each form and extracts the applicant's details.
4. The System scores each applicant against that programme and checks for duplicates.
5. The System presents the batch for review.

Xref: Section 3.2.25, Import Applications

Use case: **Review Imported Applications**

Diagram:

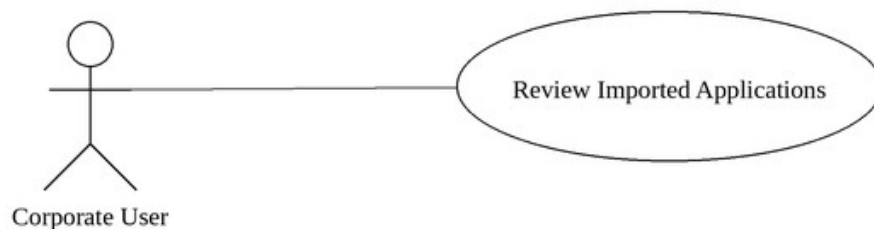


Figure 35 - Review Imported Applications Use Case

Brief Description

A Corporate User examines a processed import batch, corrects what was read wrongly, decides what to do about duplicates, and confirms which applications to bring into the programme.

Initial Step-By-Step Description

Before this use case can be initiated, an import batch has finished processing.

1. The Corporate User opens the batch.
2. The System shows the applications grouped as ready, needing review, duplicates and failures.
3. The Corporate User inspects an application beside the form it was read from.
4. The Corporate User corrects any field that was read wrongly.
5. The Corporate User confirms the import.

Xref: Section 3.2.26, Review Imported Applications

Use case: **View Applications**

Diagram:

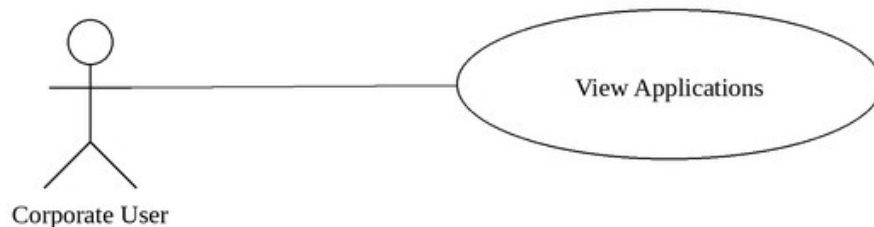


Figure 36 - View Applications Use Case

Brief Description

The Corporate User examines the applications submitted to the Organisation's programmes.

Initial Step-By-Step Description

Before this use case can be initiated, the Corporate User has signed in and accessed the Applications page.

1. The System presents the applications submitted to this Organisation's programmes, best match first.
2. The System presents counts by status across the top.
3. The Corporate User may search by applicant name or email, or filter by programme, status, institution, course, match score or eligibility verdict.
4. The System applies the filters and presents one page of results.
5. The Corporate User may select several applicants and act on them together.

Xref: Section 3.2.27, View Applications

Use case: **Review Applicant**

Diagram:

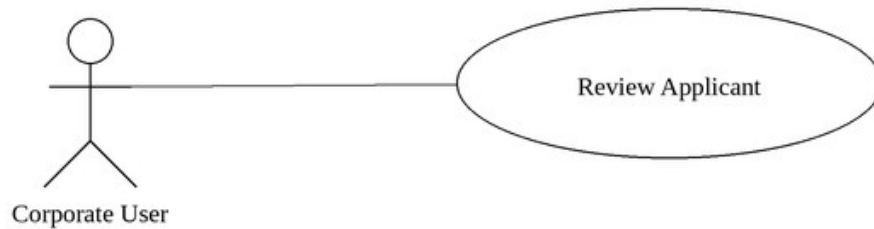


Figure 37 - Review Applicant Use Case

Brief Description

The Corporate User examines one applicant in detail and assesses them against the programme's criteria.

Initial Step-By-Step Description

Before this use case can be initiated, the Corporate User has selected an applicant from the Applications list.

1. The System presents the applicant's name, course, institution, match score, eligibility verdict and application status.
2. The System presents the application details, contact information and academic summary.
3. The System presents the applicant's study preferences and funding profile.
4. The System evaluates the applicant against the programme's current criteria and presents every criterion.
5. The System presents the documents attached and the answers given.
6. The Corporate User may move to the previous or next applicant without returning to the list.

Xref: Section 3.2.28, Review Applicant

Use case: **Shortlist Applicant**

This use case extends the Review Applicant use case.

Diagram:

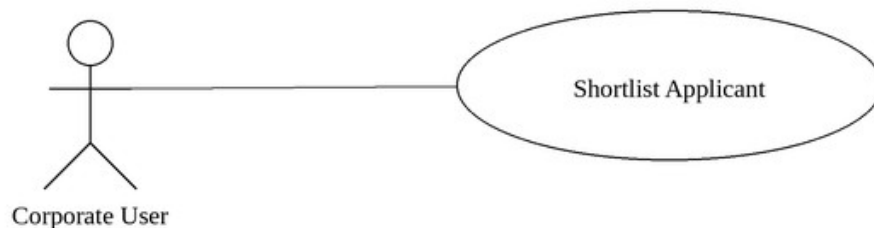


Figure 38 - Shortlist Applicant Use Case

Brief Description

The Corporate User marks an applicant as being under active consideration.

Initial Step-By-Step Description

Before it can be initiated, the Corporate User has opened an applicant or selected applicants from the list.

1. The Corporate User selects Add to Shortlist.
2. The System sets the application status to Shortlisted and records a shortlist entry.
3. The System notifies the applicant.
4. The applicant appears on the Shortlists page for the programme.
5. The Corporate User may later remove the applicant from the shortlist.

Xref: Section 3.2.29, Shortlist Applicant

Use case: **Request Information from Applicant**

This use case extends the Review Applicant use case.

Diagram:

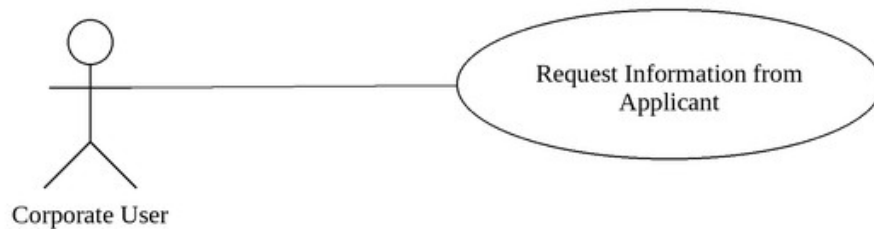


Figure 39 - Request Information from Applicant Use Case

Brief Description

The Corporate User asks an applicant for information or a document that is missing.

Initial Step-By-Step Description

Before it can be initiated, the Corporate User has opened an applicant.

1. The Corporate User selects Request Information.
2. The Corporate User writes what is required.
3. The System sets the application status to Documents Required and stores the message.
4. The System notifies the applicant, quoting the message.
5. The applicant sees the request on their application tracking page.

Xref: Section 3.2.30, Request Information from Applicant

Use case: **Select Beneficiary**

This use case extends the Review Applicant use case.

Diagram:

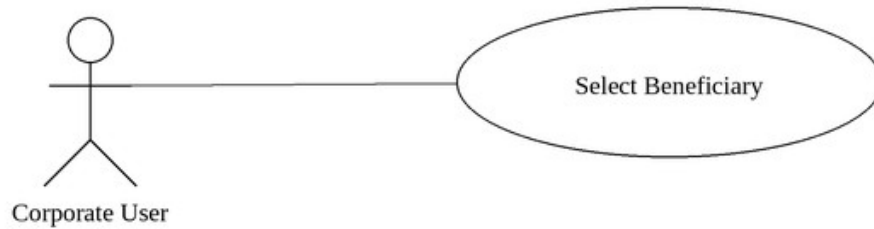


Figure 40 - Select Beneficiary Use Case

Brief Description

The Corporate User awards funding to an applicant, or records that an application was unsuccessful.

Initial Step-By-Step Description

Before it can be initiated, the Corporate User has opened an applicant or selected applicants from the shortlist.

1. The Corporate User selects Move to Selected.
2. The System sets the application status to Approved and records the applicant as a beneficiary.
3. The System notifies the applicant.
4. The applicant appears on the Beneficiaries page.
5. The Corporate User may instead reject an applicant, optionally giving a reason.

Xref: Section 3.2.31, Select Beneficiary

Use case: **View Reports**

Diagram:

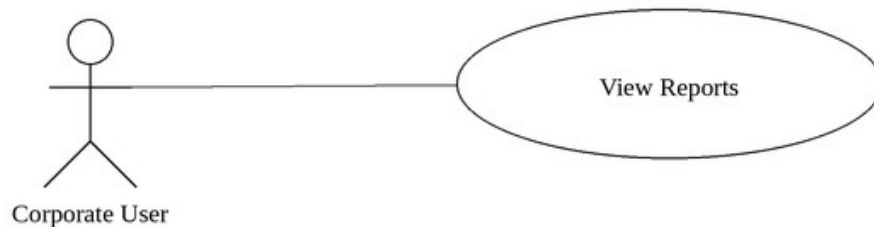


Figure 41 - View Reports Use Case

Brief Description

The Corporate User reviews the Organisation's application pipeline and reach.

Initial Step-By-Step Description

Before this use case can be initiated, the Corporate User has signed in and accessed the Reports page.

1. The System presents the totals for applications received, applicants found eligible, applicants shortlisted and beneficiaries selected.

2. The System presents the breakdown of applications by status.
3. The System presents the breakdown of applicants by province.
4. The System presents each programme with its application volume, eligible count and eligibility rate.
5. The System presents the institutions the Organisation's applicants come from.

Xref: Section 3.2.32, View Reports

2.2.4 Administrator Use Cases

The Administrator maintains the platform itself rather than any one Organisation's funding.

The Administrator does not apply for funding and does not decide who receives it.

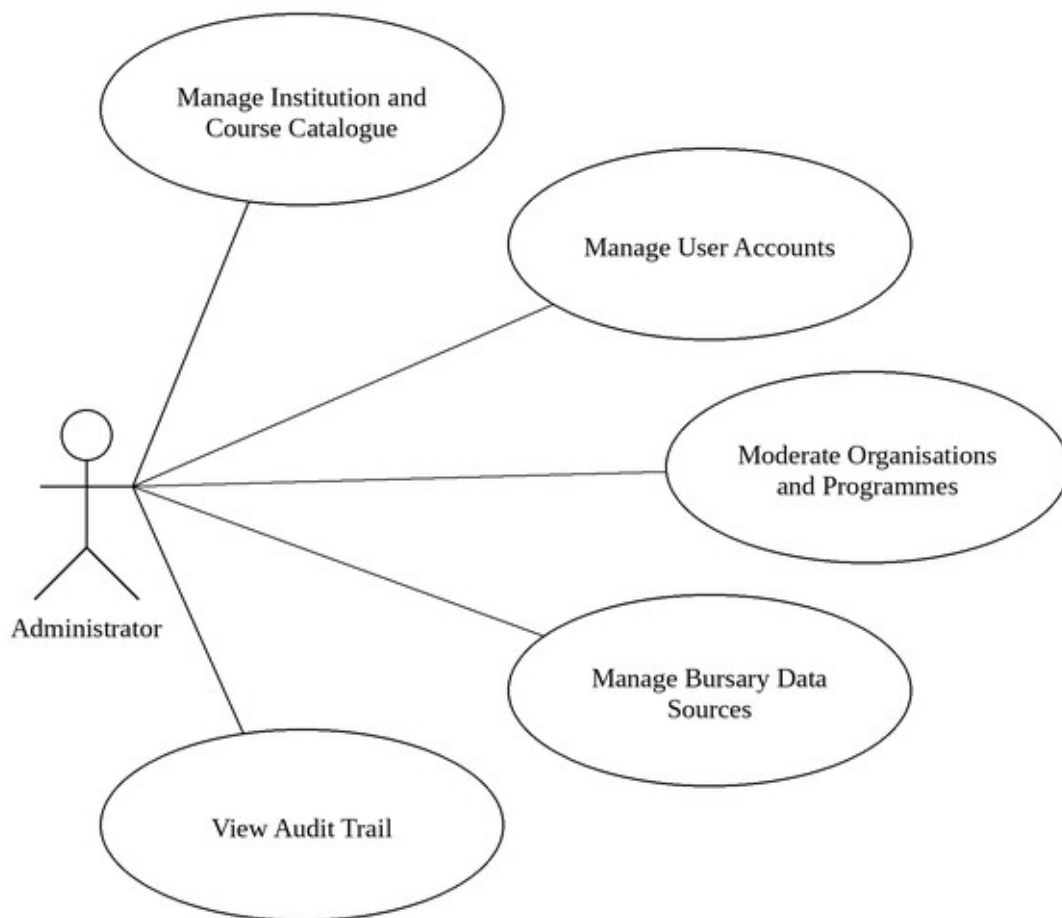


Figure 42 - Administrator Use Cases

Use case: **Manage Institution and Course Catalogue**

Diagram:

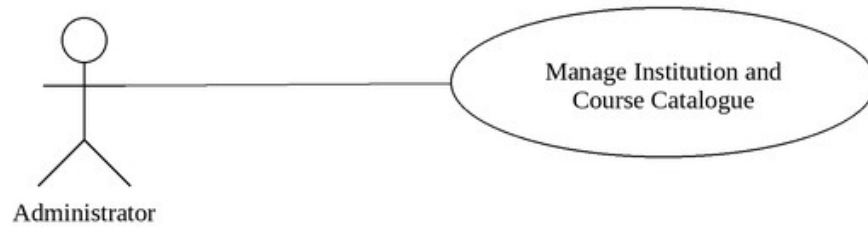


Figure 43 - Manage Institution and Course Catalogue Use Case

Brief Description

An Administrator maintains the standardised list of institutions and courses that Students and Funders choose from, and the links between them.

Initial Step-By-Step Description

Before this use case can be initiated, the Administrator has signed in and has accessed the Catalogue page.

1. The Administrator adds an institution or a course, or edits an existing one.
2. The System checks the name against the catalogue to prevent a second entry for the same thing.
3. The Administrator records which courses an institution offers.
4. The Administrator retires an entry that is no longer offered.

Xref: Section 3.2.33, Manage Institution and Course Catalogue

Use case: **Manage User Accounts**

Diagram:

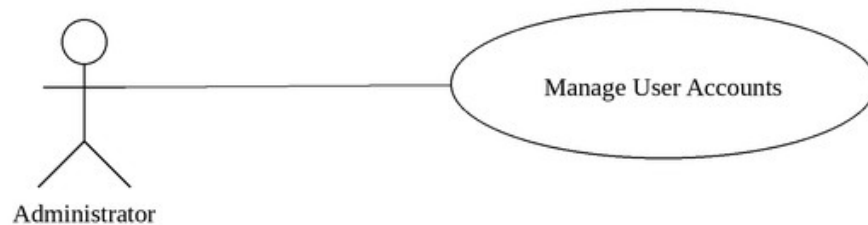


Figure 44 - Manage User Accounts Use Case

Brief Description

An Administrator suspends, restores or forces a password reset on an account, and may change the role an account holds.

Initial Step-By-Step Description

Before this use case can be initiated, the Administrator has signed in and has accessed the Users page.

1. The Administrator searches for an account.
2. The Administrator selects an action against it.
3. The System requires a reason.
4. The System applies the action and records it in the audit log.

Xref: Section 3.2.34, Manage User Accounts

Use case: **Moderate Organisations and Programmes**

Diagram:

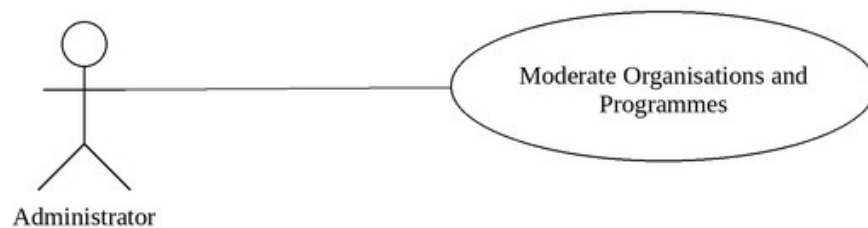


Figure 45 - Moderate Organisations and Programmes Use Case

Brief Description

An Administrator withdraws an Organisation or a published funding programme that breaches the platform's rules, and restores it if the matter is resolved.

Initial Step-By-Step Description

Before this use case can be initiated, the Administrator has signed in and has accessed the Organisations or Programmes page.

1. The Administrator opens the Organisation or funding programme.
2. The Administrator suspends it, giving a reason.
3. The System withdraws it from the Student-facing directory.
4. The Administrator restores it if the matter is resolved.

Xref: Section 3.2.35, Moderate Organisations and Programmes

Use case: **Manage Bursary Data Sources**

Diagram:

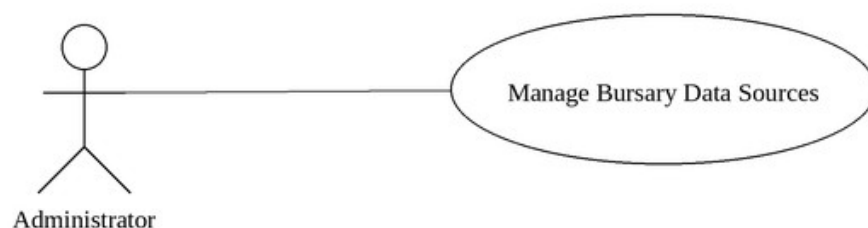


Figure 46 - Manage Bursary Data Sources Use Case

Brief Description

An Administrator runs and inspects the collection of funding opportunities from published external sources.

Initial Step-By-Step Description

Before this use case can be initiated, the Administrator has signed in and has accessed the Data Sources page.

1. The Administrator sees which sources are authorised and when each was last read.
2. The Administrator starts a collection run.
3. The System reads the authorised sources and records what it found.
4. The Administrator inspects the run and the opportunities it produced.

Xref: Section 3.2.36, Manage Bursary Data Sources

Use case: **View Audit Trail**

Diagram:

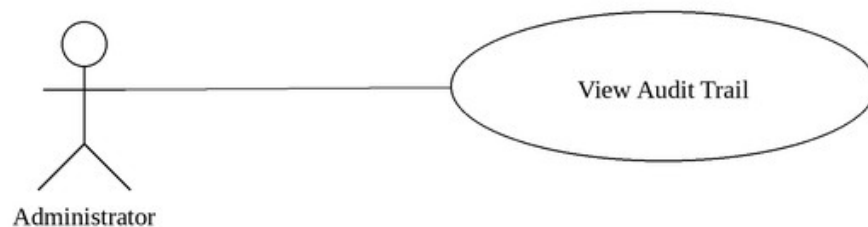


Figure 47 - View Audit Trail Use Case

Brief Description

An Administrator inspects the record of consequential actions taken across the system.

Initial Step-By-Step Description

Before this use case can be initiated, the Administrator has signed in and has accessed the Audit page.

1. The Administrator opens the audit trail.
2. The System lists the recorded actions, most recent first.
3. The Administrator filters by action, by the user who took it, or by date.

Xref: Section 3.2.37, View Audit Trail

2.2.5 Complete Use Case Diagram

Figure 48 brings together every actor and every use case described in this section. The Student, the Corporate User and the Administrator are all specialisations of a User, which is why the four use cases they share are attached once to the User actor rather than drawn three times. The diagram is set sideways on the page overleaf because it carries thirty-seven use cases; each of

them is also shown, at a larger size, on the per-actor diagrams above and on its own diagram beside its description.

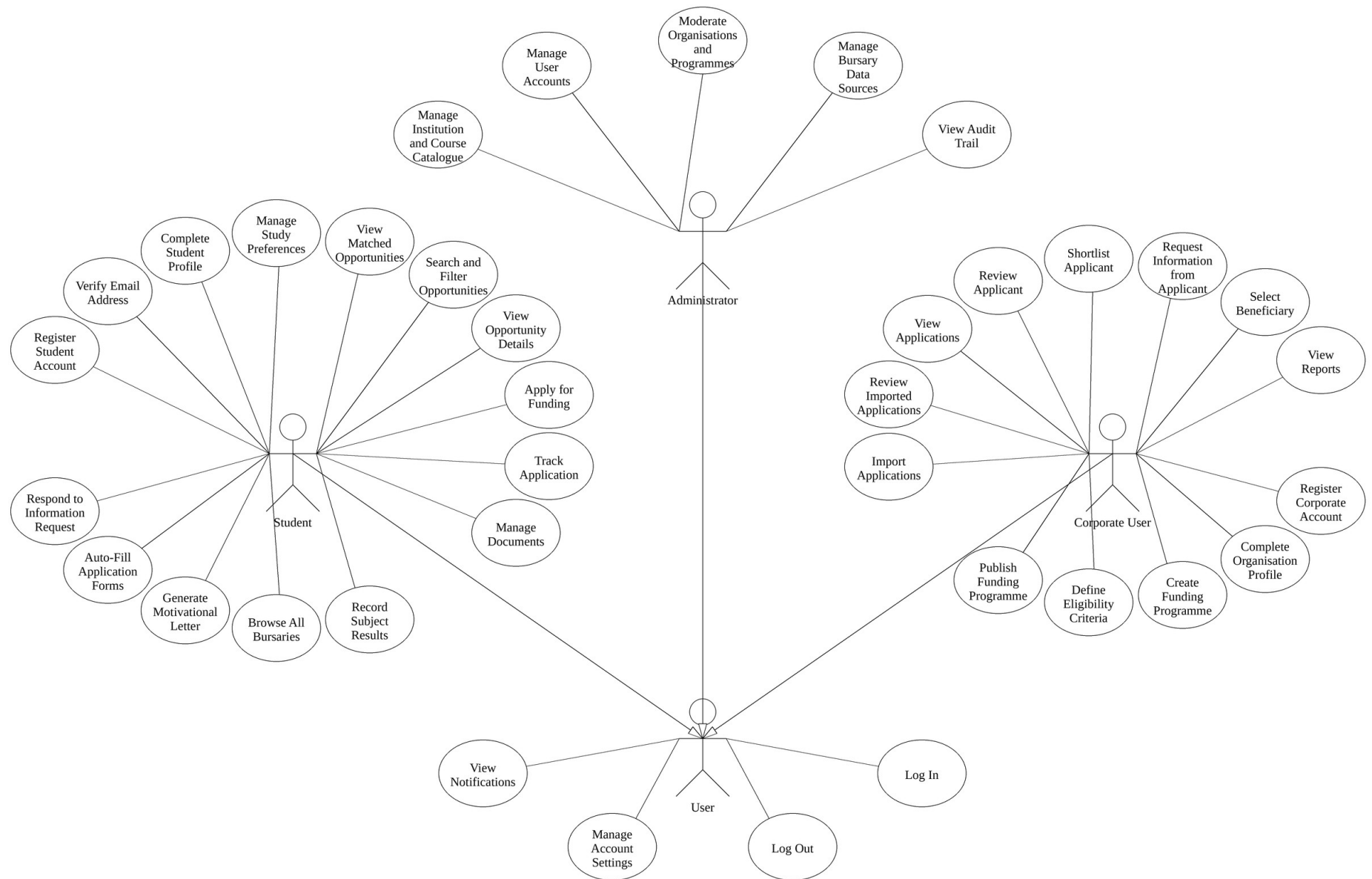


Figure 48 - Complete Use Case Diagram

2.3 *User Characteristics*

The Student is expected to be able to use a web browser and to complete an online form. Many students will reach the system on a mobile telephone, and some over a metered or intermittent connection, so the interface is laid out for small screens as well as large and the profile is gathered in short steps that can be left and resumed. No knowledge of bursary administration is assumed; wherever the system asks for something, it says why it is being asked.

The Corporate User is expected to be comfortable with an administrative web application of the kind used in an office, including tables, filters and forms. They are assumed to understand their own organisation's funding criteria, but not to have any technical knowledge, so the eligibility criteria are captured through ordinary form controls rather than through any rule language.

The Administrator is a member of the Bursary-Bridge operating team and is expected to understand the platform itself: the standardised catalogue of institutions and courses, the sources from which funding opportunities are collected, and the circumstances in which an account or a published programme should be withdrawn. The Administrator is not assumed to be a developer, and every administrative action is carried out through the interface rather than against the database.

No actor is expected to receive training. The detailed appearance of the screens is discussed in Section 3.1 below.

2.4 *Non-Functional Requirements*

Platform. Bursary-Bridge is a responsive web application. It is reached through a current web browser on a desktop computer, a laptop, a tablet or a mobile telephone, and requires no installation.

Server. The application runs on a Node.js server and stores its data in a PostgreSQL database. Both may run on a single machine for the prototype, and may be separated as demand grows.

Responsiveness. The interface is laid out for desktop, laptop, tablet and mobile widths. On a narrow screen the navigation collapses behind a menu and the layout reflows; no page requires horizontal scrolling.

Performance. Lists that may grow large, in particular the applicant list, are filtered and paged in the database so that the browser never receives more than one page of results.

Availability. The application is intended to be available at all times, subject to the hosting arrangements of the deploying institution.

Accessibility. The interface uses semantic HTML, labelled form controls, visible keyboard focus and sufficient colour contrast, so that it can be used with a keyboard alone and with a screen reader.

Usability. Each screen states where the User is, what is required of them and why it is required. Every list presents a meaningful empty state, and every operation presents a loading, success and error state. Users are never shown a technical error message.

3.0. Requirements Specification

3.1 External Interface Requirements

The system depends on two external systems. The first is the Email Service, which delivers address verification messages and status notifications. The system passes a recipient address, a subject and a body, and treats delivery as advisory: no decision anywhere in the system depends on a message having been delivered, and every journey can be completed through the interface alone.

The second is the set of published bursary sources from which funding opportunities are collected, described in Section 2.2.4, Manage Bursary Data Sources. The system reads pages published by those sources over HTTP and passes nothing to them beyond the request itself and an identifying user agent. It reads each source's robots file first and does not fetch any page that file disallows, and it waits between requests for as long as the source asks. What it takes from a page is the description of the opportunity and the address the page was read from; that address is stored against the opportunity and shown to Students, so that any statement the system makes about a bursary can be checked against its source.

Documents are written to Document Storage through an interface that names only a folder, a file name, a content type and the bytes. The prototype writes to the local file system; an object storage service may be substituted without any change to the code that calls it.

The user interface is delivered as web pages. The Student portal presents a dashboard, the matched opportunities, the complete bursary directory, the detail of a single opportunity, an application form, an application tracker, a profile, a results page, a motivational letter editor, an auto-fill workspace and a document library. The Corporate portal presents a dashboard, a programme builder, an application import workspace, an applicant list, an applicant detail screen, a shortlist, a beneficiary register and a report. The Administration portal presents the catalogue, the user register, the organisation and programme registers, the data sources and the audit trail. All three portals share a navigation sidebar which collapses behind a menu on a narrow screen.

3.2 Functional Requirements

The Logical Structure of the Data is contained in Section 3.3.1.

3.2.1 Log In

Use Case Name	Log In
XRef	Section 2.2.1, Log In
Trigger	The User submits the login form.
Precondition	The User has a registered account and is not currently signed in.
Basic Path	1. The User enters an email address and password. 2. The System looks up the account by email address. 3. The System compares the supplied password against the stored bcrypt hash. 4. The System creates a session record and returns a signed, HTTP-only session cookie. 5. The System redirects the User according to their role and onboarding state.
Alternative Paths	In step 5, if the User has not completed onboarding, the System redirects to the outstanding step rather than the dashboard.
Postcondition	The User is authenticated and holds a valid session.
Exception Paths	If the email address is not registered, or the password does not match, the System returns the same message in both cases so that the endpoint cannot be used to discover which email addresses hold accounts. After ten failed attempts from one address, or six against one account, further attempts are refused for five minutes.
Other	Sessions expire after fourteen days. Only a hash of the session token is stored, so a copy of the database cannot be replayed as a login.

3.2.2 Log Out

Use Case Name	Log Out
XRef	Section 2.2.1, Log Out
Trigger	The User selects Log out.
Precondition	The User holds a valid session.
Basic Path	1. The System deletes the session record from the database. 2. The System clears the session cookie in the browser. 3. The System returns the User to the login page.
Alternative Paths	None.
Postcondition	The session no longer exists and the cookie cannot be reused.
Exception Paths	If the session has already expired, the System still clears the cookie and returns the User to the login page.
Other	Because the session is deleted on the server, logging out invalidates the cookie immediately rather than relying on its expiry.

3.2.3 Manage Account Settings

Use Case Name	Manage Account Settings
XRef	Section 2.2.1, Manage Account Settings

Trigger	The User submits the account details form or the change password form.
Precondition	The User holds a valid session.
Basic Path	1. The System validates the submitted values on the server. 2. For a details change, the System writes the new values to the User record. 3. For a password change, the System verifies the current password, hashes the new password and stores it. 4. The System confirms the change to the User.
Alternative Paths	In step 2, if the email address has changed, the verified flag is cleared and a fresh verification email is sent.
Postcondition	The account record reflects the change. After a password change every other session has been invalidated.
Exception Paths	If the new email address already belongs to another account, the change is refused. If the current password is incorrect, the password change is refused.
Other	Changing a password deliberately signs the User out of every device, so that a stolen session cannot survive the change.

3.2.4 View Notifications

Use Case Name	View Notifications
XRef	Section 2.2.1, View Notifications
Trigger	The User opens the Notifications page.
Precondition	The User holds a valid session.
Basic Path	1. The System retrieves the fifty most recent notifications belonging to this User. 2. The System presents them newest first, showing the unread count. 3. The User selects a notification, or selects to mark all as read. 4. The System records the read timestamp and, where the notification carries a link, navigates to it.
Alternative Paths	The User may mark every unread notification as read in a single action.
Postcondition	The selected notifications are marked read and the unread badge is updated.
Exception Paths	A User can only ever read notifications addressed to their own account; a request for another User's notification affects nothing.
Other	Notifications are also mirrored to email when the User has left email notifications enabled.

3.2.5 Register Student Account

Use Case Name	Register Student Account
XRef	Section 2.2.2, Register Student Account

Trigger	The Student submits the student registration form.
Precondition	The visitor is not signed in and does not hold an account under the supplied email address.
Basic Path	1. The System validates the name fields, the email address, the South African mobile number and the password. 2. The System confirms that the two password entries match and that the terms have been accepted. 3. The System confirms that no account already exists for the email address. 4. The System hashes the password using bcrypt and creates the User record with the role Student. 5. The System creates an empty Student profile positioned at the first onboarding step. 6. The System creates a session, sends a verification email and presents the verification screen.
Alternative Paths	None.
Postcondition	A Student account and an empty profile exist, the Student is signed in, and a verification email has been sent.
Exception Paths	If the email address is already registered the System refuses the registration and invites the visitor to sign in instead. If any field fails validation the System returns the specific field errors and creates nothing. More than five registration attempts from one address within ten minutes are refused.
Other	The password must be at least eight characters and contain an upper case letter, a lower case letter and a digit. The same rules are enforced on the server and shown as a live checklist beneath the field.

3.2.6 Verify Email Address

Use Case Name	Verify Email Address
XRef	Section 2.2.2, Verify Email Address
Trigger	The Student follows the verification link, or requests that the email be sent again.
Precondition	A verification token exists for the account and has not yet been used.
Basic Path	1. The System hashes the token from the link and looks up the matching record. 2. The System confirms the token has not expired. 3. The System sets the verified timestamp on the User record. 4. The System deletes the token so that the link cannot be used twice. 5. The System redirects the Student to the first profile step.

Alternative Paths	The Student may request a fresh verification email, which replaces any existing token. No more than three such requests are accepted in five minutes.
Postcondition	The email address is recorded as verified and the token has been consumed.
Exception Paths	An unrecognised token is rejected. A token older than twenty-four hours is rejected and deleted, and the Student is invited to request a new one.
Other	Verification is not enforced before the Student may continue, so an email delivery problem never blocks the journey. Only the hash of the token is stored.

3.2.7 Complete Student Profile

Use Case Name	Complete Student Profile
XRef	Section 2.2.2, Complete Student Profile
Trigger	The Student submits any step of the profile.
Precondition	The Student holds a valid session and a Student profile record exists.
Basic Path	1. The System presents the current step with any previously saved values filled in. 2. The Student completes the step and submits it. 3. The System validates the step against the schema that governs that step alone. 4. The System saves the values, recalculates profile strength and advances the stored position. 5. The System presents the next step, and after the last step presents the review screen. 6. On confirmation the System marks the profile complete and opens the Dashboard.
Alternative Paths	A learner in Grade 10 or Grade 11 is not asked for a tertiary qualification level. A Student who is not currently enrolled is not asked for a current institution, programme or year of study. The Student may leave at any point using Save and Exit and resume later at the same step.
Postcondition	The profile holds the supplied information, profile strength has been recalculated, and once the review step is confirmed the Student may see matched opportunities.
Exception Paths	A step that fails validation is redisplayed with the specific field errors and nothing is saved. The review step is refused if no education stage has been recorded or no study preference exists.
Other	Each step is validated on the server against its own schema, so a request cannot write a field belonging to a different step. Registration deliberately does not ask for a residential address,

	identity number, banking details or financial records; such information is requested only when a specific application needs it.
--	---

3.2.8 Manage Study Preferences

Use Case Name	Manage Study Preferences
XRef	Section 2.2.2, Manage Study Preferences
Trigger	The Student submits the study preferences form.
Precondition	The Student holds a valid session and a Student profile exists.
Basic Path	1. The System presents the current preferences and the catalogue of courses and institutions. 2. The Student adds, edits, removes or reorders preferences. 3. The System confirms that every course and institution exists in the catalogue. 4. The System confirms that no two preferences name the same course and institution pair. 5. The System replaces the stored list in a single transaction and renumbers it from one. 6. The System recalculates profile strength.
Alternative Paths	A preference that is missing either its course or its institution is not saved; the System tells the Student that the incomplete rows will be discarded.
Postcondition	The Student holds between one and six numbered study preferences, each a valid course and institution pair.
Exception Paths	A seventh preference is refused. A duplicate course and institution pair is refused. A course or institution that is not in the catalogue is refused.
Other	A study preference is stored as a pair, never as two unrelated lists, because a bursary that funds a course at one institution is not necessarily open to the same course elsewhere. The Matching Engine scores both halves of the same pair, so a supported course at an unsupported institution does not count as a full match. The limit of six is enforced on the server.

3.2.9 View Matched Opportunities

Use Case Name	View Matched Opportunities
XRef	Section 2.2.2, View Matched Opportunities
Trigger	The Student opens the Student Dashboard.
Precondition	The Student has completed onboarding and holds at least one study preference.
Basic Path	1. The System loads the Student's profile projection. 2. The System loads every funding programme whose status is published and whose closing date has not passed.

	3. The Matching Engine scores each programme across the six weighted criteria. 4. The System orders the results by score, highest first. 5. The System presents the leading matches together with the reasons that produced each score.
Alternative Paths	If the Student has already applied to a programme it is marked as applied and excluded from the leading matches.
Postcondition	The Student has been shown their ranked matches and the reasons behind them.
Exception Paths	If no programme is currently open, or the profile is too sparse to score, the System explains this and directs the Student to complete the sections that would produce matches.
Other	A percentage is never presented on its own. Every score is accompanied by the criteria that were met, failed, or could not be evaluated.

3.2.10 Search and Filter Opportunities

Use Case Name	Search and Filter Opportunities
XRef	Section 2.2.2, Search and Filter Opportunities
Trigger	The Student enters a search term or changes a filter on the Opportunities page.
Precondition	The Student has completed onboarding.
Basic Path	1. The System presents the scored opportunities together with the filter controls. 2. The Student sets one or more filters. 3. The System applies the filters to the scored set. 4. The System presents the result count and the first page of results. 5. The Student pages through the results or changes the ordering.
Alternative Paths	The Student may clear all filters and return to the full list at any point.
Postcondition	The Student is presented with the opportunities that satisfy the chosen filters.
Exception Paths	If no opportunity satisfies the filters, the System says so and offers to clear them.
Other	Scoring is performed on the server and results are paged, so the browser never receives the whole catalogue at once.

3.2.11 View Opportunity Details

Use Case Name	View Opportunity Details
XRef	Section 2.2.2, View Opportunity Details
Trigger	The Student selects an opportunity.
Precondition	The programme is published and the Student has completed onboarding.

Basic Path	1. The System retrieves the programme with its eligibility rule, supported courses and supported institutions. 2. The Matching Engine scores the Student against that programme. 3. The System presents the programme detail with the score and the per-criterion explanation. 4. The System offers to start an application, or shows the existing application if one has been submitted.
Alternative Paths	If the Student has already submitted an application, the System offers to view that application instead of starting a new one. If a draft exists, the System offers to continue it.
Postcondition	The Student understands what the programme funds, who is eligible and how well they match.
Exception Paths	A programme that is still a draft, or that belongs to no published set, cannot be opened by a Student and returns a not found response.
Other	The explanation shown to the Student is produced by the same engine that produces the eligibility verdict shown to the funder, so the two can never contradict one another.

3.2.12 Apply for Funding

Use Case Name	Apply for Funding
XRef	Section 2.2.2, Apply for Funding
Trigger	The Student selects Apply Now on an opportunity.
Precondition	The programme is published, its closing date has not passed, and the Student has not already submitted an application to it.
Basic Path	1. The System presents the Student's stored profile without asking for it again. 2. The Student answers the programme's questions and attaches documents. 3. The Student confirms the declaration and submits. 4. The System validates the payload, including every required question. 5. The Matching Engine scores the Student and the Eligibility Service returns a verdict. 6. The System stores the application with the score, the verdict and the reasons, and sets the status to Submitted. 7. The System notifies the Student and opens the application tracking page.
Alternative Paths	The Student may save a draft at any point and return to it later. A draft may be edited freely until it is submitted.

Postcondition	An application exists with the status Submitted, carrying the match score and eligibility verdict as they stood at submission.
Exception Paths	An application to a closed programme is refused. A second application to the same programme is refused. Submission without confirming the declaration, or with a required question unanswered, is refused and the offending fields are named. A document that does not belong to the Student cannot be attached.
Other	The score is stored at submission so that the funder always sees the match as it stood when the Student applied, even if the Student later edits their profile.

3.2.13 Track Application

Use Case Name	Track Application
XRef	Section 2.2.2, Track Application
Trigger	The Student opens My Applications or selects an application.
Precondition	The application belongs to this Student.
Basic Path	1. The System retrieves the applications belonging to this Student. 2. The System presents them with their statuses and key dates. 3. The Student selects one. 4. The System presents the full application, its timeline and the recorded match.
Alternative Paths	If the application is still a draft, the System offers to continue it. If the funder has asked for more information, the System presents the request prominently.
Postcondition	The Student knows the current status of the application and what, if anything, is expected of them.
Exception Paths	A request for an application that belongs to another Student returns a not found response.
Other	The statuses are Draft, Submitted, Under Review, Documents Required, Shortlisted, Approved and Unsuccessful.

3.2.14 Manage Documents

Use Case Name	Manage Documents
XRef	Section 2.2.2, Manage Documents
Trigger	The Student submits the upload form, or selects to remove a document.
Precondition	The Student holds a valid session.
Basic Path	1. The Student selects a document type and a file. 2. The System confirms the file is a PDF or an image and is no larger than five megabytes. 3. The System writes the file to storage under a key belonging to this Student. 4. The System records the document and recalculates profile strength.

Alternative Paths	The Student may remove a document, which deletes both the record and the stored file. Applications already submitted keep the copy that was attached to them.
Postcondition	The document is stored against the Student's profile and is available to attach to applications.
Exception Paths	A file of an unsupported type, an empty file, or a file larger than five megabytes is refused. More than twenty uploads in ten minutes are refused.
Other	Documents are not required in order to register. They are stored outside the public web root and are released only to the Student who owns them, or to a funder that has received an application the document is attached to.

3.2.15 Record Subject Results

Use Case Name	Record Subject Results
XRef	Section 2.2.2, Record Subject Results
Trigger	The Student adds, edits, reorders or deletes a result on the My Results page.
Precondition	The Student has signed in and has completed onboarding.
Basic Path	1. The Student selects Add a result. 2. The System presents the subject list appropriate to the Student's education stage, school subjects for a school learner and modules for a tertiary student. 3. The Student selects a subject, or types one that the list does not hold. 4. The Student enters the academic year, and optionally the mark and the term. 5. The System validates that the mark, if given, is a whole percentage between 0 and 100. 6. The System stores the result and adds any typed subject to the catalogue as a custom entry attributed to the Student. 7. The System lists, beneath the result, any published funding programme whose stated subject minimum this result meets.
Alternative Paths	In step 4 the Student may leave the mark empty. Knowing which subject is being taken, without yet knowing the mark, is a real state and is stored as such; it is not recorded as a zero.
Postcondition	The result is stored against the Student's profile and is available to the Matching Engine.
Exception Paths	If the Student records a second result for the same subject and year, the System refuses the duplicate and offers to edit the existing result instead. If a typed subject name is too short or contains no letters, the System refuses it.
Other	The page uses the vocabulary of the Student's education stage: a

	Grade 12 learner records subjects, a university student records modules. Where two results exist for one subject in different years, the Matching Engine uses the later one.
--	--

3.2.16 Browse All Bursaries

Use Case Name	Browse All Bursaries
XRef	Section 2.2.2, Browse All Bursaries
Trigger	The Student selects All Bursaries from the navigation.
Precondition	The Student has signed in.
Basic Path	1. The System retrieves every funding programme with the status Published. 2. The System determines the display status of each from its dates and its recorded availability: Open, Closing Soon, Upcoming, Closed or Status Unknown. 3. The System orders the list with opportunities accepting applications first and closed ones last. 4. The System shows the count against each status so the Student can see how much of the directory each represents. 5. The Student applies a search term or a filter, and the System reduces the list accordingly.
Alternative Paths	If the Student applies filters that nothing satisfies, the System says so and offers to clear the filters, rather than presenting an empty page.
Postcondition	The Student has seen the directory as it stands, and the directory is unchanged.
Exception Paths	If the directory holds no opportunities at all, the System says that none have been collected yet and explains where they come from. It does not present example opportunities in their place.
Other	This use case is deliberately distinct from View Matched Opportunities. All Bursaries answers the question of what exists and is never narrowed to the Student's own profile; View Matched Opportunities answers what fits this Student. A closed opportunity remains listed and is marked Closed, so that a Student can see who funds what and when to return.

3.2.17 Generate Motivational Letter

Use Case Name	Generate Motivational Letter
XRef	Section 2.2.2, Generate Motivational Letter
Trigger	The Student selects Write a letter.
Precondition	The Student has signed in and has completed onboarding.
Basic Path	1. The Student chooses a published funding programme, or names a bursary the system does not hold. 2. The System presents seven optional questions covering motivation,

	field of study, goals, circumstances, achievements, suitability and the effect of the funding. 3. The Student answers any number of them. 4. The System shows the profile facts that will appear in the letter, and lists anything missing that would strengthen it. 5. The System composes the letter from the stated facts and the Student's own sentences. 6. The System stores the draft together with the answers it was built from. 7. The Student edits the text, marks the letter ready, copies it, or downloads it as a PDF.
Alternative Paths	The Student may rewrite the letter from changed answers. Where the Student has already edited the text, the System warns that rewriting replaces those edits and requires confirmation.
Postcondition	A letter is stored against the Student's profile, carrying the answers it was composed from.
Exception Paths	If the Student names a bursary the system does not hold but supplies no organisation, the System refuses to proceed. If the Student's profile carries nothing usable, the System says so and composes from the answers alone.
Other	Every sentence in a generated letter comes either from a fact already on the Student's profile or from a sentence the Student wrote. Where there is no material for a paragraph, the paragraph is omitted and the Student is told what to add; the System does not compose a plausible substitute. The letter is the Student's own statement and is never sent anywhere by the System.

3.2.18 Auto-Fill Application Forms

Use Case Name	Auto-Fill Application Forms
XRef	Section 2.2.2, Auto-Fill Application Forms
Trigger	The Student starts an auto-fill job.
Precondition	The Student holds a completed application form as a PDF.
Basic Path	1. The Student uploads a completed form. 2. The System reads the form's fields, taking their values from the form's own field definitions where it has them and from the page text where it does not. 3. The Student uploads the blank forms to be filled. 4. The System identifies the fields on each blank form and matches them against what it read. 5. The System writes the matched values into each form and records,

	for every field, the value, its confidence and the label it came from. 6. The System presents each filled form for review beside the fields it wrote. 7. The Student corrects any field, and the System rewrites that form. 8. The Student downloads the completed forms individually or together.
Alternative Paths	A form the System cannot match confidently is left blank and flagged for the Student's attention rather than filled with a guess.
Postcondition	Each blank form has been written with the values the System could establish, and the Student has been shown every value it wrote.
Exception Paths	If the completed form is password protected, or is a scan with no readable text, the System refuses the job and says why. If one blank form cannot be read, the remaining forms are still filled; the failure is reported against that form alone.
Other	The System never invents a value. A field it cannot establish is left empty and marked as needing attention, because a blank field a Student can complete is better than a populated field that is wrong. Signatures are never produced.

3.2.19 Respond to Information Request

Use Case Name	Respond to Information Request
XRef	Section 2.2.2, Respond to Information Request
Trigger	The Student opens an application against which a Funder has raised a request.
Precondition	An open information request exists against one of the Student's submitted applications.
Basic Path	1. The System presents the request above everything else on the application page, because it is the only item the Student can act on. 2. The System lists each thing asked for, and the deadline if the Funder set one. 3. The Student attaches one of their existing documents to an item, or uploads a new one. 4. The System records the attachment against that item and marks it supplied. 5. The Student submits the response once every item has been answered. 6. The System marks the request responded, notifies the Funder, and records the event in the audit log.
Alternative Paths	The Student may detach a document before submitting and attach a different one. The Student may respond to a request after its

	deadline; the System records the date and leaves the decision to the Funder.
Postcondition	The request is marked as responded and the attached documents are readable by the requesting Organisation.
Exception Paths	If the Student submits with nothing attached, the System refuses and says which items are outstanding. If the Funder has withdrawn the request, the System says so and does not accept a response.
Other	A document attached to a request becomes readable by that Organisation alone, and only for the application the request belongs to.

3.2.20 Register Corporate Account

Use Case Name	Register Corporate Account
XRef	Section 2.2.3, Register Corporate Account
Trigger	The Corporate User submits the organisation registration form.
Precondition	The visitor is not signed in and does not hold an account under the supplied email address.
Basic Path	1. The System validates the name fields, the work email address, the mobile number and the password. 2. The System confirms that no account exists for the email address. 3. The System hashes the password and creates the User record with the role Corporate. 4. The System creates an Organisation record and links a Corporate profile to it. 5. The System creates a session, sends a verification email and opens the Organisation Details step.
Alternative Paths	None.
Postcondition	A Corporate account, an Organisation record and a linked Corporate profile exist, and the Corporate User is signed in.
Exception Paths	If the email address is already registered the registration is refused. Any field that fails validation is returned with a specific message and nothing is created.
Other	The Organisation record is created immediately, so that every later action can be scoped to it. The Organisation is named properly in the next step.

3.2.21 Complete Organisation Profile

Use Case Name	Complete Organisation Profile
XRef	Section 2.2.3, Complete Organisation Profile
Trigger	The Corporate User submits any step of the Organisation profile.
Precondition	The Corporate User holds a valid session and a Corporate profile exists.
Basic Path	1. The System presents the current step with any saved values filled in.

	2. The Corporate User completes the step and submits it. 3. The System validates the step and writes the values against this Organisation only. 4. The System advances the stored position and presents the next step. 5. On confirmation the System marks the profile complete and opens the Dashboard.
Alternative Paths	The Current Process step is optional and may be submitted empty.
Postcondition	The Organisation profile is complete and the Corporate User may create funding programmes.
Exception Paths	An Organisation name already used by another Organisation is refused. A step that fails validation is redisplayed with field errors. No more than three challenges may be selected.
Other	Every write is scoped to the Organisation attached to the signed-in Corporate User, so one Organisation can never modify another's record.

3.2.22 Create Funding Programme

Use Case Name	Create Funding Programme
XRef	Section 2.2.3, Create Funding Programme
Trigger	The Corporate User selects Create New Programme.
Precondition	The Corporate User holds a valid session and the Organisation profile is complete.
Basic Path	1. The System presents the programme builder at the first of its five steps. 2. The Corporate User completes each step in turn. 3. The System validates the whole programme, including that the closing date falls after the opening date. 4. The System confirms that every selected institution and course exists in the catalogue. 5. The System creates the programme, its eligibility rule, its supported lists and its questions in a single transaction. 6. The System opens the new programme.
Alternative Paths	The Corporate User may save the programme as a draft at any point and complete it later. A draft is not visible to students.
Postcondition	A funding programme exists, together with its eligibility rule, supported courses and institutions, required documents and questions.
Exception Paths	A closing date on or before the opening date is refused. A programme with no funding coverage selected is refused. A selected institution or course that is no longer in the catalogue is refused. The programme is written as one transaction, so it can never exist

	without its eligibility rule.
Other	What the Corporate User configures here is exactly what the Matching Engine and the Eligibility Service evaluate; the criteria are stored as structured data rather than as prose.

3.2.23 Define Eligibility Criteria

Use Case Name	Define Eligibility Criteria
XRef	Section 2.2.3, Define Eligibility Criteria
Trigger	The Corporate User reaches the Eligibility Criteria step of the programme builder.
Precondition	A funding programme is being created or edited by a Corporate User of the owning Organisation.
Basic Path	1. The System presents each criterion with the catalogue values available for it. 2. The Corporate User sets the criteria that apply and leaves the rest empty. 3. The System validates the values, including that any minimum average lies between zero and one hundred. 4. The System stores the criteria as an eligibility rule attached to the programme.
Alternative Paths	Any criterion left empty places no restriction on that dimension, and the System states this beneath each control.
Postcondition	The programme carries an eligibility rule that the Matching Engine and the Eligibility Service can evaluate.
Exception Paths	A minimum average outside the range zero to one hundred is refused.
Other	This use case is part of Section 3.2.17, Create Funding Programme. The criteria drive both the match score shown to students and the eligibility verdict shown to the funder.

3.2.24 Publish Funding Programme

Use Case Name	Publish Funding Programme
XRef	Section 2.2.3, Publish Funding Programme
Trigger	The Corporate User selects to publish, close or reopen a programme.
Precondition	The programme belongs to the Corporate User's Organisation.
Basic Path	1. The System confirms the programme belongs to this Organisation. 2. The System confirms that a programme being published still has a future closing date. 3. The System updates the programme status. 4. The System records the change in the audit log.
Alternative Paths	Closing a programme withdraws it from student view but leaves applications already received untouched. A closed programme may

	be reopened.
Postcondition	The programme status reflects the change, and student visibility follows from it.
Exception Paths	Publishing a programme whose closing date has passed is refused, and the Corporate User is asked to set a future date first. A programme belonging to another Organisation returns a not found response.
Other	This use case extends Section 3.2.17, Create Funding Programme.

3.2.25 Import Applications

Use Case Name	Import Applications
XRef	Section 2.2.3, Import Applications
Trigger	The Corporate User uploads one or more application forms against a funding programme.
Precondition	The Corporate User's Organisation owns the funding programme the applications are being imported into.
Basic Path	1. The System stores every uploaded file before reading any of it, so that the batch survives an interruption. 2. The System expands any archive, treating each folder as one applicant and the documents filed beside a form as that applicant's supporting documents. 3. The System reads each application form and records every field it extracts, with the confidence of the reading and the label it was read from. 4. The System looks for an existing Student account matching the applicant on a verified identifier. 5. The System evaluates eligibility and calculates a match score against that programme's own criteria. 6. The System counts the supporting documents supplied against those the programme requires. 7. The System compares each applicant against the rest of the batch and against the applications the programme already holds, and flags any duplicate. 8. The System reports the batch as ready for review, giving the number ready, the number needing review, the number flagged as duplicates and the number that could not be read.
Alternative Paths	The Corporate User may leave the page while the batch is processing and return to it later; the progress is held against the batch rather than in the browser.

Postcondition	Every uploaded file has reached a conclusion and is held against the batch. No applicant record has yet been created.
Exception Paths	A file that is password protected, damaged, or a scan with no readable text is kept and reported with the reason rather than discarded. A file that is not a PDF is refused with the reason. One unreadable file does not stop the remainder of the batch.
Other	Importing creates nothing on its own: the applications are created only when a Corporate User confirms the batch, in Review Imported Applications. Applications created this way are marked as externally imported wherever they appear, so an imported application is never mistaken for one a Student submitted here.

3.2.26 Review Imported Applications

Use Case Name	Review Imported Applications
XRef	Section 2.2.3, Review Imported Applications
Trigger	The Corporate User opens a batch that has finished processing.
Precondition	The batch belongs to the Corporate User's own Organisation and has been processed.
Basic Path	1. The System lists the files in the batch by outcome. 2. The Corporate User opens one and sees every extracted field with its confidence, the label it was read from, and the original document. 3. The Corporate User corrects a field where the reading was wrong; the System keeps both the extracted value and the correction. 4. The Corporate User decides, for each flagged duplicate, whether to import it anyway or to leave it out. 5. The Corporate User confirms the import. 6. The System creates one application for each file marked for import, attaching it to the existing Student where one was identified and creating an external applicant record where one was not. 7. The System records how many were imported and how many were left out, and writes the confirmation to the audit log.
Alternative Paths	The Corporate User may confirm a batch in which some files remain unreviewed; those flagged as needing review or as duplicates are left out unless explicitly marked for import.
Postcondition	Applications exist in the programme for the files the Corporate User chose to import, each carrying its match score, its eligibility verdict and a link to the document it came from.
Exception Paths	If an application cannot be created, the remaining ones are still created and the failure is reported against that file. Confirming a batch a second time does not create the applications again.
Other	The original document is never replaced. The uploaded form, the values extracted from it and any correction a reviewer made are all

	retained, so a funding decision can be traced back to what the applicant actually submitted.
--	--

3.2.27 View Applications

Use Case Name	View Applications
XRef	Section 2.2.3, View Applications
Trigger	The Corporate User opens the Applications page or changes a filter.
Precondition	The Corporate User holds a valid session and belongs to an Organisation.
Basic Path	1. The System retrieves the applications belonging to this Organisation only. 2. The System applies any search terms and filters in the database. 3. The System counts the matching applications and retrieves one page of them. 4. The System presents each applicant with their match score, eligibility verdict and status.
Alternative Paths	The Corporate User may select applicants and shortlist, select or remove them in a single action.
Postcondition	The Corporate User is presented with the applications that satisfy the chosen filters.
Exception Paths	If no application satisfies the filters the System says so. An Organisation is never shown an application submitted to another Organisation's programme.
Other	Filtering and paging are performed in the database, so a programme with several thousand applicants loads in constant time.

3.2.28 Review Applicant

Use Case Name	Review Applicant
XRef	Section 2.2.3, Review Applicant
Trigger	The Corporate User selects an applicant.
Precondition	The application was submitted to a programme belonging to this Organisation.
Basic Path	1. The System retrieves the application, confirming it belongs to this Organisation. 2. The Eligibility Service evaluates the applicant against the programme's criteria as they stand now. 3. The System presents the applicant across the overview, eligibility, documents and answers views. 4. The System offers the review decisions available for this applicant.
Alternative Paths	The Corporate User may open a document the applicant attached, which the System releases only because this Organisation received the application.

Postcondition	The Corporate User has the information needed to make a decision on the applicant.
Exception Paths	An application belonging to another Organisation returns a not found response.
Other	Eligibility is recalculated when the applicant is opened, so a reviewer always sees an assessment against the programme's current criteria rather than a stale snapshot. Information the applicant has not supplied is reported as pending verification rather than counted as a failure.

3.2.29 Shortlist Applicant

Use Case Name	Shortlist Applicant
XRef	Section 2.2.3, Shortlist Applicant
Trigger	The Corporate User selects Add to Shortlist, on one applicant or on several at once.
Precondition	The application was submitted to a programme belonging to this Organisation.
Basic Path	1. The System confirms every named application belongs to this Organisation. 2. The System records a shortlist entry for each and sets the application status to Shortlisted. 3. The System notifies each applicant. 4. The System records the action in the audit log.
Alternative Paths	Removing an applicant from the shortlist returns the application to Under Review and withdraws the shortlist entry.
Postcondition	The applicant holds the status Shortlisted and appears on the Organisation's shortlist.
Exception Paths	An application belonging to another Organisation is refused. Selecting no applicant is refused.
Other	This use case extends Section 3.2.21, Review Applicant. Applicants may be shortlisted individually or in bulk from the applications list.

3.2.30 Request Information from Applicant

Use Case Name	Request Information from Applicant
XRef	Section 2.2.3, Request Information from Applicant
Trigger	The Corporate User selects Request Information.
Precondition	The application was submitted to a programme belonging to this Organisation.
Basic Path	1. The System presents a field for the reviewer's message. 2. The Corporate User writes the message and confirms. 3. The System sets the status to Documents Required and stores the message against the application. 4. The System notifies the applicant.

Alternative Paths	None.
Postcondition	The application holds the status Documents Required and the applicant has been told what is needed.
Exception Paths	A request with no message is refused. An application belonging to another Organisation is refused.
Other	This use case extends Section 3.2.21, Review Applicant.

3.2.31 Select Beneficiary

Use Case Name	Select Beneficiary
XRef	Section 2.2.3, Select Beneficiary
Trigger	The Corporate User selects Move to Selected, or selects Reject.
Precondition	The application was submitted to a programme belonging to this Organisation.
Basic Path	1. The System confirms every named application belongs to this Organisation. 2. The System sets the status to Approved and records the decision date. 3. The System records or updates the shortlist entry as selected. 4. The System notifies each applicant.
Alternative Paths	Rejecting an applicant sets the status to Unsuccessful, records any reason given and notifies the applicant. An applicant may also be returned to Under Review.
Postcondition	The applicant holds a final status of Approved or Unsuccessful and has been notified.
Exception Paths	An application belonging to another Organisation is refused.
Other	This use case extends Section 3.2.21, Review Applicant. Selection may be performed on one applicant or on several at once from the shortlist.

3.2.32 View Reports

Use Case Name	View Reports
XRef	Section 2.2.3, View Reports
Trigger	The Corporate User opens the Reports page.
Precondition	The Corporate User holds a valid session and belongs to an Organisation.
Basic Path	1. The System groups this Organisation's applications by status, by province and by institution. 2. The System computes the eligibility rate and conversion rate for each programme. 3. The System presents the figures as totals, proportions and tables.
Alternative Paths	None.
Postcondition	The Corporate User is presented with the Organisation's pipeline and reach.

Exception Paths	If the Organisation has received no applications, the System says so rather than presenting empty charts.
Other	Every figure is computed from this Organisation's own applications. Eligibility counts read the verdict recorded when each application was submitted.

3.2.33 Manage Institution and Course Catalogue

Use Case Name	Manage Institution and Course Catalogue
XRef	Section 2.2.4, Manage Institution and Course Catalogue
Trigger	The Administrator creates, edits, links or retires a catalogue entry.
Precondition	The Administrator has signed in.
Basic Path	1. The Administrator enters the name of an institution or course, with its type or field. 2. The System reduces the name to a canonical form, ignoring case, punctuation and spacing, and compares it against the catalogue. 3. The System refuses an entry that would duplicate an existing one, naming the entry it matched. 4. The System stores the entry with both the name as typed and its canonical form. 5. The Administrator records the courses offered at an institution, so that a Student choosing that institution is offered the courses it actually teaches.
Alternative Paths	The Administrator may retire an entry instead of deleting it. A retired entry is no longer offered for new selections but continues to describe the Students and programmes already pointing at it.
Postcondition	The catalogue reflects the change, and no two entries describe the same institution or course.
Exception Paths	If the Administrator attempts to retire an institution or course that Students or funding programmes currently point at, the System completes the retirement but states how many records are affected, since deleting it outright would leave those records without an institution or course.
Other	Canonical matching is what stops the same university appearing twice under two spellings, which would split its Students across two entries and weaken every match calculated for them.

3.2.34 Manage User Accounts

Use Case Name	Manage User Accounts
XRef	Section 2.2.4, Manage User Accounts
Trigger	The Administrator applies an action to a user account.
Precondition	The Administrator has signed in, and the account is not the Administrator's own.
Basic Path	1. The Administrator searches by name or email address.

	- The Administrator chooses to suspend, restore, or require a password reset. - The System requires a reason of at least ten characters. - The System applies the change, ends every session the account holds, and writes the action, the reason and the Administrator to the audit log.
Alternative Paths	An account required to reset its password may sign in only to set a new one, and is returned to the reset page until it does.
Postcondition	The account's status reflects the action, and the action is recorded against the Administrator who took it.
Exception Paths	The System refuses an action against the Administrator's own account, so that the last administrator cannot lock themselves out. It refuses an action with no reason.
Other	A suspended account cannot sign in and is told to contact support. Suspension does not delete the account or anything belonging to it.

3.2.35 Moderate Organisations and Programmes

Use Case Name	Moderate Organisations and Programmes
XRef	Section 2.2.4, Moderate Organisations and Programmes
Trigger	The Administrator suspends or restores an Organisation or a funding programme.
Precondition	The Administrator has signed in.
Basic Path	- The Administrator selects the Organisation or funding programme. - The Administrator suspends it and states the reason. - The System sets the status to Suspended and removes it from everything a Student sees. - The System records the action, the reason and the Administrator in the audit log.
Alternative Paths	The Administrator may restore a suspended programme, which returns it to the status it held before.
Postcondition	The Organisation or programme is withdrawn from the Student-facing system and the action is recorded.
Exception Paths	The owning Corporate User cannot publish a suspended programme back out of that state; an attempt is refused and explained, since otherwise the suspension could be undone by the party it was applied to.
Other	Applications already submitted to a suspended programme are retained. Suspension withdraws the programme from view; it does not erase what has happened under it.

3.2.36 Manage Bursary Data Sources

Use Case Name	Manage Bursary Data Sources
XRef	Section 2.2.4, Manage Bursary Data Sources
Trigger	The Administrator starts a collection run, or opens a previous one.
Precondition	The Administrator has signed in, and at least one source has been authorised.
Basic Path	1. The System lists each registered source, whether it is authorised, and the outcome of the last run against it. 2. The Administrator starts a run. 3. The System reads each authorised source's robots file and does not fetch any page that file disallows. 4. The System reads the source's pages at the rate the source asks for, extracts each opportunity, and records the address it was read from. 5. The System rejects anything that does not describe a real opportunity, merges entries that describe the same opportunity, and stores the remainder. 6. The System records the run with the number found, created, updated, merged and rejected.
Alternative Paths	The Administrator may inspect a run that found nothing, which is recorded distinctly from a run that could not reach its sources.
Postcondition	The directory reflects what the authorised sources carried at the time of the run, and every opportunity records the address it was read from and the date it was last confirmed.
Exception Paths	If a source's robots file disallows collection, the run records the source as not read rather than reading it anyway. If a source cannot be reached, the run is recorded as blocked, which is distinct from a run that succeeded and found nothing.
Other	A source is read only after an Administrator has authorised it by name. Registering a source in the system does not authorise reading it.

3.2.37 View Audit Trail

Use Case Name	View Audit Trail
XRef	Section 2.2.4, View Audit Trail
Trigger	The Administrator opens the audit trail.
Precondition	The Administrator has signed in.
Basic Path	1. The System lists each recorded action with the time, the user, the entity affected and any recorded reason. 2. The Administrator narrows the list by action, user or period.
Alternative Paths	None.

Postcondition	The audit trail is unchanged; it can be read but not edited.
Exception Paths	If no actions match the filters, the System says so rather than presenting an empty table.
Other	The audit log is append-only. Account actions, programme suspensions, application decisions, information requests and import confirmations are all recorded, so that a decision affecting a Student can be accounted for afterwards.

3.3 Detailed Non-Functional Requirements

3.3.1 Logical Structure of the Data

Figure 49 presents the logical structure of the data stored by Bursary-Bridge, drawn as a Unified Modeling Language class diagram in accordance with [2].

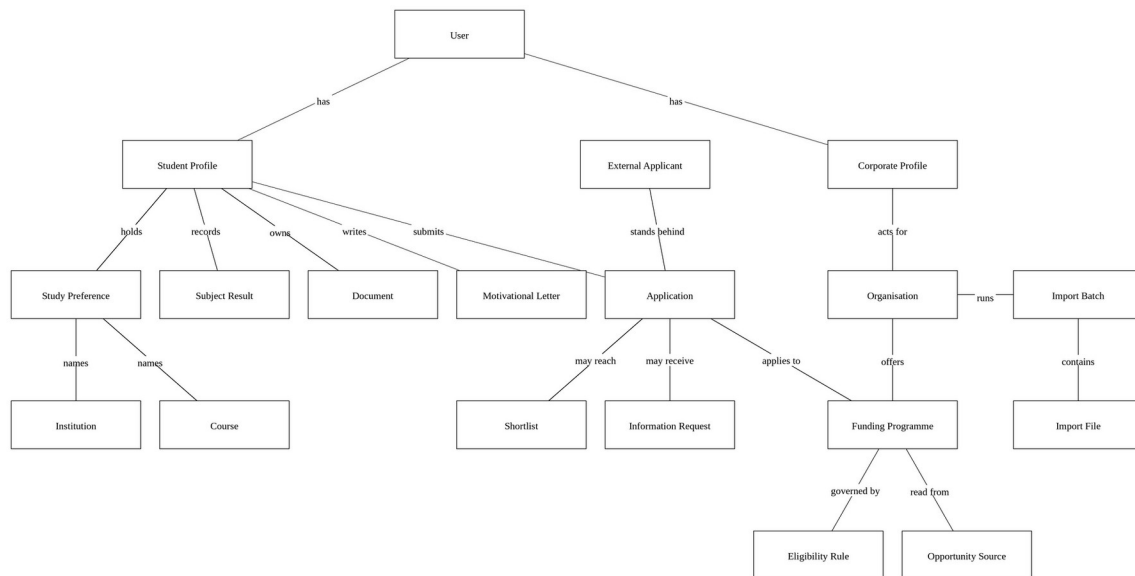


Figure 49 - Logical Structure of the Bursary-Bridge Data

Tables 2 to 17 describe each of the entities in Figure 49 in turn, giving the data items it holds, the type of each and the constraints that apply.

Table 2 - User Data Entity

Data Item	Type	Description	Comment
Email Address	Text	The address used to sign in	Unique across the system
Password Hash	Text	The bcrypt hash of the password	The password itself is never stored
Role	Enumeration	Student, Corporate or Administrator	Determines which part of the system is reachable
First Name	Text	Given name	
Last Name	Text	Family name	

Mobile Number	Text	South African mobile number	
Email Verified	Date	When the address was confirmed; null if not	

Table 3 - Student Profile Data Entity

Data Item	Type	Description	Comment
User	Pointer	The User entity	One profile per User
Education Stage	Enumeration	Grade 10 through postgraduate, or planning to study	
Qualification Level	Enumeration	Certificate through doctoral degree	Not asked of school learners
Study Status	Enumeration	Enrolled, accepted, applied, planning or returning	
Academic Average	Integer	Latest average as a percentage	Null if the Student does not know it
Household Income	Enumeration	Annual household income band	A band, never an exact figure
Citizenship	Enumeration	Citizen, permanent resident or other	
Province	Enumeration	One of the nine South African provinces	
Profile Strength	Integer	Completeness as a percentage	Recalculated after every change

Table 4 - Study Preference Data Entity

Data Item	Type	Description	Comment
Student Profile	Pointer	The Student entity	
Preference Number	Integer	Position in the Student's ordered list	One to six
Programme	Pointer	The course entity	Paired with the institution below
Institution	Pointer	The institution entity	Paired with the course above

Table 5 - Organisation Data Entity

Data Item	Type	Description	Comment
Name	Text	Registered name of the Organisation	Unique across the system
Type	Enumeration	Corporation, foundation, NGO or other	
Industry	Enumeration	Sector in which the Organisation operates	
Website	Text	Public web address	Optional
Application Volume	Enumeration	Applications received each year	

Table 6 - Funding Programme Data Entity

Data Item	Type	Description	Comment
Organisation	Pointer	The owning Organisation	Scopes every access to the programme
Name	Text	Name of the programme	
Funding Type	Enumeration	Bursary, scholarship, grant or other	
Coverage	Enumeration	What the programme pays for	May be several
Open Date	Date	When applications open	
Closing Date	Date	When applications close	
Status	Enumeration	Draft, published or closed	Only published programmes are visible to students

Table 7 - Eligibility Rule Data Entity

Data Item	Type	Description	Comment
Funding Programme	Pointer	The programme the rule governs	One rule per programme
Minimum Academic Average	Integer	Lowest acceptable average	Null if no minimum applies
Qualification Levels	Enumeration	Levels that qualify	Empty means all levels
Years of Study	Integer	Years that qualify	Empty means any year
Citizenship	Enumeration	Citizenship conditions	Empty means no condition
Maximum Household Income	Enumeration	Highest acceptable income band	Null if no condition
Provinces	Enumeration	Provinces that qualify	Empty means all provinces
Required Documents	Enumeration	Documents applicants must provide	

Table 8 - Application Data Entity

Data Item	Type	Description	Comment
Student Profile	Pointer	The applying Student	
Funding Programme	Pointer	The programme applied to	One application per Student per programme
Organisation	Pointer	The Organisation that receives it	Scopes funder access
Status	Enumeration	Draft through to approved or unsuccessful	
Match Score	Integer	The score at the moment of	Held so the funder

		submission	sees a stable figure
Eligibility Outcome	Enumeration	Eligible, not eligible or pending verification	Recorded at submission
Answers	Text	Answers to the programme's questions	
Submitted	Date	When the application was submitted	Null while it remains a draft

Table 9 - Subject Result Data Entity

Data Item	Type	Description	Comment
Student Profile	Pointer	The Student the result belongs to	
Subject	Pointer	The catalogue subject or module	May be an entry the Student typed
Percentage	Integer	The mark, as a whole percentage	Null when the Student does not yet have it
Year	Integer	The academic year the result belongs to	
Level	Enumeration	School or tertiary	Follows the Student's education stage

Table 10 - Motivational Letter Data Entity

Data Item	Type	Description	Comment
Student Profile	Pointer	The Student who wrote it	
Funding Programme	Pointer	The programme the letter is for	Null for a bursary the system does not hold
Content	Text	The letter as it stands	Editable by the Student
Answers	Structured	The Student's answers the letter was composed from	Retained so the letter can be traced to what the Student said
Status	Enumeration	Draft or Ready	
Edited by Student	Boolean	Whether the Student has changed the text	Governs the warning shown before a rewrite

Table 11 - Information Request Data Entity

Data Item	Type	Description	Comment
Application	Pointer	The application the request concerns	
Organisation	Pointer	The Organisation that raised it	
Message	Text	The Funder's own wording	Shown to the Applicant unaltered
Deadline	Date	When the Funder needs a response	Optional

Status	Enumeration	Open, Responded or Cancelled	
Items	Collection	Each thing asked for, and the document supplied against it	

Table 12 - Application Import Batch Data Entity

Data Item	Type	Description	Comment
Organisation	Pointer	The Organisation importing	
Funding Programme	Pointer	The programme the applications are imported into	
Reference	Text	The batch's reference, shown in the import history	
Status	Enumeration	Uploading, Extracting, Ready for Review, Importing, Completed or Failed	
Counts	Integers	Files uploaded, processed, ready, needing review, duplicate, failed and imported	Recomputed from the files, never tracked alongside them

Table 13 - Application Import File Data Entity

Data Item	Type	Description	Comment
Batch	Pointer	The batch the file arrived in	
File Name	Text	The name the Organisation gave the file	
Storage Key	Text	Where the original document is held	The original is never replaced
Status	Enumeration	Pending, Processing, Ready, Needs Review, Duplicate, Failed, Imported or Discarded	
Failure Reason	Text	Why it could not be read, in words a reviewer can act on	Null when it was read
Match Score	Integer	The score against this batch's programme	Null when the applicant is not eligible
Eligibility Outcome	Enumeration	Eligible, Not Eligible or Pending Verification	
Duplicate Of	Pointer	The file or application it appears to duplicate	A flag for a reviewer, never acted on automatically
Extracted Fields	Collection	Each field read from the form, with its confidence, the label it came from and any correction	
Application	Pointer	The application created from this file	Set when a reviewer confirms the batch; null until then

Table 14 - External Applicant Data Entity

Data Item	Type	Description	Comment
Organisation	Pointer	The Organisation that imported the application	An applicant record belongs to one Organisation
Full Name	Text	The applicant's name as the form gave it	
Identity Number	Text	The identity number, where the form stated one	Held because Funders deduplicate on it
Institution	Pointer	The catalogue institution, where the name matched one	Null when it did not
Course	Pointer	The catalogue course, where the name matched one	Null when it did not
Academic Average	Integer	The average the form stated	Null when the form did not state one

Table 15 - Opportunity Source Data Entity

Data Item	Type	Description	Comment
Funding Programme	Pointer	The opportunity this source describes	
Address	Text	The page the opportunity was read from	Shown to Students
Name	Text	The name of the publication or funder	
Official	Boolean	Whether this is the funder's own site	An official source outranks a listing
Last Verified	Date	When the opportunity was last confirmed against this page	

Table 16 - Shortlist Data Entity

Data Item	Type	Description	Comment
Application	Pointer	The application concerned	One entry per application
Organisation	Pointer	The Organisation shortlisting	
Status	Enumeration	Shortlisted, selected or withdrawn	
Added	Date	When the applicant was shortlisted	
Selected	Date	When the applicant was selected	Null unless selected

Table 17 - Document Data Entity

Data Item	Type	Description	Comment
Student Profile	Pointer	The owning Student	Governs who may read the file

Type	Enumeration	Identity document, academic record, proof of income and so on	
File Name	Text	Name of the uploaded file	
Size	Integer	Size of the file in bytes	Limited to five megabytes
Storage Key	Text	Where the file is held in storage	Outside the public web root

Two aspects of this structure carry the weight of the design. A Study Preference holds a course and an institution together as one record, never as two unrelated lists, because a bursary that funds a course at one institution is not necessarily open to that course elsewhere. An Eligibility Rule holds a funder's criteria as typed columns rather than as prose, because those columns are exactly what the Matching Engine evaluates.

3.3.2 Security

The system holds personal information about students, including their academic results and an indication of their household income, and it holds funding criteria that organisations may regard as commercially sensitive. The following requirements apply.

- Passwords are hashed with bcrypt at cost twelve and are never written to logs.
- Sessions are random values held in an HTTP-only cookie. Only a hash of the session token is stored, so a copy of the database cannot be replayed as a login. Logging out deletes the session, and changing a password ends every other session.
- An unknown email address and an incorrect password produce identical responses, so the login endpoint cannot be used to discover which addresses hold accounts.
- Registration, login, verification email requests, document uploads and application submissions are rate limited, per address and per account.
- Every request is authorised on the server before any data is read. A Student may read only their own profile, applications and documents. A Corporate User may read only the programmes and applications belonging to their own Organisation. These rules are applied in the database query rather than in the interface.
- Uploaded documents are stored outside the public web root and are released only through an authorised handler, which admits the Student who owns the document and any Organisation that has received an application the document is attached to.
- Every field is validated on the server. The rules shown in the browser are generated from the same definitions, so the two cannot diverge.

- Changes to applications, programmes, shortlists and accounts are written to an append-only audit log.

Registration deliberately does not ask for a residential address, an identity number, a passport number, banking details, a parent's identity number or detailed financial records. Registration establishes identity and the matching profile; an application collects what a specific programme requires. Collecting less reduces both the value of the database to an attacker and the obligations of the operator under the Protection of Personal Information Act [3].



Department of Computer Science

Software Design Description

Bursary-Bridge

Supervisor
Ms Zulu

Submitted by

Student Name	Student Number
1. CPP Mchunu	202242486
2. A Nsibande	240079346
3. KN Bikile	202356797
4. SP Tshazi	240103222
5.	
6.	

September 2026

Table of Contents

1.0. Introduction.....	4
1.1. Purpose.....	4
1.2. Scope.....	4
1.3. Glossary.....	5
1.4. References.....	6
1.5. Overview of document.....	7
2.0. Deployment diagram.....	7
3.0. Architecture design.....	9
3.1. Web System Architecture.....	9
Student Portal.....	10
Corporate Portal.....	12
Administration Portal.....	13
Bursary Ingestion Pipeline.....	15
Document Extraction Engine.....	16
Application Import Pipeline.....	18
Motivational Letter Composer.....	20
Authentication and Session Module.....	21
Matching Engine.....	23
Eligibility Service.....	25
Application Programming Interface.....	26
Document Storage Service.....	27
Notification and Email Service.....	30
Bursary-Bridge Database.....	31
4.0. Data structure design.....	33
4.1. Data field types and sizes.....	33
5.0. Use case realizations.....	36
Use Case: Log In.....	37
Use Case: Complete Student Profile.....	38
Use Case: View Matched Opportunities.....	39
Use Case: Apply for Funding.....	40
Use Case: Browse All Bursaries.....	41
Use Case: Generate Motivational Letter.....	42
Use Case: Auto-Fill Application Forms.....	43
Use Case: Create Funding Programme.....	44
Use Case: Import Applications.....	45
Use Case: Review Imported Applications.....	46
Use Case: Review Applicant.....	47
Use Case: Shortlist Applicant.....	48
6.0. Interface design.....	49
7.0. Help system design.....	81
8.0. Index.....	82

Table of Figures

Figure 1 Deployment Diagram.....	8
Figure 2 Architecture Design.....	9
Figure 3 System Sequence Diagram.....	37
Figure 4 Log In Sequence Diagram.....	38
Figure 5 Complete Student Profile Sequence Diagram.....	39
Figure 6 View Matched Opportunities Sequence Diagram.....	40
Figure 7 Apply for Funding Sequence Diagram.....	41
Figure 8 Browse All Bursaries Sequence Diagram.....	42
Figure 9 Generate Motivational Letter Sequence Diagram.....	43
Figure 10 Auto-Fill Application Forms Sequence Diagram.....	44
Figure 11 Create Funding Programme Sequence Diagram.....	45
Figure 12 Import Applications Sequence Diagram.....	46
Figure 13 Review Imported Applications Sequence Diagram.....	47
Figure 14 Review Applicant Sequence Diagram.....	48
Figure 15 Shortlist Applicant Sequence Diagram.....	49
Figure 16 Landing page.....	51
Figure 17 Log in page.....	52
Figure 18 Registration, choice of account type.....	53
Figure 19 Student registration.....	54
Figure 20 Organisation registration.....	55
Figure 21 Student dashboard.....	56
Figure 22 Student profile.....	57
Figure 23 Opportunities, ranked by match.....	58
Figure 24 Opportunity detail, with the match explained.....	59
Figure 25 Application form.....	60
Figure 26 Student application tracker.....	61
Figure 27 Application detail.....	62
Figure 28 Student document library.....	63
Figure 29 Student notifications.....	64
Figure 30 Student account settings.....	65
Figure 31 Corporate dashboard.....	66
Figure 32 Funding programmes held by the Organisation.....	67
Figure 33 Funding programme editor.....	68
Figure 34 Applications received.....	69
Figure 35 Applicant review, with the eligibility verdict.....	70
Figure 36 Shortlists.....	71
Figure 37 Beneficiaries.....	72
Figure 38 Corporate reports.....	73
Figure 39 Organisation profile.....	74
Figure 40 Landing page at a tablet width of 834 pixels.....	75
Figure 41 Opportunities at a tablet width of 834 pixels.....	76
Figure 42 Applications received at a tablet width of 834 pixels.....	77
Figure 43 Landing page at a mobile width of 390 pixels.....	78
Figure 44 Student dashboard at a mobile width of 390 pixels.....	79
Figure 45 Opportunities at a mobile width of 390 pixels.....	80
Figure 46 Corporate dashboard at a mobile width of 390 pixels.....	81

1.0. Introduction

1.1. Purpose

This Software Design Description presents the design of Bursary-Bridge, a web application that connects South African students with bursaries, scholarships and other education funding, and gives the organisations that fund education a place to publish those opportunities and administer the applications they attract. It describes the design entities that make up the system, the responsibilities of each, the interfaces between them, the structure of the stored data and the way the use cases stated in the Software Requirements Specification are realised by those entities.

The expected audience is the Department of Computer Science at the University of Zululand, including the project supervisor and the studio committee, together with any developer who is asked to extend or maintain the system. The reader is assumed to be familiar with the Software Requirements Specification for Bursary-Bridge [2], to which this document is a companion; requirements are not restated here except where a design decision cannot be understood without them.

1.2. Scope

This document contains a complete description of the design of Bursary-Bridge as implemented in the prototype.

The architecture is a single web application served from one process. It is built on Next.js 15 using the App Router [3], with React 19 components rendered on the server by default, TypeScript throughout and Tailwind CSS for presentation. Persistent state is held in PostgreSQL and reached through Prisma [4]. The application exposes three portals over one codebase and one database: a Student portal, a Corporate portal and an Administration portal.

The system recognises three roles, STUDENT, CORPORATE and ADMIN. A Student builds a profile, receives ranked funding opportunities with the reasons behind each score, and applies using the profile already captured. A Corporate User states an organisation's funding criteria as structured data, publishes funding programmes, and moves applicants through review, shortlisting and selection. An Administrator maintains the catalogue of institutions, qualifications and subjects that the other two roles select from, and suspends or

restores accounts; the Administration Portal in Section 3 is the design entity that carries that role. No university or government role exists in the system.

The prototype does not disburse funds, does not verify submitted documents against any external authority, and does not integrate with the administration systems of universities or of government. Those lie outside the scope of this release and are not designed for in this document.

1.3. Glossary

Table 1 defines the terms used throughout this document. Each term carries the same meaning here as in the Software Requirements Specification [2].

Table 1 Glossary of Terms

Term	Definition
Applicant	A Student who has submitted an Application to a Funding Programme.
Application	A Student's formal request for funding under a specific Funding Programme, carrying a status, the match score recorded at submission, answers to programme questions and any attached Documents.
App Router	The Next.js routing model used by the system, in which the directory structure under app/ defines both the pages and the HTTP route handlers.
Corporate User	A representative of an Organisation who administers that Organisation's Funding Programmes. Also referred to as the Funder.
Design Entity	A named component of the design with a distinct responsibility, described in Section 3 by its type, attributes, resources and operations.
Eligibility Outcome	The verdict returned when an Applicant is evaluated against a programme's Eligibility Rule: ELIGIBLE, NOT_ELIGIBLE or PENDING_VERIFICATION.
Eligibility Rule	The machine-readable conditions attached to a Funding Programme, held as typed columns rather than free text.
Funding Programme	A bursary, scholarship or grant published by an Organisation, to which Students may apply.
Guard	A server-side function that establishes the caller's identity and role and returns the identifiers used to scope every subsequent database query.
Match Score	A percentage from 0 to 100 expressing how closely a Student's profile fits a Funding Programme, produced by the Matching Engine.

Matching Engine	The pure component that scores a Student profile against a Funding Programme across six weighted criteria and returns both the score and the reasons behind it.
Organisation	A corporation, foundation, non-governmental organisation or other body that funds education through the system.
Prisma	The type-safe object relational mapper through which all database access passes.
Programme	A standardised course of study held in the catalogue, distinct from a Funding Programme.
Route Handler	A server-side function under app/api that answers an HTTP request, validates its body and calls the service layer.
Server Component	A React component rendered on the server, able to query the database directly and never shipped to the browser.
Service Layer	The modules under services/ that hold the read and write logic shared by pages and route handlers.
Session	A server-side record identifying a signed-in user, addressed by an opaque token held in an httpOnly cookie.
Study Preference	A course paired with the institution at which the Student wishes to study it, held as one record.
Zod	The schema library used to validate every request body on the server.

1.4. References

The sources below are cited by bracketed number throughout this document.

Each is cited at least once in the text, and every bracketed number in the text appears here.

[1] IEEE, IEEE Std 1016-2009, IEEE Standard for Information Technology - Systems Design - Software Design Descriptions. New York: Institute of Electrical and Electronics Engineers, 2009. The structure of this document, and the description of each design entity by its name, type, attributes, resources and operations, follow this standard.

[2] Department of Computer Science, University of Zululand, “Software Requirements Specification: Bursary-Bridge, A Bursary and Scholarship Matching Platform,” 2026. Every Section 3.2 cross reference in Section 5 of this document refers to that specification.

[3] Vercel, Next.js Documentation, Version 15: App Router. [Online]. Available: <https://nextjs.org/docs>.

[4] Prisma Data, Inc., Prisma ORM Documentation, Version 6. [Online]. Available: <https://www.prisma.io/docs>.

[5] Republic of South Africa, Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer, 2013. [Online]. Available: <https://www.justice.gov.za/legislation/acts/2013-004.pdf>.

1.5. Overview of document

The remaining chapters and their contents are listed below.

Section 2 is a Deployment Diagram showing the physical nodes on which the system runs and the artefacts placed on each, so that the location of every part of the system is unambiguous.

Section 3 is the Architecture Design. It identifies the design entities that collaborate to perform the functions of the system and describes each by its name, type, description, attributes, resources and operations, in the manner recommended by IEEE 1016 [1].

Section 4 concerns the Data Structure Design, giving the logical structure of the stored data and the type and size of each field.

Section 5 contains the Use Case Realizations. Each use case is traced from the Software Requirements Specification to the design objects that carry it out, and is illustrated by a sequence diagram.

Section 6 discusses the User Interface Design and shows the implemented screens.

Section 7 covers the help system.

Section 8 is the index.

2.0. Deployment diagram

Figure 1 shows the physical nodes on which the system runs and the artefacts placed on each, so that the location of every part of the system is unambiguous. The paragraphs that follow describe each node and the protocol by which it is reached.

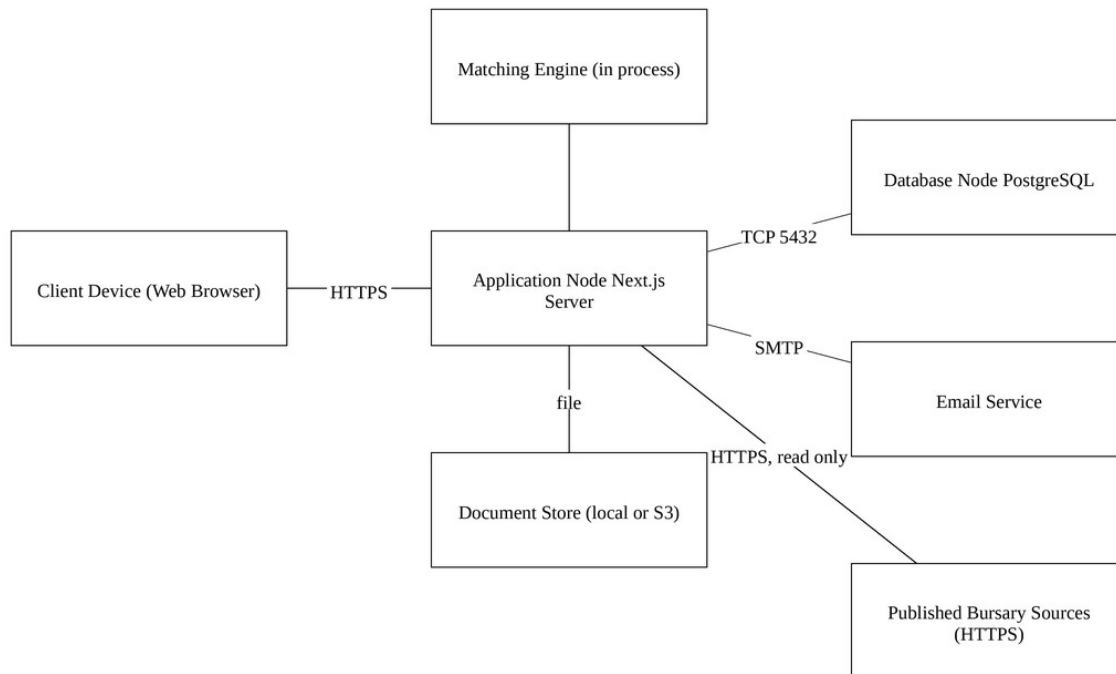


Figure 1 Deployment Diagram

Bursary-Bridge is deployed as a single Node.js process serving both portals, with PostgreSQL as the only persistent store. The browser is the sole client; no native application is distributed.

The application node runs the Next.js server, which renders the React server components, executes the route handlers under `app/api` and hosts the Matching Engine in process. The Matching Engine is not a separate service: it is a pure TypeScript module with no input or output of its own, which is what allows the same scoring code to run when a Student views their matches and when a Corporate User reviews an applicant.

The database node runs PostgreSQL and is reached only by the application node, over a connection string held in the environment rather than in source control. No client ever addresses the database directly.

Uploaded documents are held outside the public web root. In the prototype they are written to a directory on the application node; when `STORAGE_DRIVER` is set to `s3` the same interface writes to S3-compatible object storage instead, and no calling code changes. Documents are never served as static files: every retrieval passes through an authorised route handler.

The one cooperating system is an electronic mail service, used for address verification and for notifications raised when an application changes status. The prototype's default driver writes these messages to the server console; the system takes no decision that depends on their delivery.

3.0. Architecture design

3.1. Web System Architecture

Figure 2 presents the design entities that make up the system and the collaborations between them. Each entity is then described in turn, under the headings that follow.



Figure 2 Architecture Design

The system is layered. The browser holds presentation only. The application node holds the pages, the route handlers, the service layer and the pure domain modules. The database holds all persistent state. Requests move downward through the layers and data moves upward; no layer reaches past the one beneath it.

Two properties of this arrangement carry most of the design. The first is that authorisation is established at the top of every server-side entry point by a guard, which returns the identifiers that then scope every query, so an authorisation failure is a missing identifier rather than a forgotten check. The second is that the Matching Engine is pure: it imports neither Prisma nor React, receives plain data structures and returns a score with its reasons, which makes it independently testable and prevents the score shown to a Student from diverging from the verdict shown to a Funder.

The design entities that follow are the components of the system. Each is described by its name, type, description, attributes, resources and operations.

Student Portal

Name: Student Portal

Type: Set of web pages

Description: The Student Portal is the set of screens presented to a user holding the STUDENT role. It comprises the dashboard, the opportunities list, the opportunity detail and application screens, the applications tracker, the document library, the notification list, the account settings and the seven-step onboarding sequence that builds the profile.

The pages are React server components. They query the service layer directly during rendering rather than fetching from the application's own HTTP interface, so the first paint carries real data and no loading state is shown for content the server already holds. Only the interactive parts, principally the forms, are client components.

Every page under /student and /onboarding calls a guard before it reads anything. A visitor without a session never reaches these pages: the edge middleware redirects them to the login screen and preserves the path they were heading for.

Attributes:

Route prefix: /student, /onboarding/student

Rendering: server components, with client components for forms

Guard: requireStudent(), requireOnboardedStudent()

Resources:

Service layer: matching, student-profile, student-summary, profile-strength, notifications

Authentication and Session Module

Operations:

Name: viewDashboard()

Arguments: None

Returns: Rendered page

Pre-condition: The Student holds a valid session

Post-condition: The dashboard is displayed with the profile strength, the strongest matches and the state of any open applications

Exceptions: No session: the user is redirected to the login page

Flow of Events:

The Student requests the dashboard.

The guard establishes the identity and returns the student profile identifier.

The page asks the service layer for the profile summary and the ranked opportunities.

The Matching Engine scores each published programme against the profile.

The page is rendered with the results and returned.

Name: completeOnboardingStep()

Arguments: Step name, submitted form values

Returns: Redirect to the next step

Pre-condition: The Student holds a valid session

Post-condition: The step is saved, the profile strength is recalculated and the Student is advanced to the next step

Exceptions: Validation failure: the step is redisplayed with messages against the offending fields and nothing is written

Flow of Events:

The Student completes the fields of the current step.

The values are submitted to the onboarding route handler.

The handler validates them against the step's schema.

The service layer writes the values and records the step reached, so an interrupted profile can be resumed.

The Student is advanced to the next step.

Corporate Portal

Name: Corporate Portal

Type: Set of web pages

Description: The Corporate Portal is the set of screens presented to a user holding the CORPORATE role. It comprises the dashboard, the funding programme list and editor, the applications table, the applicant review screen, the shortlists, the beneficiaries, the reports, the organisation profile and the four-step organisation onboarding sequence.

Every read performed by these pages is scoped to the organisation identifier returned by the guard. A Corporate User cannot address another organisation's programmes or applications, because no query in the portal is written without that identifier in its filter.

The applicant review screen presents the eligibility verdict and the reasons behind it alongside the applicant's profile. The verdict is produced by the same criteria that generated the Student's match score, so the two views of a single pairing cannot disagree.

Attributes:

Route prefix: /corporate, /onboarding/organisation

Rendering: server components, with client components for forms and filters

Guard: requireCorporate()

Resources:

Service layer: applicants, shortlists, corporate-stats, organisation-summary, notifications

Eligibility Service

Authentication and Session Module

Operations:

Name: reviewApplicant()

Arguments: Application identifier

Returns: Rendered page

Pre-condition: The Corporate User holds a valid session and the application belongs to their Organisation

Post-condition: The applicant's profile, eligibility verdict and reasons are displayed

Exceptions: The application belongs to another Organisation: the record is not found and the request is refused

Flow of Events:

The Corporate User selects an application.

The guard returns the organisation identifier.

The service layer loads the application, filtered by that identifier as well as by the application identifier.

The Eligibility Service evaluates the applicant against the programme's Eligibility Rule.

The page is rendered with the verdict and the reason for each criterion.

Name: publishProgramme()

Arguments: Funding programme identifier

Returns: Updated programme status

Pre-condition: The programme is in DRAFT and its eligibility criteria are complete

Post-condition: The programme status becomes PUBLISHED and it becomes visible to matching Students

Exceptions: Incomplete criteria: publication is refused and the missing items are named

Flow of Events:

The Corporate User opens a draft programme.

The system checks that the programme carries an Eligibility Rule and the fields a match requires.

The Corporate User confirms publication.

The status is set to PUBLISHED and the change is written to the audit log.

The programme enters the pool the Matching Engine scores.

Administration Portal

Name: Administration Portal

Type: Set of web pages

Description: The Administration Portal is the set of screens presented to a user holding the ADMIN role. It comprises the catalogue of institutions and courses, the user register, the organisation and funding programme registers, the bursary data sources and their run history, and the audit trail.

The portal maintains the platform rather than any one Organisation's funding. It holds no application data of its own: every screen acts on records owned elsewhere in the system, and every action it takes is written to the audit log with the acting administrator and a stated reason.

Every page under /admin calls requireAdmin() before it reads anything. The guard is applied in the layout, so a page added later inherits it rather than having to remember it.

Attributes:

Route prefix: /admin

Rendering: server components, with client components for forms

Guard: requireAdmin(), apiAdmin()

Resources:

Service layer: admin-catalogue, admin-users, admin-organisations, admin-programmes, admin-ingestion, admin-audit

Authentication and Session Module

Operations:

Name: setCatalogueEntryStatus()

Arguments: Entry identifier, target status, entry kind

Returns: The updated entry

Pre-condition: The administrator holds a valid session

Post-condition: The entry is active or retired; records already pointing at it are unaffected

Exceptions: Unknown entry: the request is refused and nothing is written

Flow of Events:

The administrator retires or restores an institution or course.

The service counts the student profiles and funding programmes that point at the entry.

The status is changed rather than the row deleted, so those records keep the institution or course they name.

The count is returned to the administrator with the result.

Name: runIngestion()

Arguments: None

Returns: The identifier and outcome of the run

Pre-condition: The administrator holds a valid session and at least one source is authorised

Post-condition: A run is recorded, with the opportunities it created, updated, merged and rejected

Exceptions: A source whose robots file disallows collection is recorded as not read; a source that cannot be reached makes the run BLOCKED rather than SUCCEEDED

Flow of Events:

The administrator starts a run.

The Bursary Ingestion Pipeline reads each authorised source's robots file and honours it.

Each listing and detail page is fetched at the rate the source asks for.

Each candidate is validated, deduplicated and stored with the address it was read from.

The run is closed with its counts and any note.

Bursary Ingestion Pipeline

Name: Bursary Ingestion Pipeline

Type: Server-side module

Description: The Bursary Ingestion Pipeline collects funding opportunities from published external sources. It is the only part of the system that creates an opportunity nobody has entered by hand, and it is constrained accordingly: a source is read only after an administrator has authorised it by name, its robots file is read first and honoured, and the rate it asks for is observed.

Extraction is deliberately literal. Each field is either found on the page or left unset; nothing is inferred from a neighbouring value. A candidate that cannot be shown to describe a real opportunity is rejected rather than stored with gaps, and every stored opportunity carries the address it was read from and the date it was last confirmed.

The module is written against a fetcher interface rather than the network directly, so the same pipeline runs over pages supplied offline. That is what allows it to be tested without reading a live site.

Attributes:

Location: lib/ingest, services/opportunity-ingest

Authorised sources: declared in the source registry, disabled by default

Provenance: source address, source name, source type and verification dates on every opportunity

Resources:

Bursary-Bridge Database

The published pages of the authorised sources, over HTTP

Operations:

Name: runSource()

Arguments: A registered source, and optionally a page limit

Returns: The candidates found and the rejections, with a status

Pre-condition: The source is authorised and has listing pages configured

Post-condition: Nothing is written; the outcome is returned for the caller to persist

Exceptions: Disallowed by robots: the source is reported as not read. Unreachable: the source is reported as blocked, which is distinct from having found nothing

Flow of Events:

The robots file for the source's host is read and parsed.

Each listing page permitted by that file is fetched.

The listing is parsed into candidates and links to detail pages.

Each detail page is fetched and parsed, waiting between requests for as long as the source asks.

Each candidate is validated, and those describing the same opportunity are merged.

Document Extraction Engine

Name: Document Extraction Engine

Type: Server-side module

Description: The Document Extraction Engine reads a completed application form and returns the values on it as canonical fields, each with the confidence of the reading and the label it was read from. It serves two features: the Student's auto-fill of blank forms, and the Organisation's import of applications received elsewhere.

Values are taken from a form's own field definitions where it has them and from the page text where it does not, and the two are not treated as equally reliable: a value read from a real form field is exact, while one recovered from the text layer was inferred from position and is trusted less.

A form that is encrypted, or that carries no readable text because it is a scan, is refused with a reason rather than guessed at. The engine never invents a value; a field it cannot establish is left unset and reported as needing attention.

Attributes:

Location: lib/pdf

Canonical fields: thirty-three, covering personal, contact, education and funding information

Confidence: HIGH, MEDIUM or LOW, recorded per field

Resources:

Document Storage Service

pdf-lib for writing forms, pdfjs-dist for reading them

Operations:

Name: readSourceForm()

Arguments: The bytes of a completed form, and its name

Returns: The canonical values, or a refusal with a reason

Pre-condition: The document is a PDF

Post-condition: Nothing is written; the values are returned to the caller

Exceptions: Encrypted, damaged, or a scan with no text layer: the form is refused and the reason is returned in words the user can act on

Flow of Events:

The document is opened and its fields and text are analysed.

Each field label is matched against the canonical field names and their recognised aliases.

Each matched value is normalised and recorded with its confidence and the label it came from.

Labels that matched nothing are returned separately rather than discarded.

Application Import Pipeline

Name: Application Import Pipeline

Type: Server-side module

Description: The Application Import Pipeline turns application forms an Organisation received outside the platform into scored, reviewable applicant records. It expands an upload, reads each form through the Document Extraction Engine, looks for an existing Student account, scores the applicant against the programme's own criteria, counts the supporting documents supplied and flags duplicates.

The pipeline creates nothing on its own. Extraction produces a batch for a person to review; the applications are created only when a Corporate User confirms it. Duplicates are flagged and never merged automatically, and a file that cannot be read is kept with the reason rather than discarded.

Duplicate detection runs as a single ordered pass after every file in the batch has been read. Deciding during extraction would be a race: two copies processed in the same window would each examine the other before either had been recorded, and both would pass.

Attributes:

Location: lib/import, services/application-import

Concurrency: four files at a time

Identity: matched on verified identifiers only, never on a name

Resources:

Document Extraction Engine

Matching Engine and Eligibility Service

Document Storage Service

Bursary-Bridge Database

Operations:

Name: createBatch()

Arguments: Organisation, funding programme, acting user, the uploaded files

Returns: The batch identifier and its reference

Pre-condition: The funding programme belongs to the acting Organisation

Post-condition: Every uploaded file is stored and recorded against the batch before any of it is read

Exceptions: A file that is not a PDF, or an archive that cannot be opened, is recorded as failed with the reason

Flow of Events:

The upload is expanded, each folder in an archive treated as one applicant.

Every file is written to Document Storage.

A row is created for each, so the batch survives an interruption.

Files that could not be used are recorded as failures rather than dropped, so the totals account for everything sent.

Name: processBatch()

Arguments: Batch identifier

Returns: Completion, with the counts held on the batch

Pre-condition: The batch exists and holds files awaiting processing

Post-condition: Every file has reached a conclusion and the batch is ready for review

Exceptions: A file that cannot be processed is marked failed with its reason; the remainder of the batch continues

Flow of Events:

Each pending file is read through the Document Extraction Engine.

The applicant is projected from the extracted values and matched against the catalogue.

An existing Student is looked for on verified identifiers.

The Matching Engine and the Eligibility Service score the applicant against this programme.

The supporting documents supplied are counted against those the programme requires.

Once every file has been read, one ordered pass flags duplicates within the batch and against the applications the programme already holds.

Name: confirmImport()

Arguments: Batch identifier, Organisation, acting user, the reviewer's decisions

Returns: The number imported and the number left out

Pre-condition: The batch belongs to the acting Organisation and has been processed

Post-condition: An application exists for each file marked for import, carrying its score, its eligibility verdict and a link to the document it came from

Exceptions: A file whose application cannot be created is reported and left for review; the rest are still created

Flow of Events:

Each file marked for import is taken in turn.

Where a Student was identified, the application is attached to that profile; where none was, an external applicant record is created.

The application is created with the source recorded as EXTERNAL, so it is never mistaken for one submitted here.

The file is marked imported and linked to the application, so confirming twice cannot create it again.

Motivational Letter Composer

Name: Motivational Letter Composer

Type: Pure domain module

Description: The Motivational Letter Composer assembles a draft letter from facts already held on a Student's profile and from sentences the Student wrote in answer to the questions put to them. There is no third source.

Where there is no material for a paragraph, the paragraph is omitted and the Student is told what to add, rather than a plausible substitute being written. The letter is sent under the Student's name to somebody deciding whether to fund them, and a sentence about an achievement they never mentioned would be a claim they never made.

The composer is deterministic and is reached through a provider interface, so a different writer can be substituted without the service, the interface or the stored letters changing.

Attributes:

Location: lib/letters

Questions: seven, all optional

Interface: LetterProvider

Resources:

The Student's profile, subject results and study preferences

The funding programme's stated requirements, where the letter names a programme the system holds

Operations:

Name: compose()

Arguments: The profile facts, the opportunity and the Student's answers

Returns: The letter, and the answers it used

Pre-condition: The Student holds a profile

Post-condition: Nothing is written; the text is returned to the service to store

Exceptions: None: a letter composed from nothing is short rather than an error

Flow of Events:

The salutation is addressed to the funder where one is named.

Each paragraph is assembled only where there is material for it.

A subject requirement the Student demonstrably meets is cited against the minimum the funder states.

The Student's own sentences are carried through unaltered rather than paraphrased.

Authentication and Session Module

Name: Authentication and Session Module

Type: Server-side module

Description: This module establishes and carries identity. It hashes and verifies passwords, issues and destroys sessions, resolves the current user from the request, and provides the guards that every protected entry point calls before it reads or writes anything.

A session token is a random 32-byte value encoded for transport and handed to the browser in a cookie marked httpOnly, SameSite=Lax and, outside development, Secure. Only the SHA-256 hash of the token, salted with the application secret, is stored, so a database dump cannot be replayed as a login. Tokens expire fourteen days after issue.

Passwords are hashed with bcrypt at a cost factor of 12. The password policy is declared once and used both by the client-side strength hint and by the server-side schema, so the two cannot drift apart.

The guards are the authorisation mechanism of the whole system. `requireStudent()` and `requireCorporate()` return the profile or organisation identifier of the caller, and that identifier is placed in the filter of every query the caller's request goes on to make. Authorisation is therefore not a check that can be omitted but a value without which no query can be written.

Attributes:

Session cookie name: `bb_session`

Session lifetime: 14 days

Token: 32 random bytes, stored as a salted SHA-256 hash

Password hash: bcrypt, cost factor 12

Roles: STUDENT, CORPORATE

Resources:

Bursary-Bridge Database (User, Session and VerificationToken tables)

`AUTH_SECRET` from the environment

Operations:

Name: `createSession()`

Arguments: User identifier, user agent, network address

Returns: No return value

Pre-condition: The credentials presented have been verified

Post-condition: A session record exists and the browser holds the cookie

Exceptions: None

Flow of Events:

A token of 32 random bytes is generated.

Its salted SHA-256 hash is written to the Session table with an expiry fourteen days ahead.

The token itself is set as an `httpOnly` cookie on the response.

Name: `requireStudent()`

Arguments: None

Returns: The signed-in user and their student profile identifier

Pre-condition: None

Post-condition: The caller holds an identifier that scopes its queries

Exceptions: No session, an expired session, or a user whose role is not STUDENT: the request is redirected to the login page, or answered 401 when it arrived at the application programming interface

Flow of Events:

- The session cookie is read from the request.

- Its hash is looked up in the Session table.

- An expired or unknown session is rejected.

- The user's role is checked.

- The student profile identifier is returned to the caller.

Name: verifyPassword()

Arguments: Submitted password, stored hash

Returns: Boolean

Pre-condition: None

Post-condition: None

Exceptions: None

Flow of Events:

- The submitted password is compared against the stored bcrypt hash.

- The result is returned without revealing whether the address or the password was at fault.

Matching Engine

Name: Matching Engine

Type: Pure domain module

Description: The Matching Engine scores a Student profile against a Funding Programme and returns both a percentage and the reason for every criterion that contributed to it. It is the component that gives the system its purpose, and it is deliberately pure: it imports

neither Prisma nor React, takes plain data structures as arguments and performs no input or output. It can therefore be tested in isolation and executed in any context.

Six criteria are evaluated, each carrying a fixed weight: course 30, institution 25, academic 20, qualification level 10, location 10 and financial 5, totalling 100. A criterion that cannot be evaluated because the profile does not yet carry the information earns half credit rather than zero, so an incomplete profile is not treated as a failed one, while the gap still lowers the score and is reported as needing verification. When more than 25 points of weight are unevaluable the result is reported as needing more information whatever the remaining criteria scored.

The score is classified as a strong match at 85 and above, a potential match from 60 to 84, and more information needed below 60. The reasons are returned with the score and are always displayed with it; a percentage is never shown on its own.

All tuning is held in a single configuration module so the weights and thresholds can be changed, or in future loaded per funder, without altering evaluation logic or the user interface.

Attributes:

Criterion weights: course 30, institution 25, academic 20, qualification 10, location 10, financial 5

Unknown credit ratio: 0.5

Maximum unknown weight: 25

Strong match threshold: 85

Potential match threshold: 60

Resources:

None. The engine performs no input or output and holds no state.

Operations:

Name: score()

Arguments: Student profile, funding programme with its eligibility rule

Returns: Score, classification and the reason for each criterion

Pre-condition: Both arguments have been adapted from database records into the engine's own data structures

Post-condition: None; the engine holds no state

Exceptions: None

Flow of Events:

Each of the six criteria is evaluated in turn.

A criterion that is met contributes its full weight; one that cannot be evaluated contributes half.

The contributions are summed and expressed as a percentage.

The unevaluable weight is compared against the maximum.

The score is classified and returned with its reasons.

Name: rank()

Arguments: Student profile, list of published funding programmes

Returns: The programmes ordered by descending score

Pre-condition: As for score()

Post-condition: None

Exceptions: None

Flow of Events:

Each programme is scored against the profile.

The results are ordered by score, highest first.

The ordered list is returned with each score's reasons intact.

Eligibility Service

Name: Eligibility Service

Type: Pure domain module

Description: The Eligibility Service answers the Funder's question, which is different from the Student's. Where the Matching Engine asks how well a profile fits and returns a percentage, the Eligibility Service asks whether the criteria are met and returns a verdict: ELIGIBLE, NOT_ELIGIBLE or PENDING_VERIFICATION.

The two share their criteria definitions, which is what guarantees that the score shown to a Student and the verdict shown to a Funder are two readings of the same evaluation rather than two independent calculations that may disagree.

The verdict is evaluated when an application is submitted and stored on the Application record. Recording it at submission rather than recomputing it on demand means a Funder reviewing an application sees the assessment as it stood when the Student applied, and that counts reported on the dashboard remain stable as profiles and programmes change afterwards.

Attributes:

Outcomes: ELIGIBLE, NOT_ELIGIBLE, PENDING_VERIFICATION

Criteria: shared with the Matching Engine

Resources:

None. The service performs no input or output.

Operations:

Name: evaluate()

Arguments: Student profile, eligibility rule

Returns: Outcome and the reason for each criterion

Pre-condition: The programme carries an eligibility rule

Post-condition: None

Exceptions: None

Flow of Events:

Each condition of the rule is tested against the profile.

A condition the profile contradicts yields NOT_ELIGIBLE.

A condition the profile cannot yet answer yields PENDING_VERIFICATION.

A profile that satisfies every condition yields ELIGIBLE.

The outcome is returned with the reason for each condition.

Application Programming Interface

Name: Application Programming Interface

Type: Set of HTTP route handlers

Description: The application exposes twenty-one route handlers under app/api. They serve the browser's write operations; reads performed during page rendering go to the service layer directly and do not pass through this interface.

Each handler follows the same shape. It calls a guard to establish identity and role; it parses the request body against a Zod schema and answers 400 with field-level messages if parsing fails; it calls the service layer with the identifiers the guard returned; and it answers with JSON. Validation is performed on the server in every case. The same schemas are used in the browser to give immediate feedback, but the browser's result is never trusted.

The handlers answer 401 rather than redirecting, because they are called by scripts rather than navigated to; the edge middleware excludes /api from its redirect rules for this reason. The authentication handlers are rate limited by address and by account, as are registration, verification email requests, document uploads and application submissions.

Attributes:

Base path: /api

Handlers: 21

Groups: auth, student, corporate, documents, account, catalog

Validation: Zod schemas under lib/validation

Resources:

Authentication and Session Module

Service layer

Document Storage Service

Operations:

Name: POST /api/auth/login

Arguments: Email address, password

Returns: The signed-in user, or an error

Pre-condition: None

Post-condition: A session exists and the cookie is set

Exceptions: Unknown address or wrong password: 401 with a single message that does not reveal which was wrong. Too many attempts: 429 with the seconds to wait.

Flow of Events:

The body is validated against the login schema.

The rate limiter is consulted for the address and the account.

The user is looked up and the password compared against the stored hash.

A session is created and the cookie set.

The user's role determines the dashboard the browser is sent to.

Name: POST /api/student/applications

Arguments: Funding programme identifier, answers, attached document identifiers

Returns: The created application

Pre-condition: The Student holds a valid session and a completed profile

Post-condition: An application exists carrying the match score, classification, reasons and eligibility outcome recorded at submission

Exceptions: A second application to the same programme is refused. A closed or unpublished programme is refused. A missing required answer or document is refused with the field named.

Flow of Events:

The guard returns the student profile identifier.

The body is validated and the programme checked to be open.

The Matching Engine scores the profile against the programme.

The Eligibility Service evaluates the same pairing.

The application is written with the score, the reasons and the outcome, a notification is raised and the action is recorded in the audit log.

Document Storage Service

Name: Document Storage Service

Type: Server-side module

Description: This service holds the files Students upload in support of their applications: identity documents, academic records, proof of income and similar. It presents one interface with two implementations, selected by configuration: a local disk provider used by the prototype and an S3-compatible provider for deployment. No calling code touches the filesystem, so the change is to one environment variable.

Files are held outside the public web root and are never served as static assets. Every retrieval passes through a route handler that first establishes who is asking: the Student who owns the document, or an Organisation to which that document has been attached as part of an application. Any other request is refused.

Uploads are constrained on the server. The declared content type must be one of PDF, JPEG, PNG or WebP, and the file must not exceed five megabytes. Both checks are applied before anything is written.

Attributes:

Drivers: local, s3

Permitted types: application/pdf, image/jpeg, image/png, image/webp

Maximum size: 5 MB

Location: outside the public web root

Resources:

Local filesystem, or S3-compatible object storage

Bursary-Bridge Database (Document and ApplicationDocument tables)

Operations:

Name: put()

Arguments: File contents, declared content type, owning student profile

Returns: The stored object's key

Pre-condition: The Student holds a valid session

Post-condition: The file is stored and a Document record refers to it

Exceptions: A type that is not permitted, or a file above five megabytes, is rejected before any write occurs

Flow of Events:

The content type is checked against the permitted list.

The size is checked against the maximum.

The file is written through the configured provider.

A Document record is created against the owning profile.

Name: get()

Arguments: Object key, requesting user

Returns: The file contents

Pre-condition: The requester owns the document, or holds an application to which it is attached

Post-condition: None

Exceptions: A requester with neither relationship is refused and the document is reported as not found

Flow of Events:

The route handler establishes the requester's identity.

The document's ownership is checked, and the applications it is attached to are checked against the requester's organisation.

The file is streamed through the handler if either holds.

Notification and Email Service

Name: Notification and Email Service

Type: Server-side module

Description: This service raises the in-application notifications a user sees in their notification list and sends the corresponding electronic mail. Notifications are written to the database and are therefore durable; mail is a secondary channel and the system takes no decision that depends on its delivery.

The mail transport is a seam with two drivers. The prototype's driver writes each message to the server console, which makes address verification demonstrable without a mail server. Setting EMAIL_DRIVER to smtp directs the same messages to a development mail catcher or a transactional provider without altering any call site.

Notifications are raised when an address requires verification, when an application is submitted, when its status changes, when further information is requested and when an applicant is shortlisted or selected.

Attributes:

Drivers: console, smtp

Persistence: Notification table, read state per user

Resources:

Bursary-Bridge Database (Notification table)

Electronic mail service, when configured

Operations:

Name: notify()

Arguments: Recipient, subject, heading, body, optional action

Returns: No return value

Pre-condition: The recipient exists

Post-condition: A notification record exists and mail has been dispatched through the configured driver

Exceptions: A mail failure is logged and does not fail the operation that raised the notification

Flow of Events:

The notification is written to the database.

The message is passed to the configured mail driver.

The result of dispatch is logged and discarded.

Bursary-Bridge Database

Name: Bursary-Bridge Database

Type: Relational database

Description: The database is PostgreSQL and is reached only through Prisma. It holds twenty tables and thirty-five enumerated types. No component outside the application node addresses it, and no raw connection string appears in source control.

Two aspects of the structure carry most of the design. The first is the Study Preference, which pairs a course with the institution at which the Student wishes to study it and holds the two together as one record. Courses and institutions are not stored as unrelated lists, because a Student who wishes to read law at one university and engineering at another is stating two different things, and a matching engine given two flat lists cannot tell the difference.

The second is the Eligibility Rule, which holds a funder's conditions as typed columns rather than as prose. This is what allows the same conditions to drive both the Student's match score and the Funder's eligibility verdict.

Referential actions are stated explicitly. A profile's dependants cascade when it is deleted; catalogue references are restricted so that an institution in use cannot be removed; optional references are set to null.

Attributes:

Engine: PostgreSQL

Access: Prisma, version 6

Tables: 20

Enumerated types: 35

Migrations: applied from version-controlled files

Resources:

DATABASE_URL from the environment

Operations:

Name: query()

Arguments: A Prisma query carrying the identifier returned by a guard

Returns: Typed records

Pre-condition: The caller has passed a guard

Post-condition: None

Exceptions: A record outside the caller's scope is simply not returned, because the scope is part of the filter rather than a check applied afterwards

Flow of Events:

The service layer composes the query with the caller's identifier in its filter.

Prisma issues the statement.

Typed records are returned to the service layer.

Name: audit()

Arguments: Actor, action, subject

Returns: No return value

Pre-condition: A change has been made

Post-condition: An append-only audit record exists

Exceptions: None

Flow of Events:

The change to an application, programme, shortlist or account is written.

An audit record naming the actor, the action and the subject is appended.

4.0. Data structure design

The data is stored in a relational database managed by PostgreSQL and reached through Prisma. The schema is declared in one file and applied by version-controlled migrations, so the structure of a deployed database is always reproducible from source.

Identifiers are collision-resistant strings generated by the application rather than sequential integers, so that a record's identifier discloses neither the number of records held nor the order in which they were created. Textual fields are stored in PostgreSQL's text type, which is not fixed width; the sizes given below are therefore the limits imposed by the validation schemas on the server, which are the limits a user actually encounters. Fields whose type is an enumerated type are constrained by the database itself to the values listed in the schema.

Table 2 gives the fields of the principal entities. Relationship fields are shown by the entity they refer to rather than by their identifier column.

4.1. Data field types and sizes.

Table 2 gives every stored field, grouped by the entity that holds it, with its type and the size or constraint that applies.

Table 2 Data Field Types and Sizes

Attribute Name	Attribute Type	Attribute Size
User.email*	Text	254
User.passwordHash*	Text	60 (bcrypt)
User.role*	Enumerated	STUDENT, CORPORATE, ADMIN
User.firstName*	Text	60
User.lastName*	Text	60
User.mobile	Text	20
User.emailVerifiedAt	Timestamp	8 bytes
Session.tokenHash*	Text	64 (SHA-256)
Session.expiresAt*	Timestamp	8 bytes
StudentProfile.qualificationLevel	Enumerated	See schema
StudentProfile.academicAverage	Integer	0 to 100
StudentProfile.yearOfStudy	Integer	1 to 8
StudentProfile.householdIncome	Enumerated	Income bands
StudentProfile.citizenship	Enumerated	See schema
StudentProfile.province	Enumerated	9 provinces

StudentProfile.city	Text	80
StudentProfile.profileStrength	Integer	0 to 100
StudyPreference.preferenceNumber*	Integer	1 to 6
StudyPreference.programme*	Reference	Programme
StudyPreference.institution*	Reference	Institution
Organisation.name*	Text	120
Organisation.type*	Enumerated	See schema
FundingProgramme.name*	Text	120
FundingProgramme.fundingType*	Enumerated	Bursary, scholarship, grant
FundingProgramme.openDate*	Timestamp	8 bytes
FundingProgramme.closingDate*	Timestamp	8 bytes
FundingProgramme.status*	Enumerated	DRAFT, PUBLISHED, CLOSED
EligibilityRule.minAcademicAverage	Integer	0 to 100
EligibilityRule.qualificationLevels	Enumerated list	See schema
EligibilityRule.yearsOfStudy	Integer list	1 to 8
EligibilityRule.maxHouseholdIncome	Enumerated	Income bands
EligibilityRule.provinces	Enumerated list	9 provinces
Application.status*	Enumerated	See Section 3
Application.matchScore	Integer	0 to 100
Application.matchClassification	Enumerated	Strong, potential, more info
Application.eligibilityOutcome	Enumerated	ELIGIBLE, NOT_ELIGIBLE, PENDING_VERIFICATION
Document.mimeType*	Text	PDF, JPEG, PNG, WebP
Document.sizeBytes*	Integer	Maximum 5 242 880
StudentSubjectResult.subjectId*	Reference	SubjectCatalogue
StudentSubjectResult.percentage	Integer	0 to 100; null when not yet known
StudentSubjectResult.year*	Integer	4
StudentSubjectResult.level*	Enumerated	SCHOOL, TERTIARY
MotivationalLetter.opportunityName*	Text	200
MotivationalLetter.content*	Text	Unbounded
MotivationalLetter.answers	Structured	The Student's own answers, as supplied
MotivationalLetter.status*	Enumerated	DRAFT, READY
MotivationalLetter.editedByStudent*	Boolean	1
InformationRequest.message	Text	Unbounded; the Funder's own wording
InformationRequest.deadline	Date	Null when the Funder set none
InformationRequest.status*	Enumerated	OPEN, RESPONDED,

		CANCELLED
ApplicationImportBatch.reference*	Text	40
ApplicationImportBatch.status*	Enumerated	See schema; UPLOADING to COMPLETED
ApplicationImportBatch.totalFiles*	Integer	Recomputed from the files
ApplicationImportFile.fileName*	Text	255
ApplicationImportFile.storageKey*	Text	255, unique
ApplicationImportFile.status*	Enumerated	See schema; PENDING to IMPORTED
ApplicationImportFile.failureReason	Text	Unbounded; null when the file was read
ApplicationImportFile.matchScore	Integer	0 to 100; null when not eligible
ApplicationImportFile.duplicateReason	Text	Unbounded; why it was flagged
ImportExtractedField.canonicalKey*	Text	60
ImportExtractedField.value*	Text	Unbounded
ImportExtractedField.confidence*	Enumerated	HIGH, MEDIUM, LOW
ImportExtractedField.correctedValue	Text	Unbounded; the extracted value is kept
ExternalApplicant.fullName*	Text	200
ExternalApplicant.idNumber	Text	20
ExternalApplicant.academicAverage	Integer	0 to 100
Institution.canonicalName*	Text	200, unique
Programme.canonicalName*	Text	200, unique
OpportunitySource.url*	Text	2048
OpportunitySource.lastVerifiedAt	Date	When the opportunity was last confirmed
ProgrammeCriterionWeight.criterion*	Text	40
ProgrammeCriterionWeight.weight*	Integer	Relative, not a percentage

The structure records personal information of the kinds the Protection of Personal Information Act [5] treats as requiring protection: identity numbers, dates of birth, contact details, academic results and an indication of household income. Two design decisions follow from that. Each of these fields is held once, on the profile that owns it, rather than being copied onto every application, so there is a single place to correct or remove it. And every query that reads them is scoped by the identifiers a guard returns, as Section 3 describes, so the structure itself cannot be read across the boundary between one user and another.

Fields marked with an asterisk are required. Fields shown as Reference are foreign keys to the named entity. Fields shown as Enumerated are constrained by the database to the values declared in the schema; where the set is long the schema is cited rather than reproduced.

Two uniqueness constraints carry business meaning rather than mere housekeeping. A Study Preference is unique on the profile and the preference number, so the ordering a Student expresses cannot contain duplicates or gaps in position; and it is unique on the profile, the course and the institution, so the same pairing cannot be entered twice under different numbers. An Application is unique on the profile and the funding programme, so a Student cannot apply twice to the same programme.

Two columns are deliberately nullable where a stricter schema might not allow it. An Application holds a student profile only when one exists: an application an Organisation imported stands behind an external applicant record instead, and requiring a profile would have meant creating an empty account for somebody who never asked for one. A subject result holds no percentage until the Student has the mark, because knowing which subject is being taken without yet knowing the result is a real state, and recording it as a zero would tell a Funder something false.

5.0. Use case realizations

Figure 3 shows the interaction common to every request the system serves. A request from the browser reaches the application node, where the guard resolves the session before anything else occurs, the service layer composes a query already scoped to the caller, and the result is rendered and returned. The realizations that follow elaborate this pattern for individual use cases and omit the steps it has already established.

Each realization that follows is introduced by its cross reference to the Software Requirements Specification [2], described in prose, and illustrated by a sequence diagram of its own.

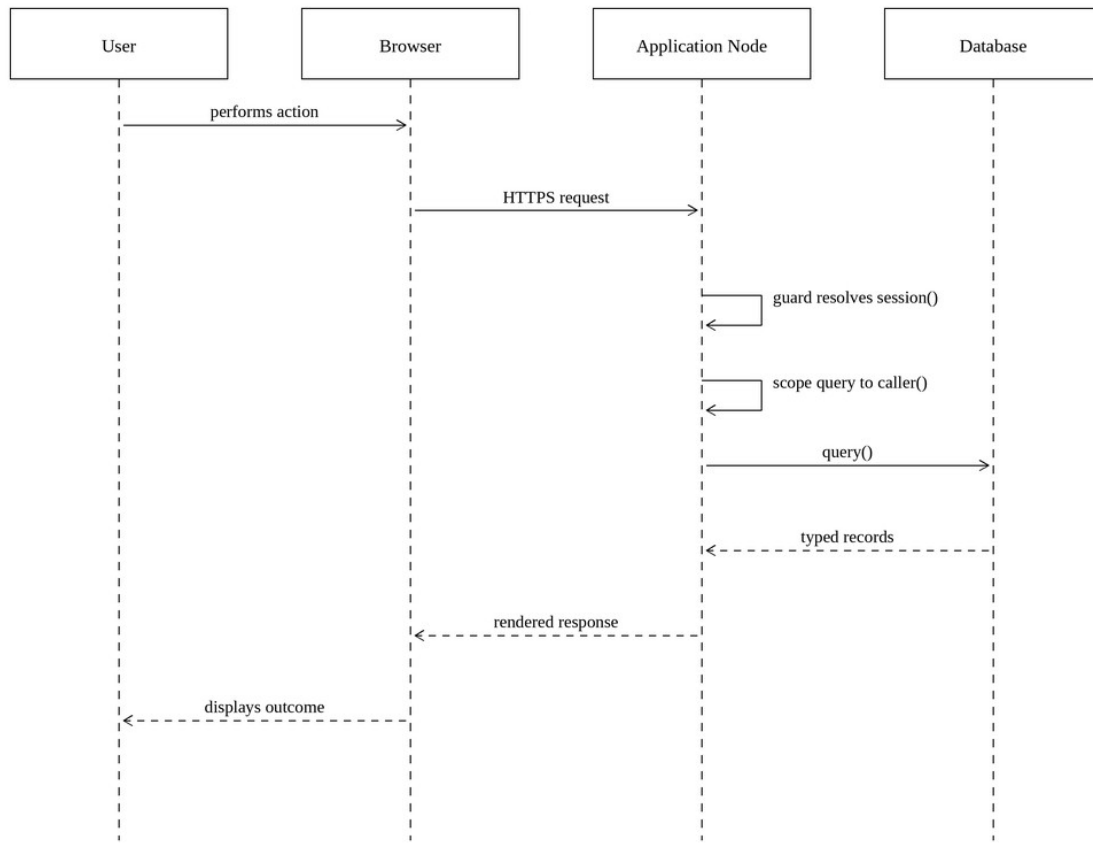


Figure 3 System Sequence Diagram

Use Case: Log In

See also Log In in Software Requirements Specification 3.2.1.

The Student or Corporate User presents their credentials. The password is compared against the stored bcrypt hash, a session is created and its token returned as an httpOnly cookie. The user's role determines the dashboard they are sent to. A failure names neither the address nor the password as the cause, and repeated failures are rate limited by address and by account.

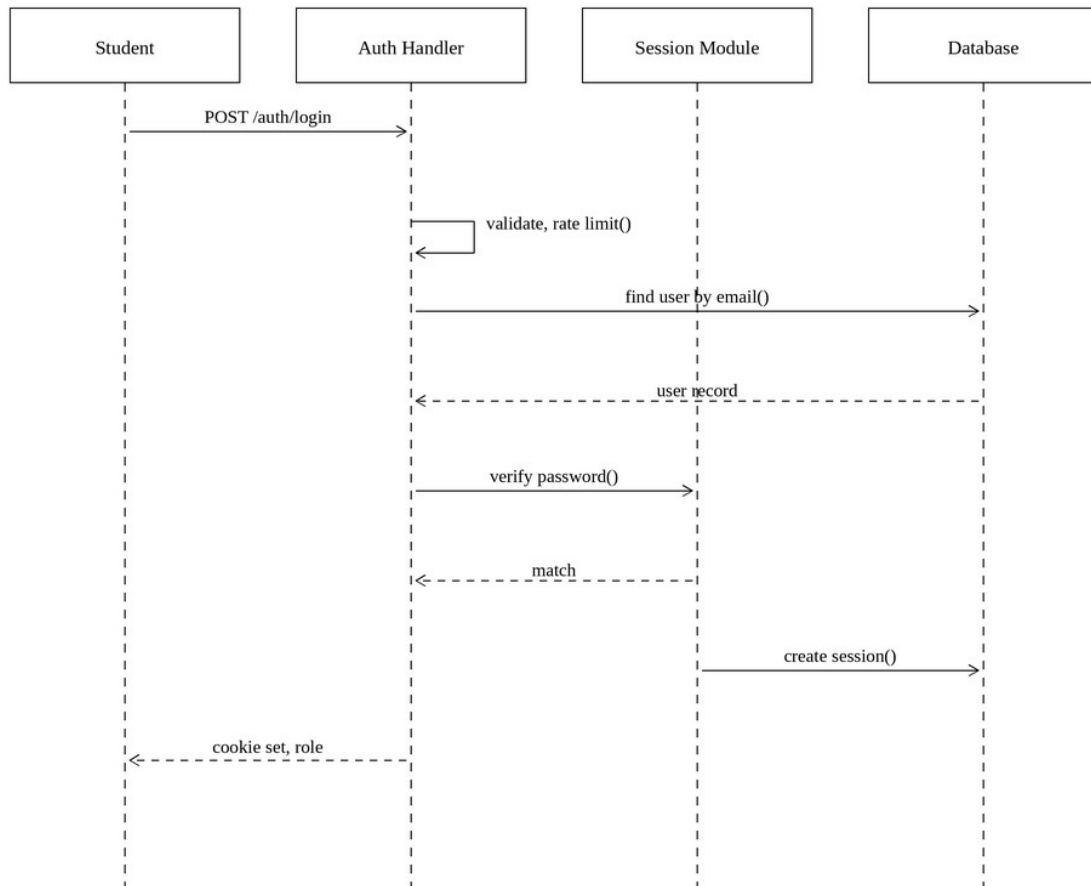


Figure 4 Log In Sequence Diagram

Use Case: Complete Student Profile

See also Complete Student Profile in Software Requirements Specification 3.2.7.

The Student advances through the onboarding steps. Each step is validated on the server against its own schema and written before the next is shown, and the step reached is recorded on the profile, so a Student who leaves part way through resumes where they stopped rather than starting again. The profile strength is recalculated after each step and is what prompts the Student to supply the information that would most improve their matches.

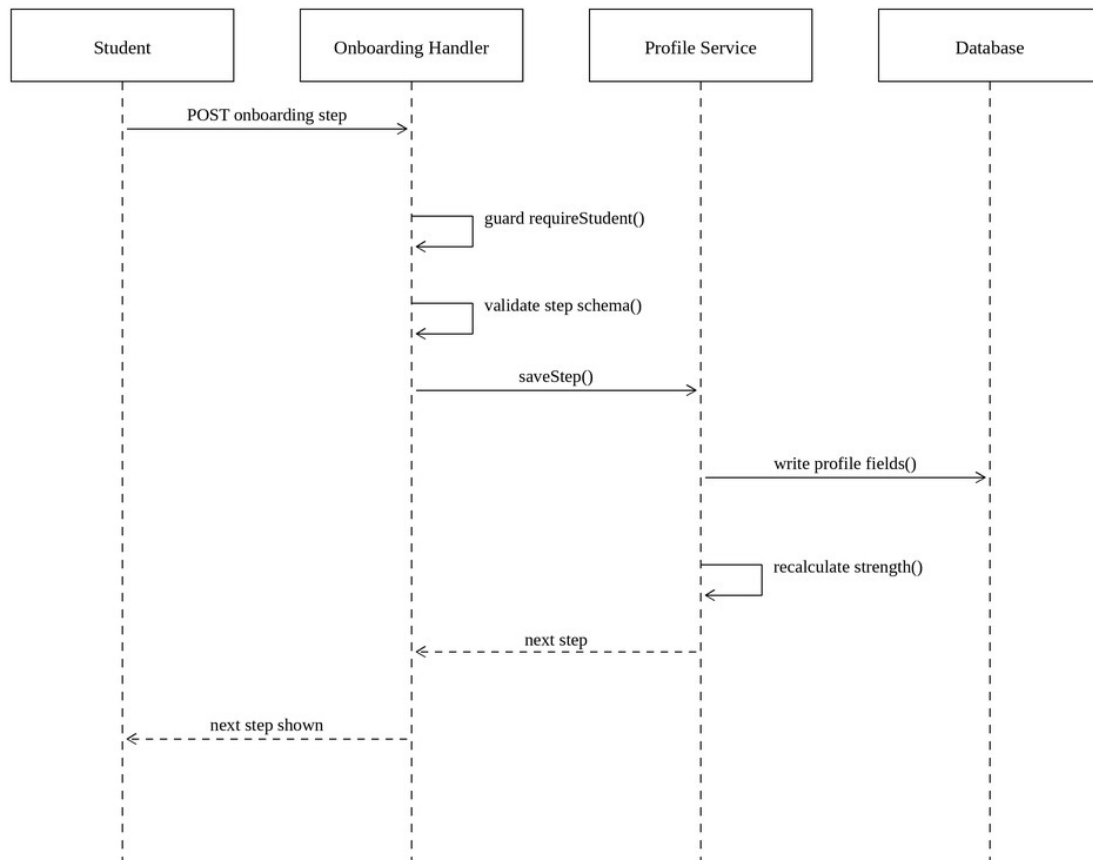


Figure 5 Complete Student Profile Sequence Diagram

Use Case: View Matched Opportunities

See also View Matched Opportunities in Software Requirements Specification 3.2.9.

The dashboard and the opportunities list both ask the service layer to rank the published programmes against the Student's profile. The Matching Engine scores each pairing across the six weighted criteria and returns the score with the reason for every criterion. The list is ordered by score, and each entry is displayed with its reasons; a percentage is never presented on its own.

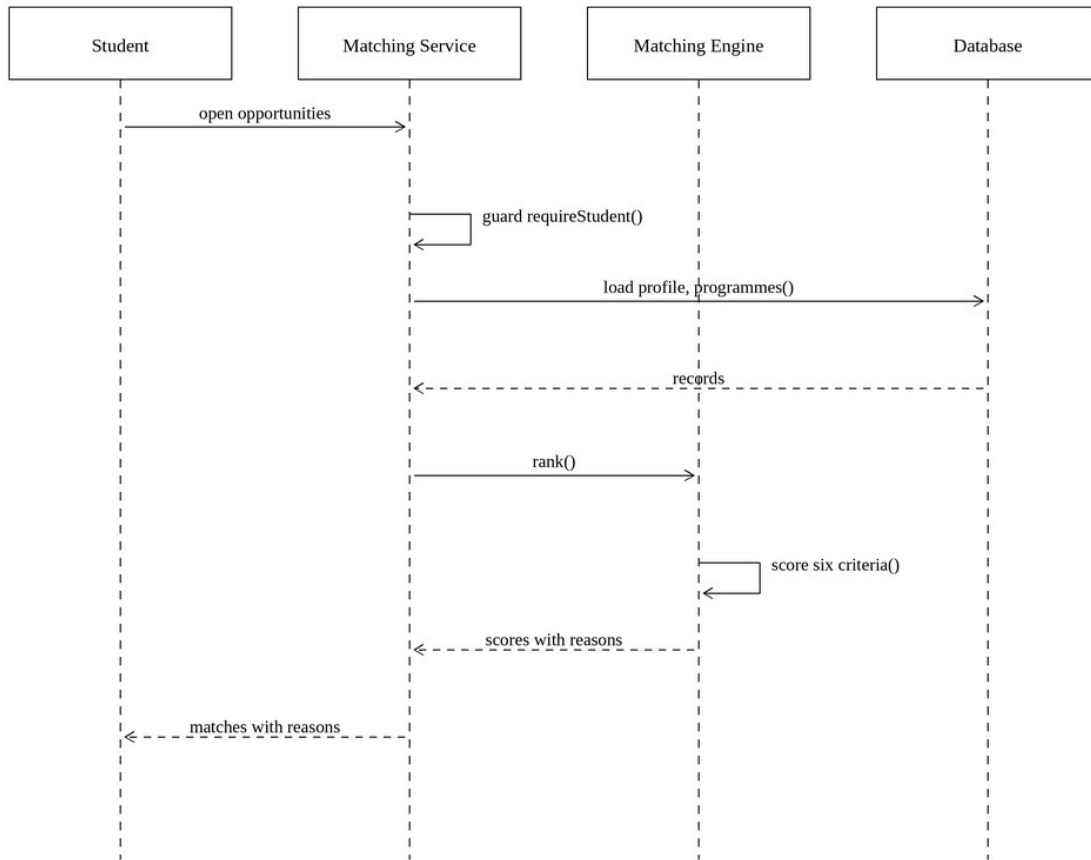


Figure 6 View Matched Opportunities Sequence Diagram

Use Case: Apply for Funding

See also Apply for Funding in Software Requirements Specification 3.2.12.

The Student submits an application against an open programme. The system scores the pairing and evaluates eligibility at that moment and stores both results on the application, so the Funder later reviews the assessment as it stood at submission. A notification is raised for the Student and the action is appended to the audit log. A second application to the same programme is refused by a uniqueness constraint rather than by a check that could be omitted.

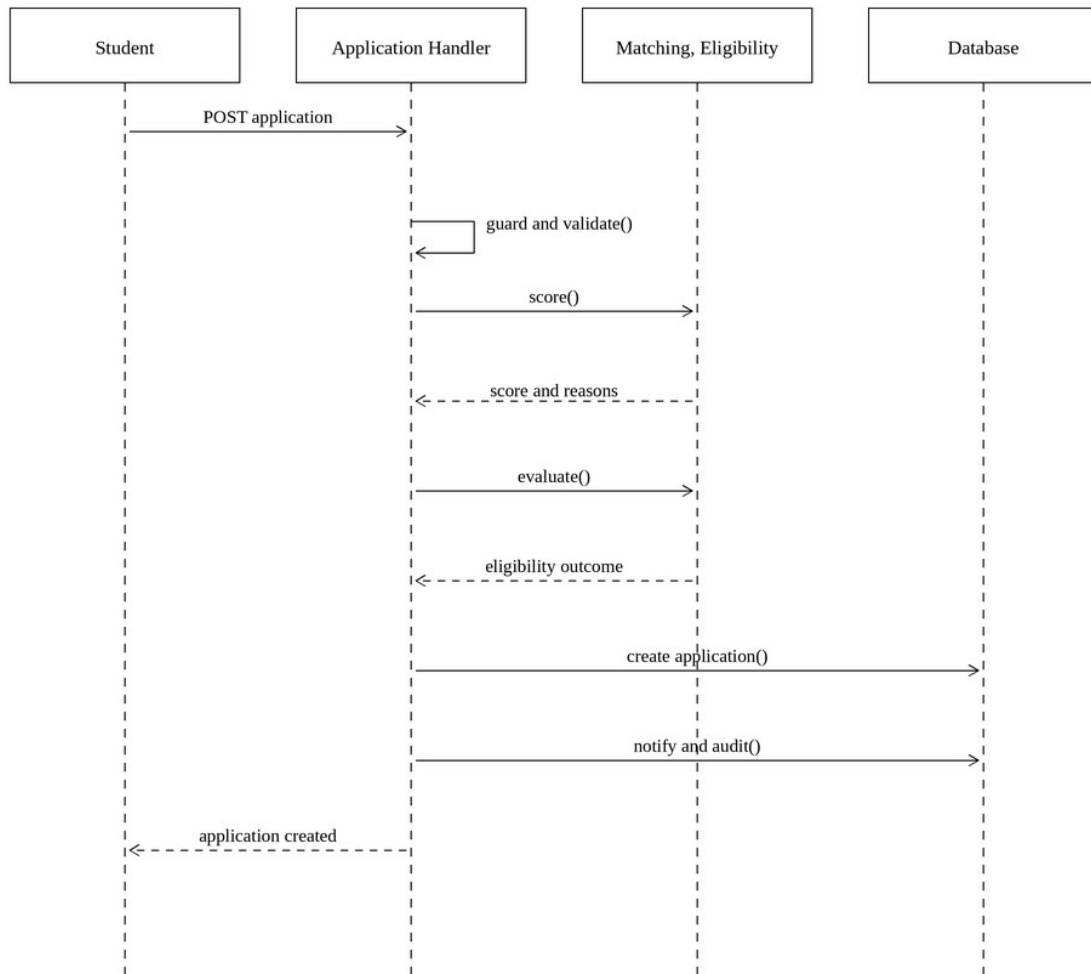


Figure 7 Apply for Funding Sequence Diagram

Use Case: Browse All Bursaries

See also Browse All Bursaries in Software Requirements Specification 3.2.16.

The Student opens the directory of every published opportunity the system holds. The service reads the published programmes, derives a display status for each from its dates and its recorded availability, and orders the list so that those accepting applications come first. The directory is never narrowed to the Student's own profile: it answers what exists, where the matched list answers what fits. A closed opportunity stays listed and is marked closed, so a Student can see who funds what and when to return.

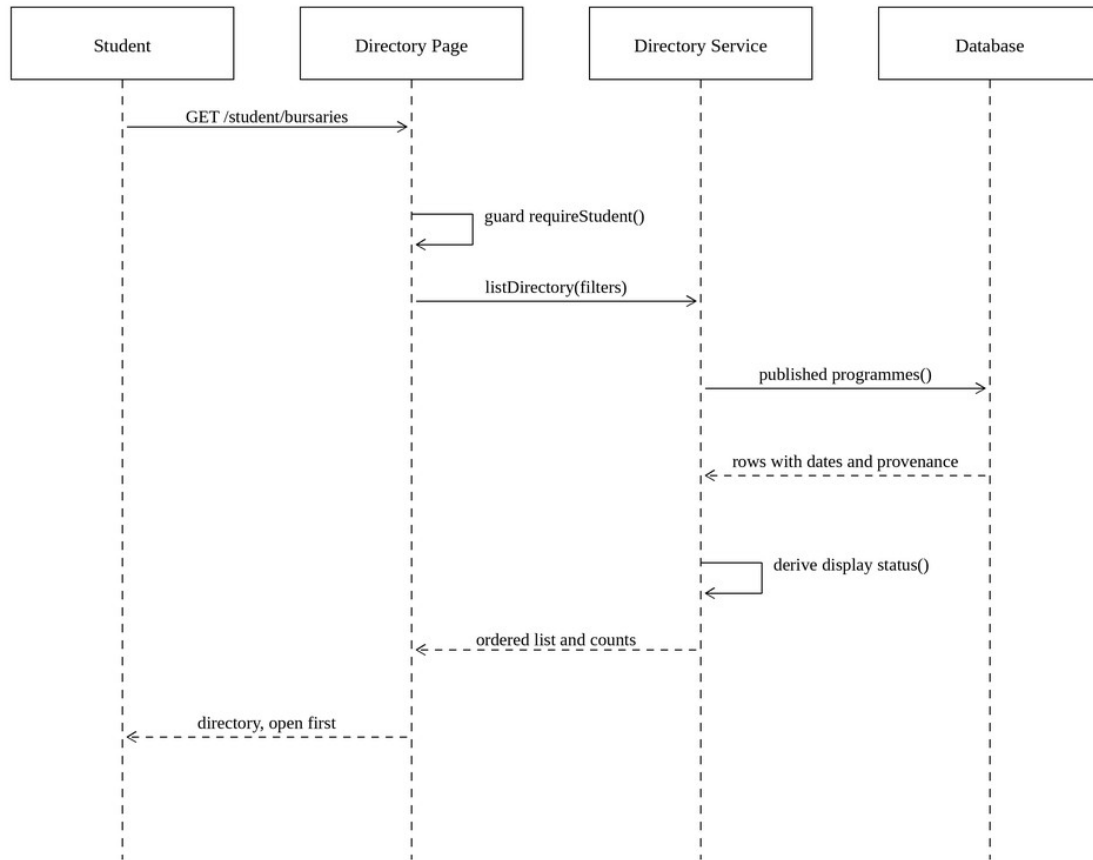


Figure 8 Browse All Bursaries Sequence Diagram

Use Case: Generate Motivational Letter

See also Generate Motivational Letter in Software Requirements Specification 3.2.17.

The Student chooses a programme and answers as many of the seven questions as they wish. The service gathers the facts held on their profile, the requirements the funder states, and the answers, and the Motivational Letter Composer assembles the draft from those alone. The letter is stored with the answers it was built from, so it can be traced to what the Student actually said and rewritten if they change it.

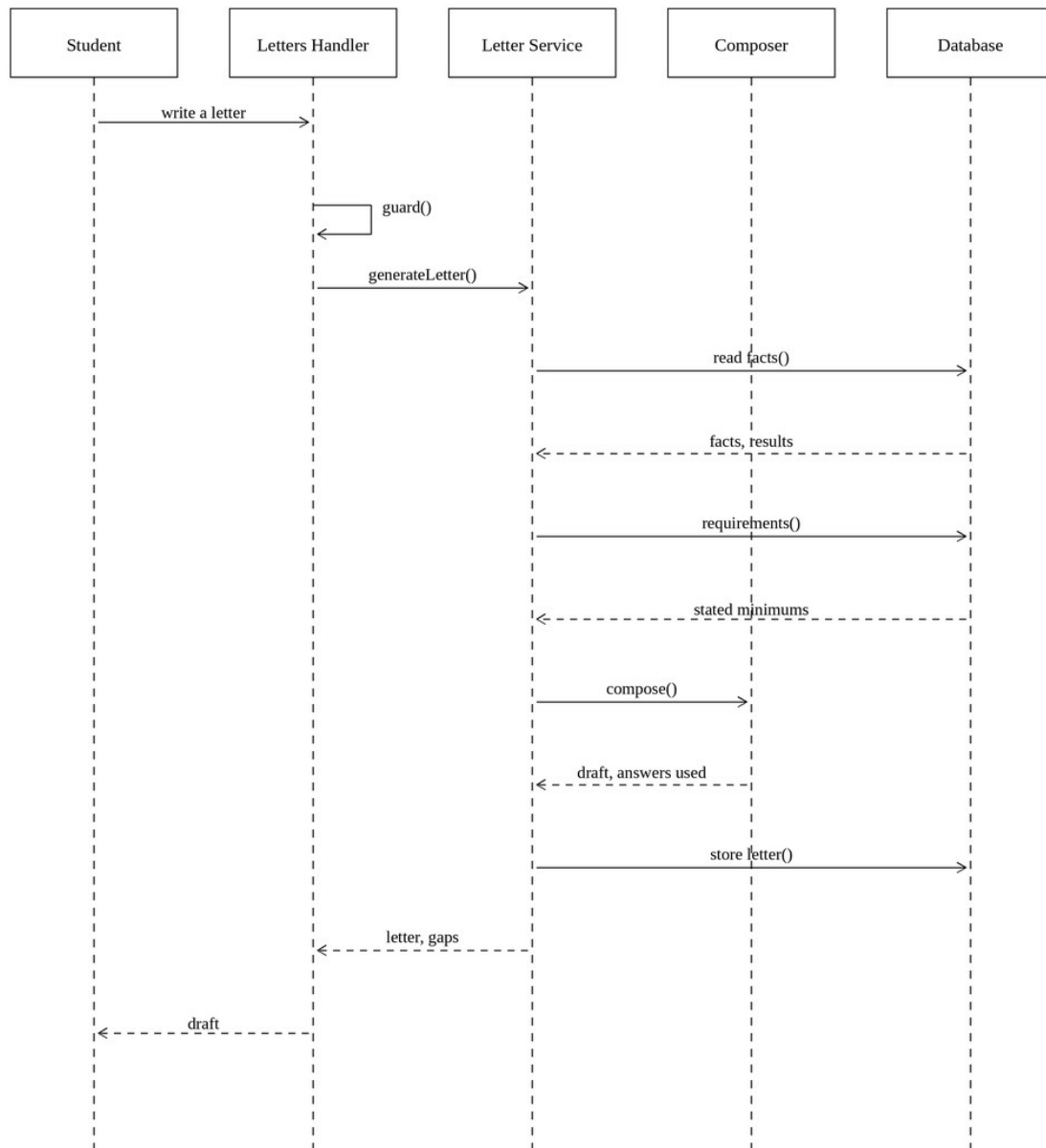


Figure 9 Generate Motivational Letter Sequence Diagram

Use Case: Auto-Fill Application Forms

See also Auto-Fill Application Forms in Software Requirements Specification 3.2.18.

The Student supplies one completed form and a number of blank ones. The Document Extraction Engine reads the completed form into canonical values, each carrying the confidence of the reading, then identifies the fields on each blank form and writes the values it can match. A field it cannot establish is left empty and flagged, because a blank a

Student can complete is better than a populated field that is wrong. A form that cannot be read fails on its own without stopping the others.

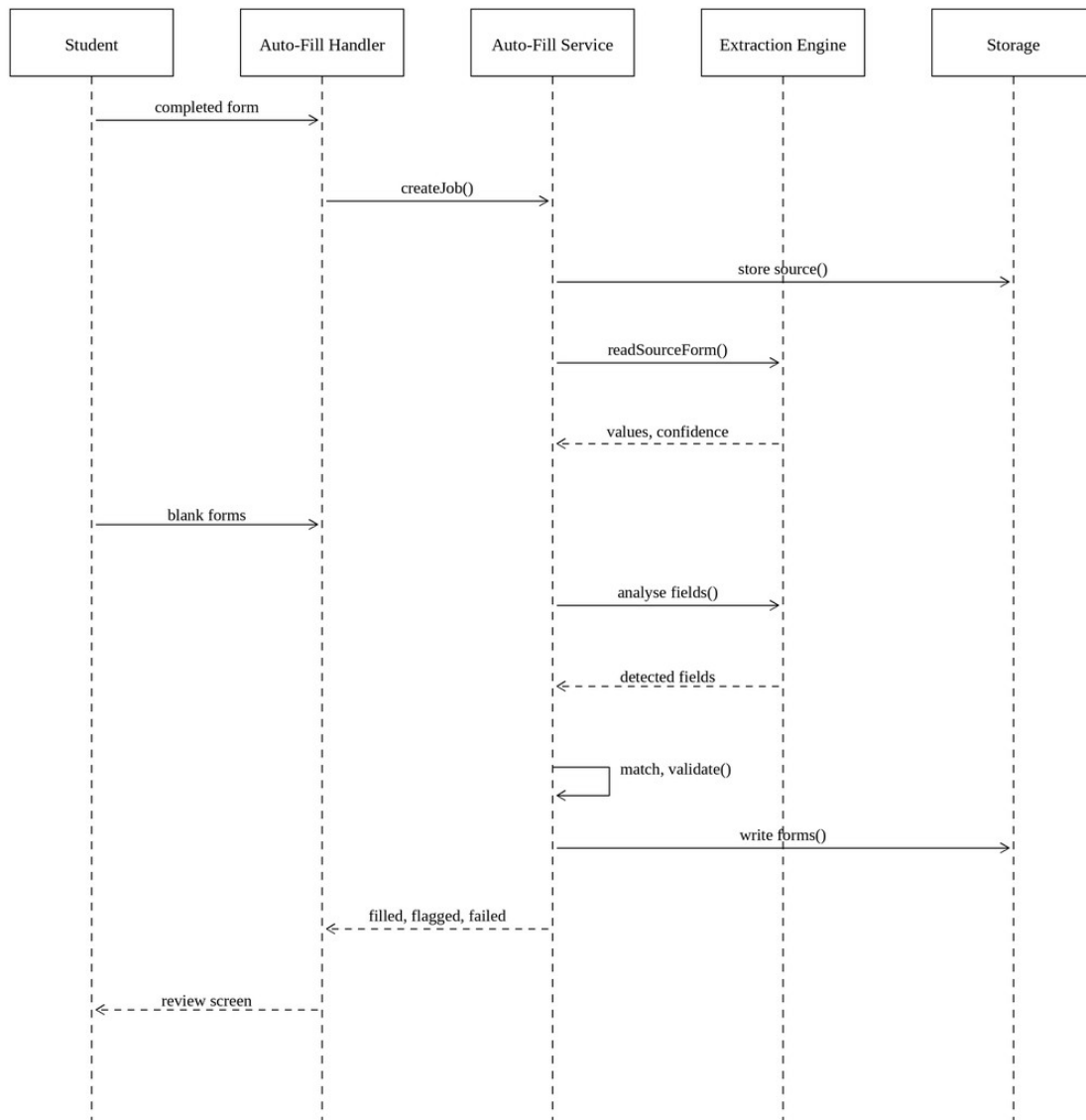


Figure 10 Auto-Fill Application Forms Sequence Diagram

Use Case: Create Funding Programme

See also Create Funding Programme in Software Requirements Specification 3.2.22.

The Corporate User describes a programme and states its eligibility criteria through ordinary form controls. The criteria are written to the Eligibility Rule as typed columns, not as prose, which is what allows the same conditions to drive both the Student's match score

and the Funder's eligibility verdict. The programme is created in draft and is invisible to Students until it is published.

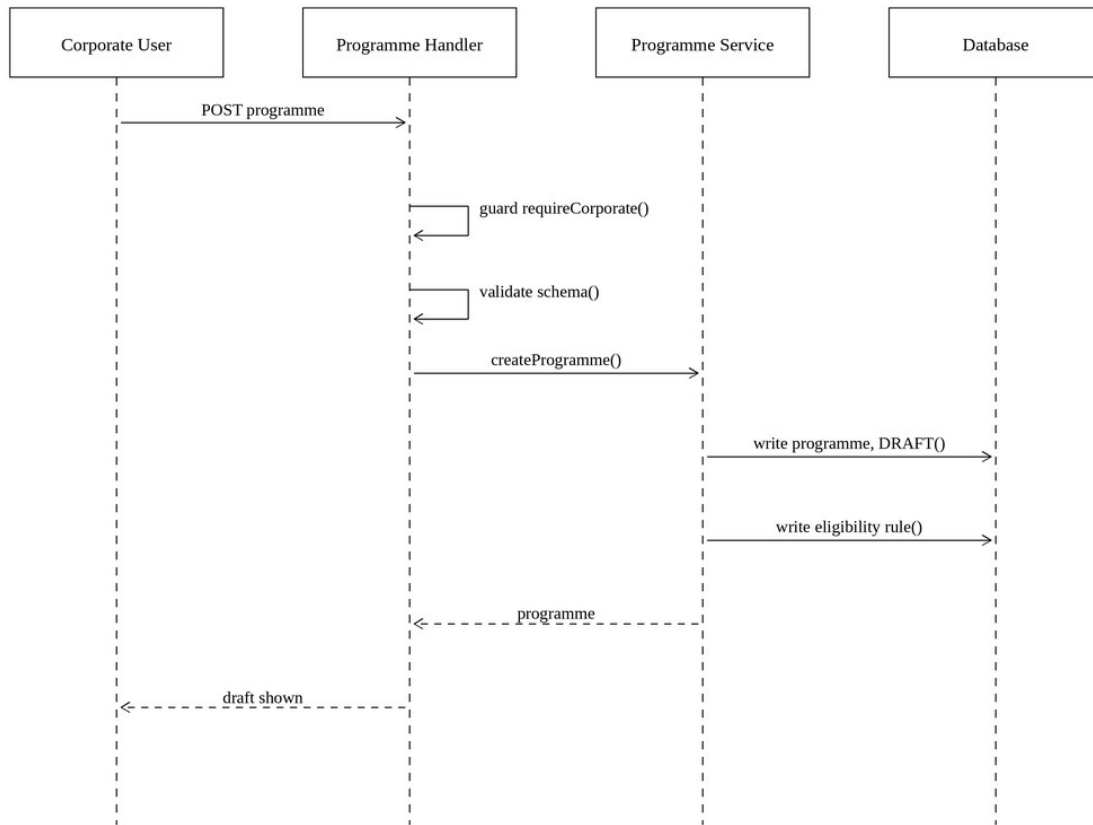


Figure 11 Create Funding Programme Sequence Diagram

Use Case: Import Applications

See also Import Applications in Software Requirements Specification 3.2.25.

The Corporate User uploads application forms the Organisation received elsewhere. Every file is stored before any of it is read, so the batch survives an interruption. Each form is read through the Document Extraction Engine, the applicant is looked for among existing Students on verified identifiers, the Matching Engine and the Eligibility Service score them against that programme's own criteria, and the supporting documents are counted against those the programme requires. Once every file has been read, one ordered pass flags duplicates. Nothing becomes an application until a Corporate User confirms the batch.

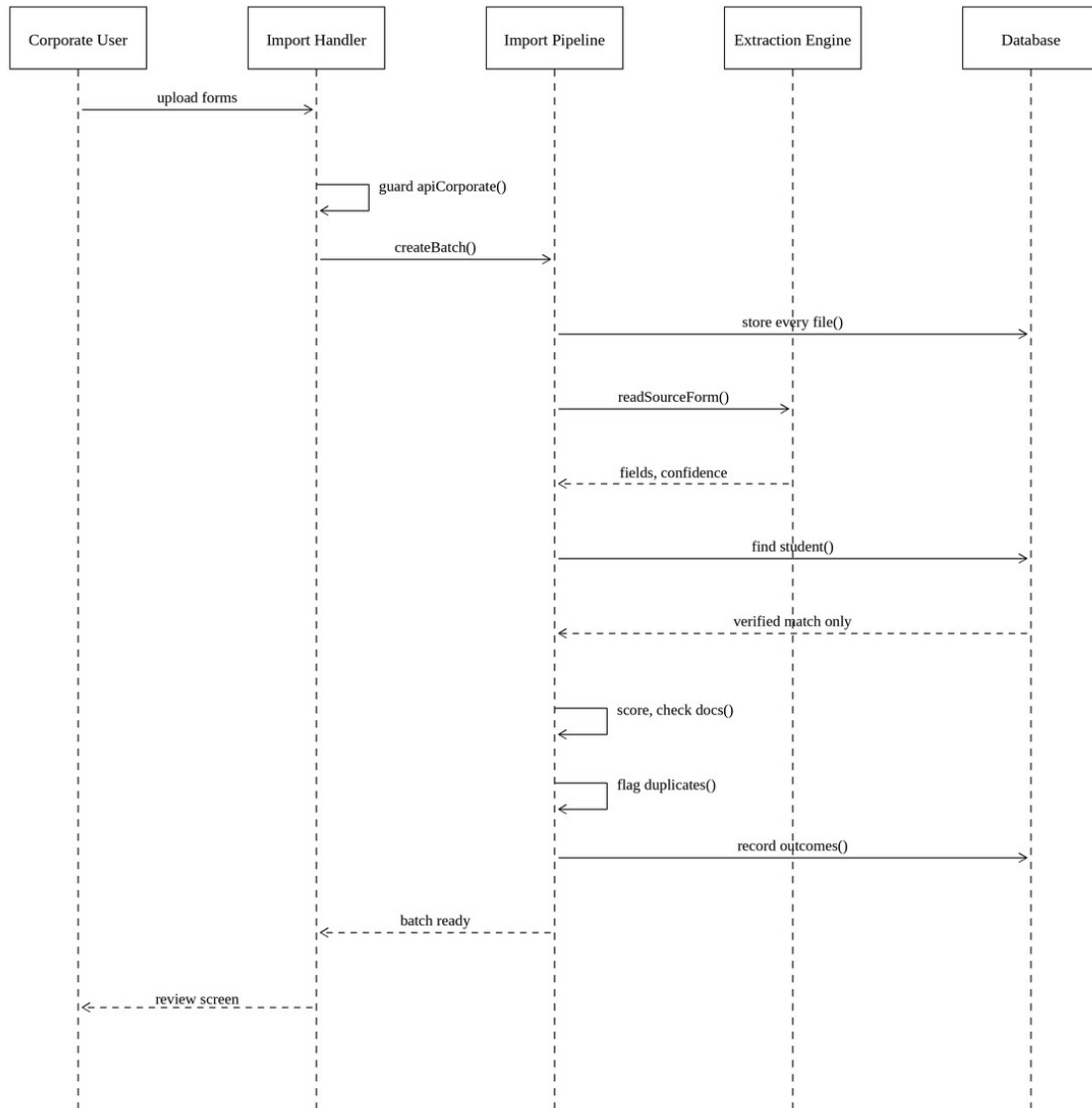


Figure 12 Import Applications Sequence Diagram

Use Case: Review Imported Applications

See also Review Imported Applications in Software Requirements Specification 3.2.26.

The Corporate User examines a processed batch. Each application is shown beside the document it was read from, with every extracted field, its confidence and the label it came from. A correction is stored alongside the extracted value rather than replacing it, so the record keeps both what the form said and what a person decided it meant. On confirmation an application is created for each file marked for import, attached to the identified Student

where there was one and to a new external applicant record where there was not. The file is linked to the application it produced, so confirming twice cannot create it again.

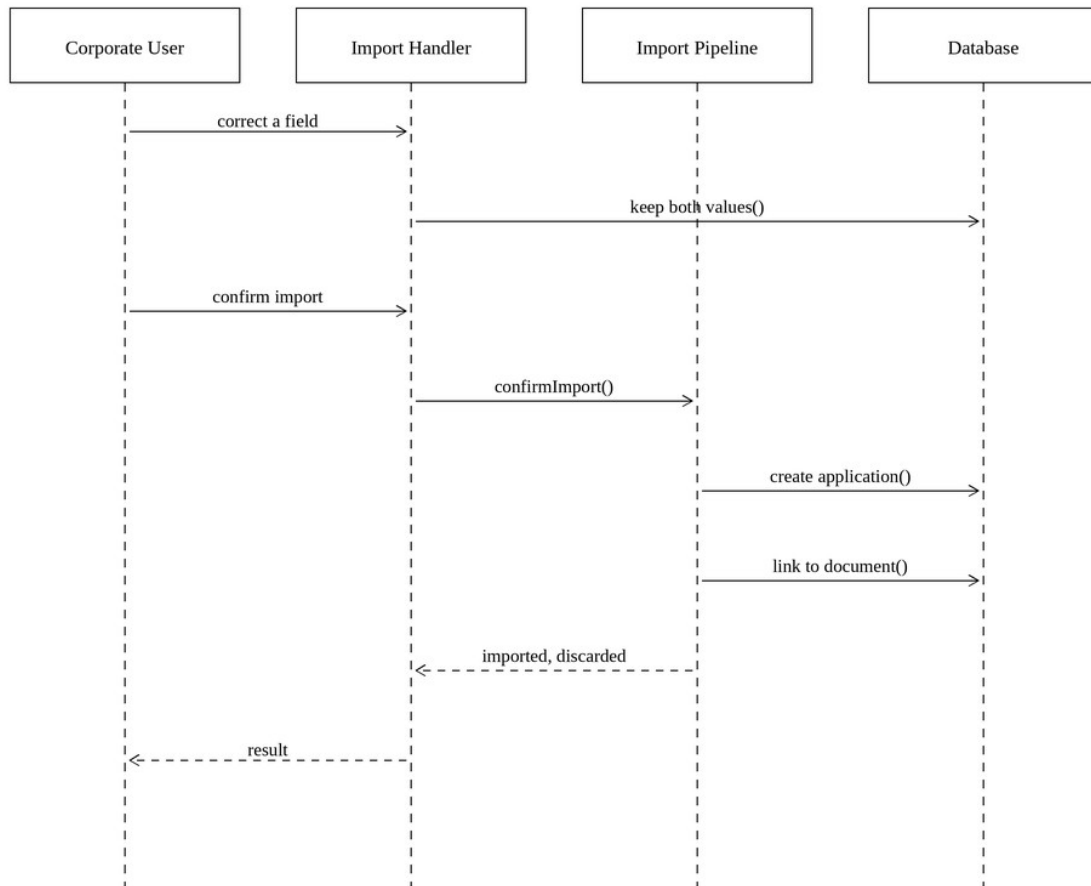


Figure 13 Review Imported Applications Sequence Diagram

Use Case: Review Applicant

See also Review Applicant in Software Requirements Specification 3.2.28.

The Corporate User opens an application belonging to their Organisation. The guard returns the organisation identifier and it forms part of the query, so an application belonging to another Organisation is not found rather than refused after being read. The Eligibility Service evaluates the applicant against the programme's rule and the verdict is displayed with the reason for each condition.

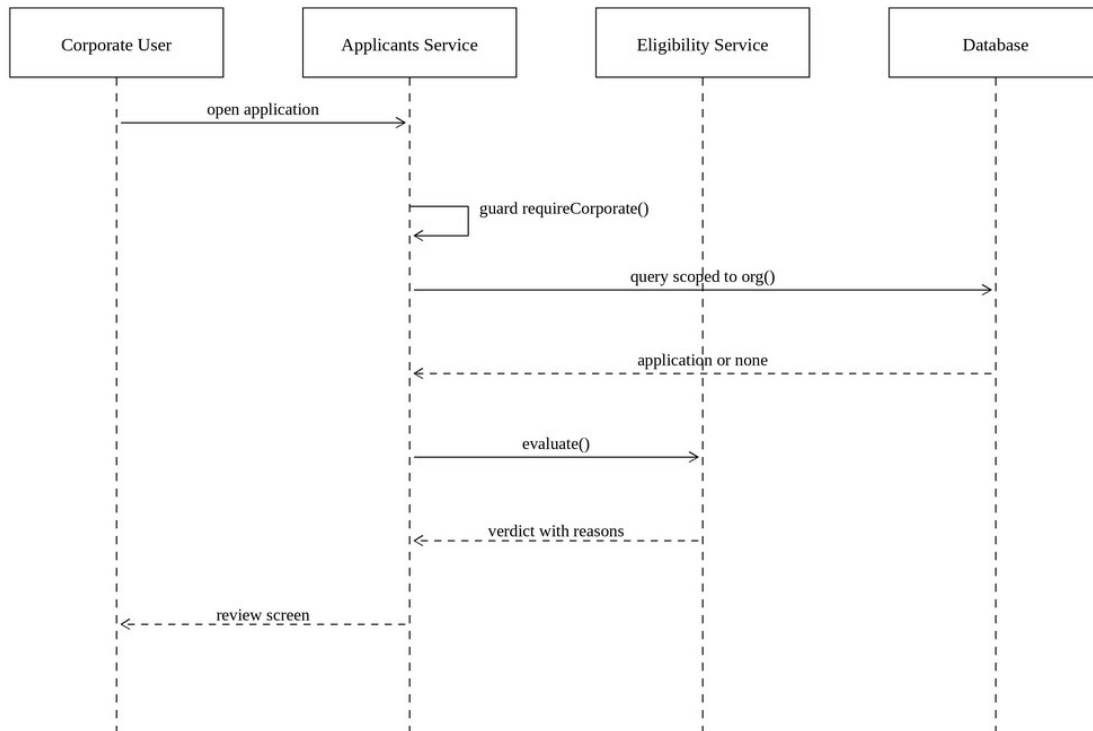


Figure 14 Review Applicant Sequence Diagram

Use Case: Shortlist Applicant

See also Shortlist Applicant in Software Requirements Specification 3.2.29.

The Corporate User adds an applicant to the shortlist for a programme. The application's status is advanced, a shortlist entry is created, the applicant is notified and the change is appended to the audit log. The operation is scoped to the Organisation throughout.

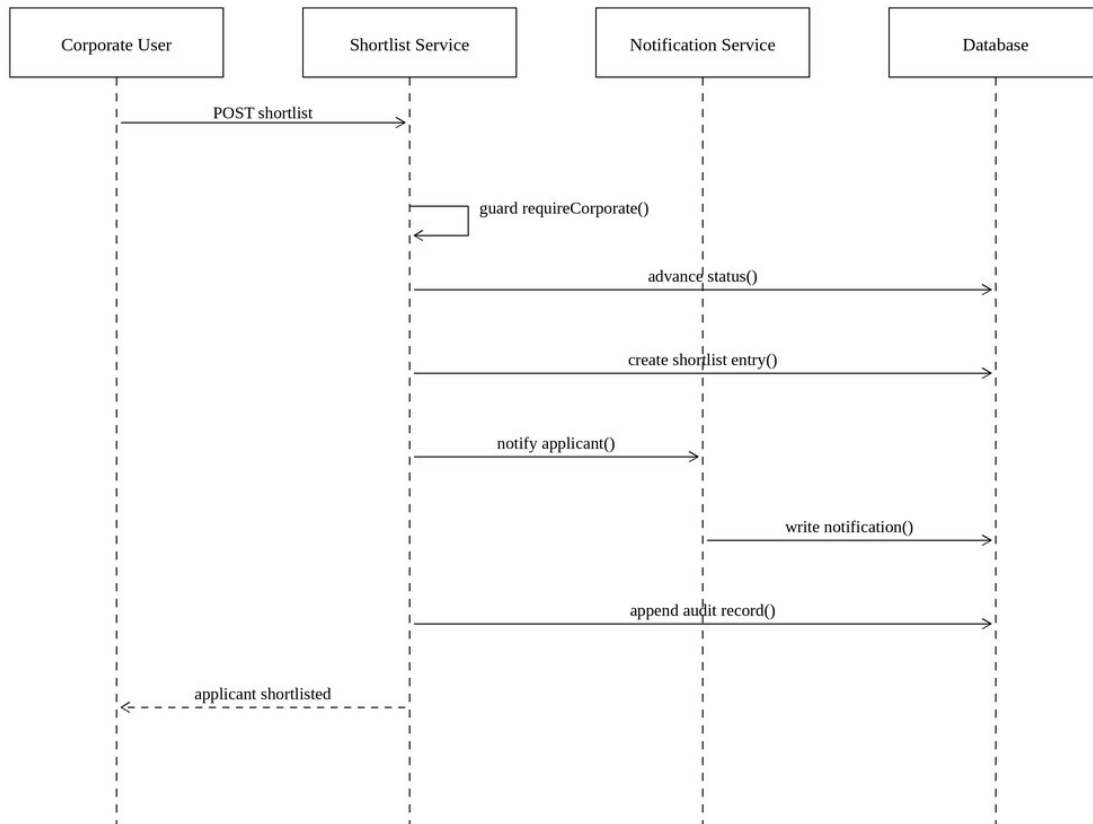


Figure 15 Shortlist Applicant Sequence Diagram

6.0. Interface design

The interface follows the approved reference designs, which are the visual source of truth for the system. It is laid out on a single palette and type scale expressed as Tailwind tokens, so that a change to a colour or a spacing step is made once and applies everywhere.

The layout is responsive rather than merely legible at small sizes. Many Students will reach the system on a mobile telephone, and some over a metered or intermittent connection, so the screens are laid out for narrow viewports as well as wide, wide content such as tables and reports scrolls within its own container rather than forcing the page sideways, and the profile is gathered in short steps that can be left and resumed.

Two principles govern the presentation of a match. The first is that a score is explained rather than merely asserted: the opportunity detail screen sets out every criterion, states whether it was met and names the weight it carried, so a Student can see why a programme scored as it did and what would improve it. The second is that a criterion the system could not evaluate is labelled as needing verification rather than being silently treated as a failure.

The screens reproduced below are captured from the running system rather than drawn, so they show the interface as implemented.

The Student-facing screens are captured against the real directory, whose bursaries were imported from published sources, so the programmes, funders and closing dates on them are the ones the system actually holds. The Organisation-facing screens are different: they show applications, shortlists and beneficiaries, and none of those exists until an Organisation has published a programme through the system and Students have applied to it, which has not yet happened. Those screens are therefore captured against a demonstration Organisation and demonstration Applicants created for the purpose, and the names, averages and figures on them are not real people or real awards. That data was removed from the database once the captures were taken.

Figures 16 to 46 present the implemented screens in the order a user meets them: the public pages first, then the Student portal, then the Corporate portal, and finally the same screens again at tablet and mobile widths.

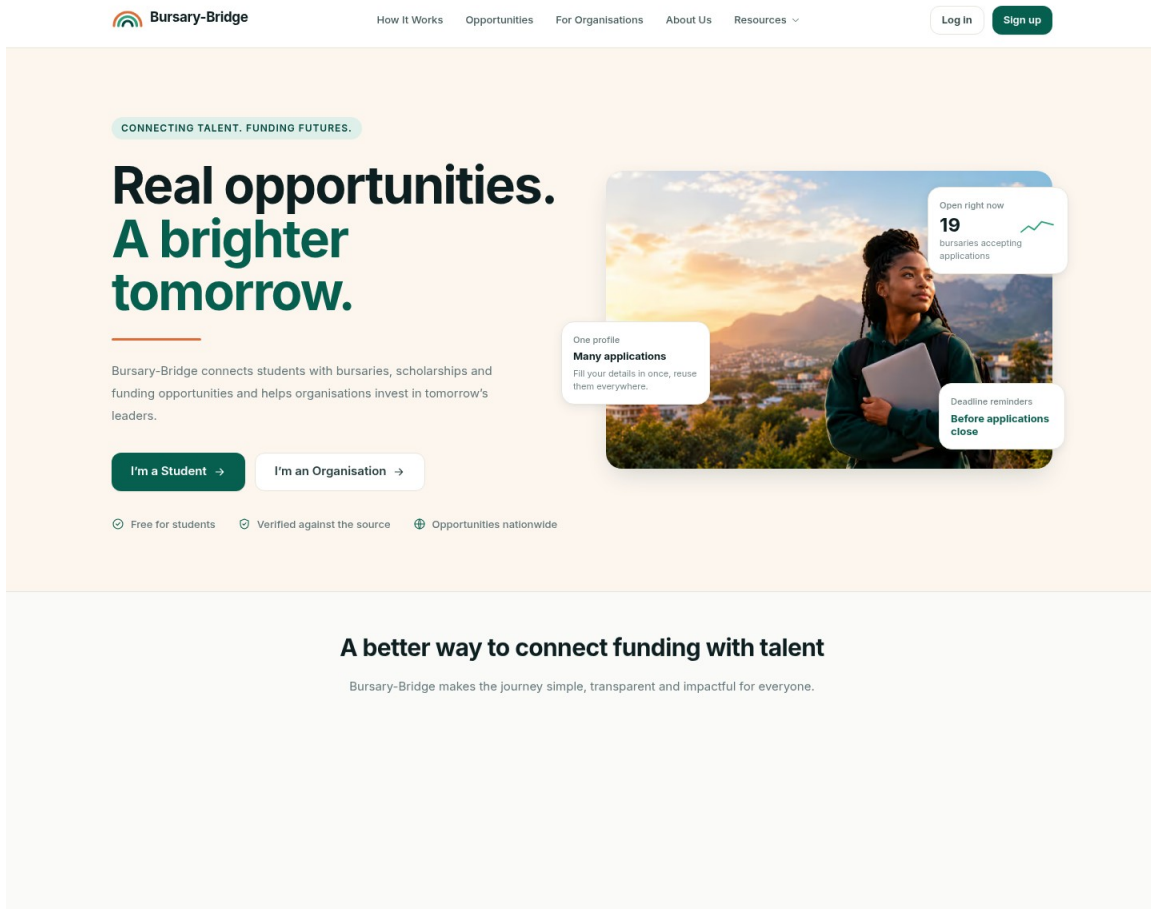


Figure 16 Landing page

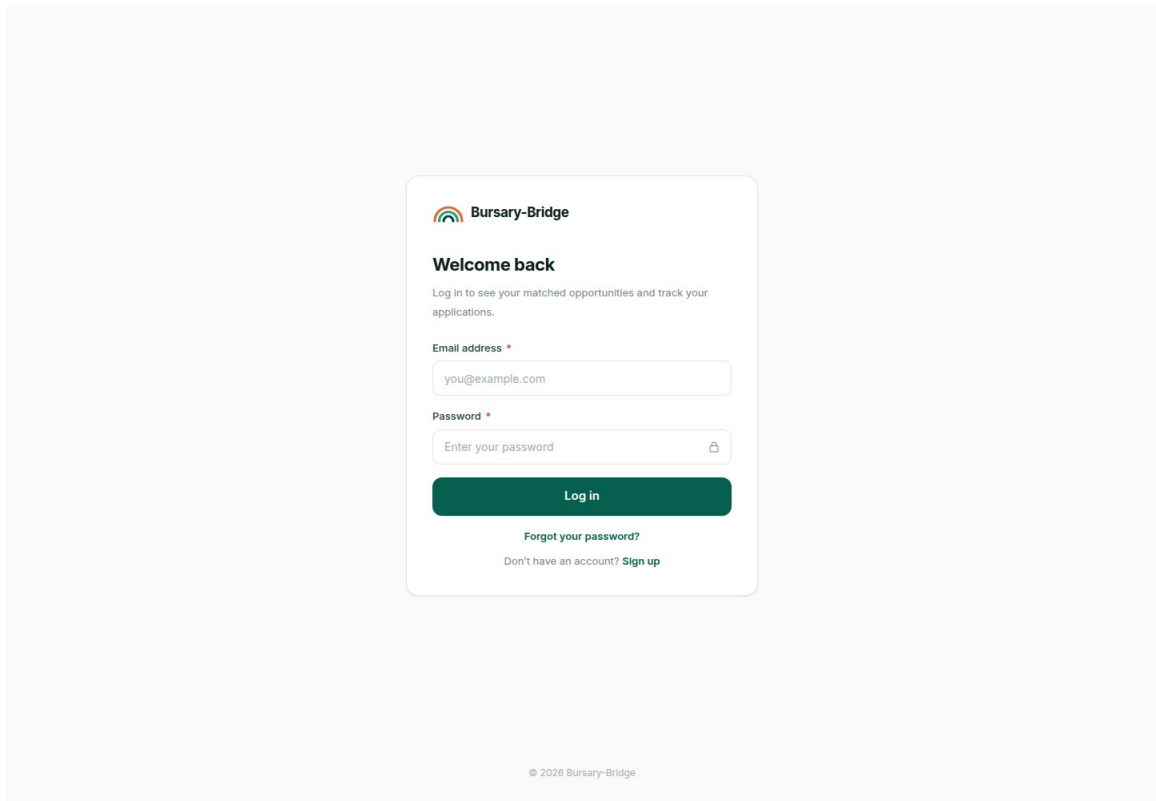


Figure 17 Log in page

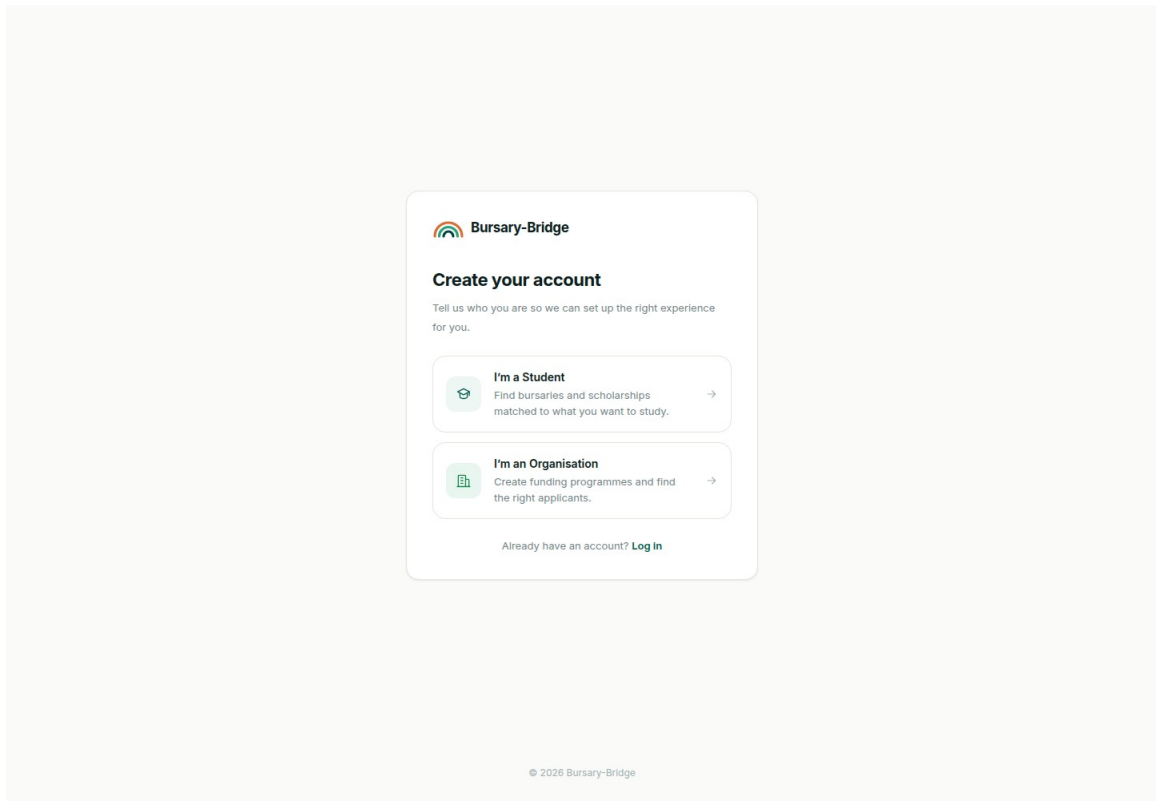


Figure 18 Registration, choice of account type

 **Bursary-Bridge**

Create your account

Create your account to discover funding opportunities matched to your education journey.

First Name *

Last Name *

Email Address *

you@example.com

Mobile Number *

082 123 4567

Password *

Confirm Password *

☒ Receive bursary, scholarship and funding opportunity notifications via email

☐ I agree to the Bursary-Bridge Terms of Service and Privacy Notice.

Create Account

[Already have an account? Log in](#)

© 2026 Bursary-Bridge

Figure 19 Student registration

 **Bursary-Bridge**

Create your corporate account

Join Bursary-Bridge to manage funding programmes and connect with students.

First Name *

Last name *

Work email *

you@organisation.co.za

Mobile number *

082 123 4567

Password *

Confirm password *

☒ Receive Bursary-Bridge notifications via email

☐ I agree to the Bursary-Bridge Terms of Service and Privacy Notice.

Create Account

Already have an account? [Log in](#)

© 2026 Bursary-Bridge

Figure 20 Organisation registration

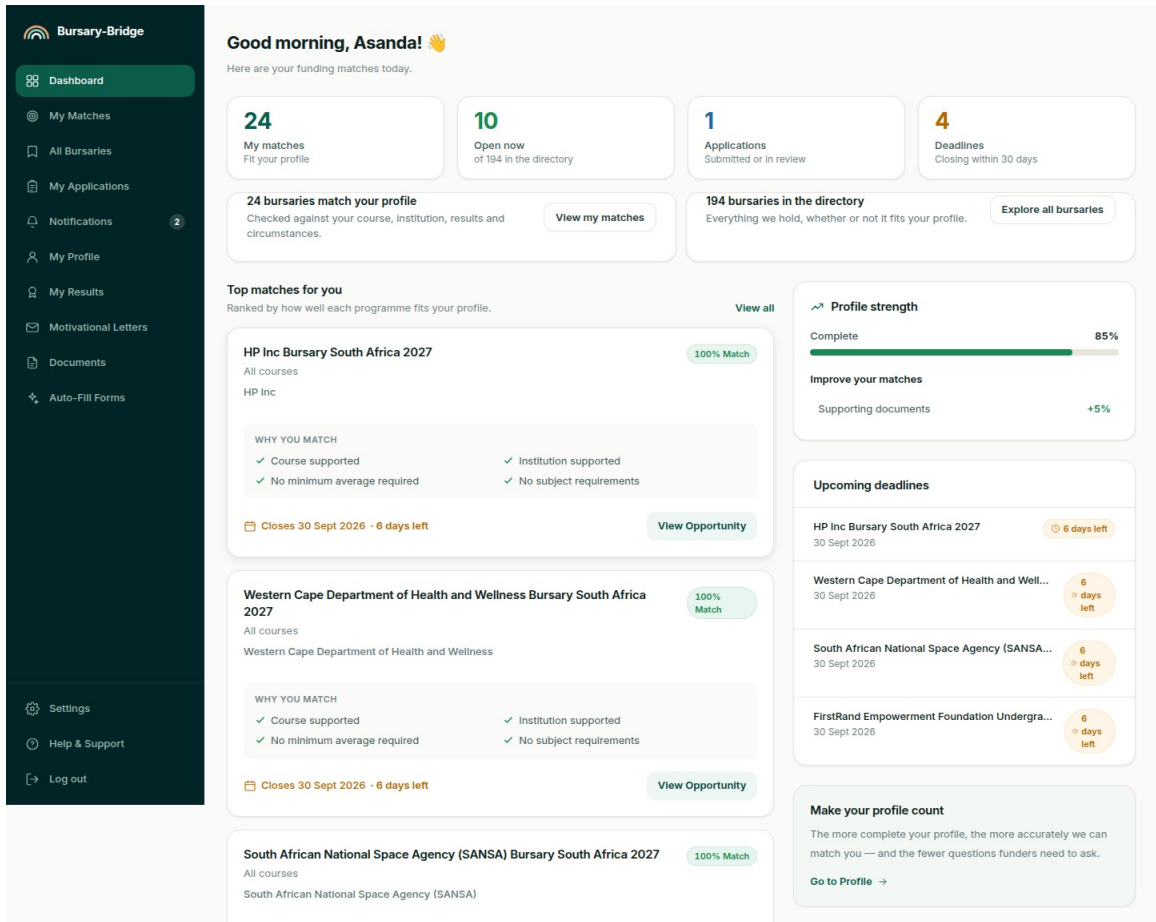


Figure 21 Student dashboard

Bursary-Bridge

Dashboard

My Matches

All Bursaries

My Applications

Notifications2

My Profile

My Results

Motivational Letters

Documents

Auto-Fill Forms

Settings

Help & Support

Log out

My Profile

Create it once, and every application reuses it. Keep it current for the most accurate matches.

Personal

Edit

Name

Asanda Nsibande

Email

student@demo.bursarybridge.local

Mobile

0821234567

Education

Edit

Education stage

University / College — Current Student

Qualification level

Bachelor's Degree

Study status

Currently enrolled

Current institution

University of Pretoria

Current programme

Computer Science

Year of study

Year 2

Study Preferences

Edit

Preference 1

Computer Science · University of Pretoria

Preference 2

Computer Science · University of Johannesburg

Preference 3

Information Technology · Tshwane University of Technology

Preference 4

Data Science · University of the Witwatersrand

Academic

Edit

Latest average

82%

Results available

Matric results, University transcript

Achievements

Subject distinctions, Academic awards

Funding Needs

Edit

Needs funding for

Tuition fees, Accommodation, Books / study materials, Meals / living allowance, Full funding

Current situation

I currently have no funding

Financial

Edit

Household income

R100,001 – R200,000

Receiving a bursary

No

Eligibility

Edit

Date of birth

26 May 2004

Citizenship

South African citizen

First-generation student

No

Disability

No

Edit study preferences

Profile strength

Complete

85%

Improve your matches

Supporting documents

+5%

Completed

✓ Education journey

✓ Study preferences

✓ Academic results

✓ Achievements

✓ Funding needs

✓ Financial profile

✓ Personal eligibility

✓ Location & interests

Why we ask

Funders set rules about courses, institutions, results, income and location. The more of your profile we hold, the more accurately we can tell you what you qualify for — and the fewer questions you face at application time.

Figure 22 Student profile

Page 57 of 83

Bursary-Bridge

Dashboard

My Matches

All Bursaries

My Applications

Notifications2

My Profile

My Results

Motivational Letters

Documents

Auto-Fill Forms

Settings

Help & Support

Log out

Opportunities

24 funding opportunities are open right now, ranked by how well they match your profile.

Best match

Filters

Showing 8 of 24 opportunities

HP Inc Bursary South Africa 2027

Bursary

100% Match

HP Inc

HP Inc. is an information technology company known for its personal computers (PCs), printers and related supplies, as well as 3D printing solutions. HP Inc. was formed in 2015, as the legal successor of the original Hewlett-Packard Company (which was formed in 1939), after the original Hewlett-Pack

SUPPORTED COURSES

All courses

SUPPORTED INSTITUTIONS

All institutions

Closes 30 Sept 2026 · 6 days left

View Opportunity

Western Cape Department of Health and Wellness Bursary South Africa 2027

Bursary

100% Match

Western Cape Department of Health and Wellness

The Western Cape Department of Health and Wellness, formed in 1994, is responsible for delivering public healthcare and health services to people who reside within the province. They are also in charge of various specialised facilities, which include rehabilitation centres; psychiatric hospitals and

SUPPORTED COURSES

All courses

SUPPORTED INSTITUTIONS

All institutions

Closes 30 Sept 2026 · 6 days left

View Opportunity

South African National Space Agency (SANSa) Bursary South Africa 2027

Bursary

100% Match

South African National Space Agency (SANSa)

The South African National Space Agency (SANSa), established in 2010, are a public entity and government agency operating under the National Department of Science and Technology (DST). SANSa are responsible developing and promoting aeronautics and aerospace space research, with the companies main fo

SUPPORTED COURSES

All courses

SUPPORTED INSTITUTIONS

All institutions

Closes 30 Sept 2026 · 6 days left

View Opportunity

Figure 23 Opportunities, ranked by match

Page 58 of 83

- Dashboard
- My Matches**
- All Bursaries
- My Applications
- Notifications 2
- My Profile
- My Results
- Motivational Letters
- Documents
- Auto-Fill Forms
- Settings
- Help & Support
- Log out

[← Back to opportunities](#)

Bursary

100% Match

HP Inc Bursary South Africa 2027

HP Inc

CLOSING
30 Sept 2026
6 days left

FUNDING
Partial funding
0 items covered

STUDY LEVEL
All levels

Overview

Eligibility

What's Covered

How to Apply

About this bursary

HP Inc. is an information technology company known for its personal computers (PCs), printers and related supplies, as well as 3D printing solutions. HP Inc. was formed in 2015, as the legal successor of the original Hewlett-Packard Company (which was formed in 1939), after the original Hewlett-Packard Company split its personal computer and printer divisions and its enterprise product and business

Supported courses

Open to all courses and programmes.

Supported institutions

Open to students at any recognised institution.

Why you match

100% Match

Strong Match

- ✓ **Course supported**
Course - worth 26%
- ✓ **Institution supported**
Institution - worth 22%
- ✓ **No minimum average required**
Academic requirement - worth 18%
- ✓ **No subject requirements**
Subject requirements - worth 12%
- ✓ **Open to all qualification levels**
Qualification level - worth 8%
- ✓ **Open to all provinces**
Location requirement - worth 8%
- ✓ **No financial requirement**
Financial requirement - worth 4%

Your profile is reused automatically — you'll only be asked for what this programme needs in addition.

Apply Now

Write a motivational letter

6 days left

About the funder

HP Inc

Other

South Africa

Figure 24 Opportunity detail, with the match explained


Bursary-Bridge

Dashboard

My Matches

All Bursaries

My Applications

Notifications 2

My Profile

My Results

Motivational Letters

Documents

Auto-Fill Forms

Settings

Help & Support

Log out

[← Back to opportunity](#)

Apply to HP Inc Bursary South Africa 2027

HP Inc · 6 days left

Step 1 of 4 Your Information

This is the information Bursary-Bridge will send with your application. You created it once — there's nothing to retype here.

Personal		Edit
Name	Asanda Nsibande	
Email	student@demo.bursarybridge.local	
Mobile	0821234567	
Education		Edit
Education stage	University / College — Current Student	
Qualification level	Bachelor's Degree	
Study status	Currently enrolled	
Current institution	University of Pretoria	
Current programme	Computer Science	
Year of study	Year 2	
Study Preferences		Edit
Preference 1	Computer Science · University of Pretoria	
Preference 2	Computer Science · University of Johannesburg	
Preference 3	Information Technology · Tshwane University of Technology	
Preference 4	Data Science · University of the Witwatersrand	
Academic		Edit
Latest average	82%	
Results available	Matric results, University transcript	
Achievements	Subject distinctions, Academic awards	
Funding Needs		Edit
Needs funding for	Tuition fees, Accommodation, Books / study materials, Meals / living allowance, Full funding	
Current situation	I currently have no funding	
Financial		Edit
Household income	R100,001 – R200,000	
Receiving a bursary	No	

Figure 25 Application form


Bursary-Bridge

- Dashboard
- My Matches
- All Bursaries
- My Applications**
- Notifications 2
- My Profile
- My Results
- Motivational Letters
- Documents
- Auto-Fill Forms

- Settings
- Help & Support
- Log out

My Applications

Track every application you've started or submitted.

[All \(2\)](#)
[Drafts \(1\)](#)
[Submitted \(0\)](#)
[In Review \(1\)](#)
[Shortlisted \(0\)](#)
[Decided \(0\)](#)

Mining & Geoscience Bursary
Draft

Amandla Mining Group

Saved 24 Sept 2026
Closes 28 Dec 2026
Last update 24 Sept 2026

2026 Technology Bursary Programme
100% Match
Documents Required

Kgotsotso Holdings

Applied 08 Sept 2026
Closes 24 Oct 2026
Last update 08 Sept 2026

Figure 26 Student application tracker

Bursary-Bridge

Dashboard

My Matches

All Bursaries

My Applications

Notifications 2

My Profile

My Results

Motivational Letters

Documents

Auto-Fill Forms

Settings

Help & Support

Log out

Back to my applications

Mining & Geoscience Bursary

Amandla Mining Group

Draft

DATE APPLIED

Not submitted

CLOSING DATE

28 Dec 2026

LAST UPDATE

24 Sept 2026

Continue Application

Profile sent with this application

Captured when you applied. Editing your profile does not change a submitted application.

Personal

View

Name

Asanda Nsibande

Email

student@demo.bursarybridge.local

Mobile

0821234567

Education

View

Education stage

University / College — Current Student

Qualification level

Bachelor's Degree

Study status

Currently enrolled

Current Institution

University of Pretoria

Current programme

Computer Science

Year of study

Year 2

Study Preferences

View

Preference 1

Computer Science · University of Pretoria

Preference 2

Computer Science · University of Johannesburg

Preference 3

Information Technology · Tshwane University of Technology

Preference 4

Data Science · University of the Witwatersrand

Academic

View

Latest average

82%

Results available

Matric results, University transcript

Achievements

Subject distinctions, Academic awards

Funding Needs

View

Needs funding for

Tuition fees, Accommodation, Books / study materials, Meals / living

Progress

Application started

24 Sept 2026

Submitted

Under review by funder

Shortlisted

Decision

Documents

No documents attached.

REQUIRED BY THIS PROGRAMME

ID document

Academic record

Proof of residence

Applications close 28 Dec 2026

View opportunity

Figure 27 Application detail

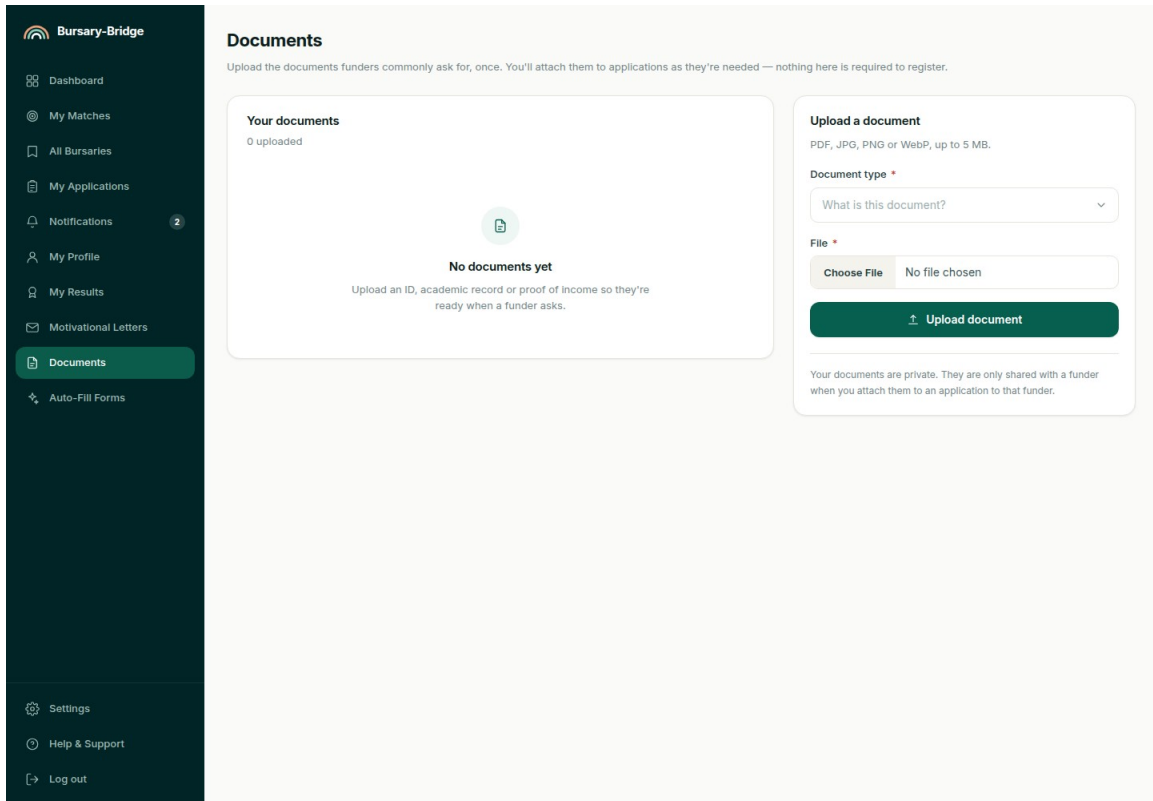


Figure 28 Student document library

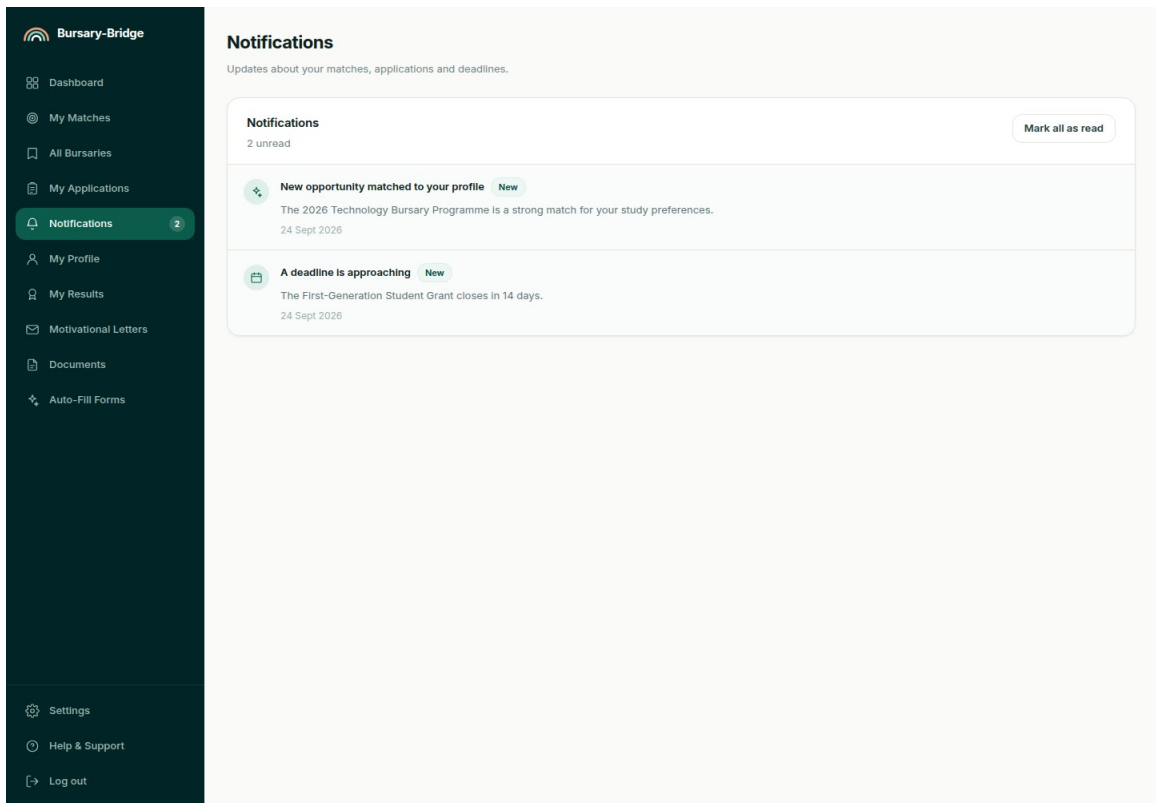


Figure 29 Student notifications

 Bursary-Bridge

Dashboard

My Matches

All Bursaries

My Applications

Notifications2

My Profile

My Results

Motivational Letters

Documents

Auto-Fill Forms

Settings

Help & Support

Log out

Settings

Manage your account details and password.

Account details

Used to identify you and to reach you.

First name *

Last name *

Asanda

Nsibande

Email address *

student@demo.bursarybridge.local

Mobile number *

0821234567

☒ Receive Bursary-Bridge notifications via email

Save changes

Password

Changing your password signs you out of every device.

Current password *

New password *

Confirm new password *

Update password

Figure 30 Student account settings

Page 65 of 83

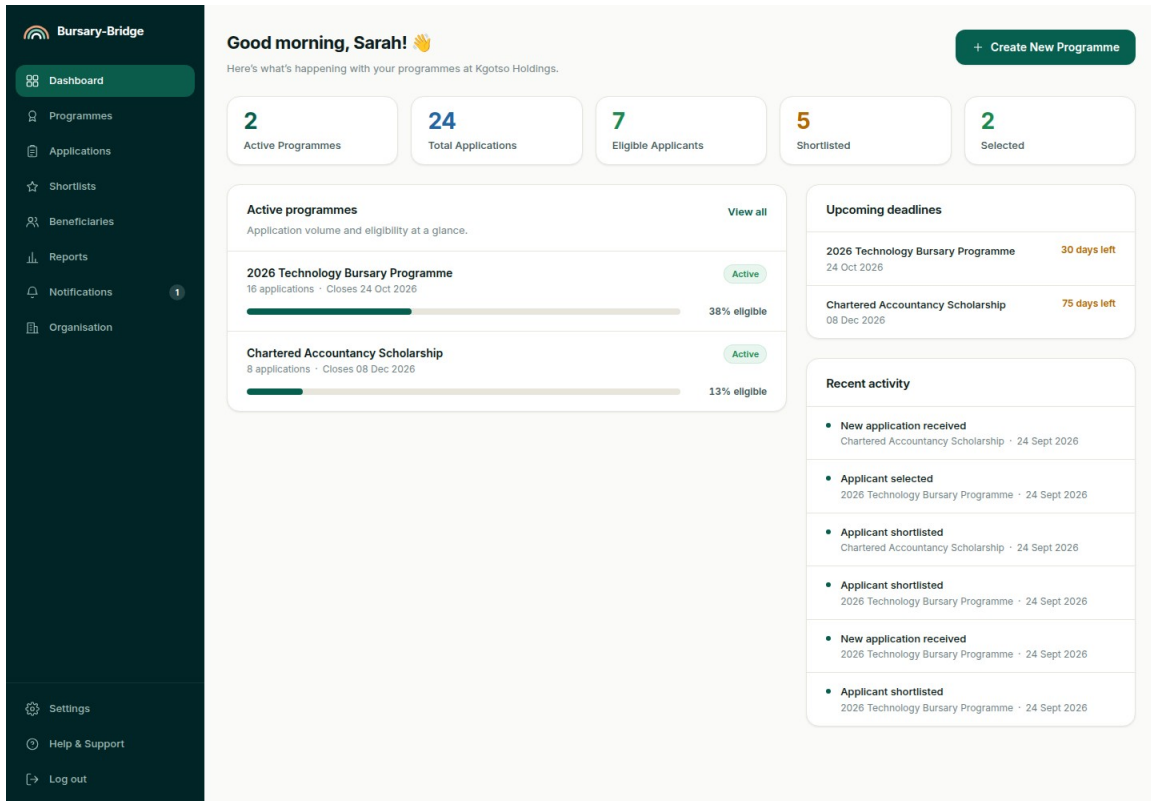


Figure 31 Corporate dashboard

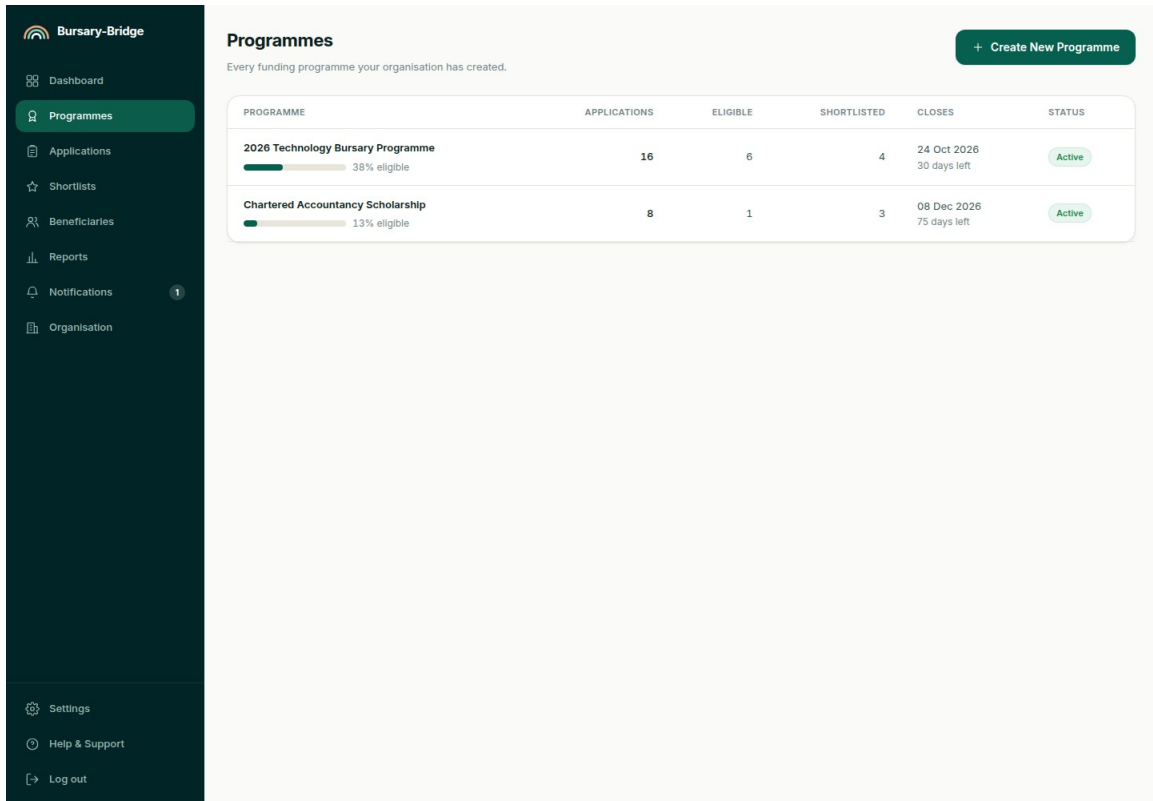


Figure 32 Funding programmes held by the Organisation

Bursary-Bridge

Dashboard

Programmes

Applications

Shortlists

Beneficiaries

Reports

Notifications

Organisation

Settings

Help & Support

Log out

Programmes > Create New Programme

Create New Programme

Set up your funding programme and eligibility criteria.

1 Programme Details

2 Eligibility Criteria

3 Funding & Benefits

4 Documents & Questions

5 Review & Publish

Programme details

Set up your funding programme. Students see this information on the opportunity page.

Programme name *

e.g. 2026 Engineering Bursary

Programme description *

A one-line summary shown on opportunity cards.

Supporting talented engineering students pursuing accredited degrees.

Full description *

The detail students read before applying.

Programme type *

Select a programme type

Application open date *

09/24/2026

Application closing date *

mm/dd/yyyy

Approximate number of awards Optional

Shown to students as an indication of how many awards are available.

Save as Draft

Save & Continue

Figure 33 Funding programme editor

Page 68 of 83

Bursary-Bridge

Dashboard

Programmes

Applications

Shortlists

Beneficiaries

Reports

Notifications1

Organisation

Settings

Help & Support

Log out

Applications

Every application submitted to your organisation's programmes.

All (24)SubmittedIn Review (6)Shortlisted (5)Selected (2)Rejected (5)

Q Search applicants by name or email

All programmes

Filters

☐	APPLICANT	MATCH %	AVERAGE	ELIGIBILITY	STATUS	DATE APPLIED
☐	FM Farhaan Mthembu Software Engineering - Bachelor's Degree University of Cape Town	100% Match	86%	Eligible	Shortlisted	17 Sept 2026
☐	TB Thabo Botha Accounting - Bachelor's Degree University of Pretoria	100% Match	93%	Eligible	Submitted	12 Sept 2026
☐	PM Palesa Mabaso Computer Science - Bachelor's Degree University of Pretoria	100% Match	90%	Eligible	Unsuccessful	08 Sept 2026
☐	AN Asanda Nalibande Computer Science - Bachelor's Degree University of Pretoria	100% Match	82%	Eligible	Documents Required	08 Sept 2026
☐	PM Palesa Motaung Computer Science - Bachelor's Degree University of Pretoria	100% Match	86%	Eligible	Documents Required	06 Sept 2026
☐	MD Mpho Dube Software Engineering - Bachelor's Degree University of Cape Town	100% Match	74%	Eligible	Submitted	22 Aug 2026
☐	KN Kagiso Nalidoo Computer Science - Bachelor's Degree University of Pretoria	100% Match	93%	Eligible	Shortlisted	20 Aug 2026
☐	AZ Amahle Zulu Accounting - Bachelor's Degree University of Pretoria	89% Match	—	Pending verification	Shortlisted	18 Sept 2026
☐	MM Megan Mabaso Software Engineering - Bachelor's Degree University of Cape Town	89% Match	—	Pending verification	Unsuccessful	15 Sept 2026
☐	TJ Tumelo Jacobs Computer Science - Bachelor's Degree University of Pretoria	89% Match	—	Pending verification	Submitted	31 Aug 2026
☐	TV Thabo Van Zyl Software Engineering - Bachelor's Degree	89% Match	—	Pending verification	Under Review	17 Aug 2026

Figure 34 Applications received

Bursary-Bridge

Dashboard

Programmes

Applications

Shortlists

Beneficiaries

Reports

Notifications

Organisation

Settings

Help & Support

Log out

Back to Applications

FM

Farhaan Mthembu

Software Engineering · Bachelor's Degree

University of Cape Town

100% Match

Eligible

Shortlisted

Overview

Eligibility

Documents (0)

Requests

Answers

Application details

DATE APPLIED

17 Sept 2026

PROGRAMME APPLIED FOR

2026 Technology Bursary Programme

STATUS

Shortlisted

Contact information

EMAIL

farhaan.mthembu.9@demo.bursarybridge.local

PHONE

0837246081

LOCATION

Durban, KwaZulu-Natal

Academic summary

LATEST AVERAGE

86%

QUALIFICATION

Bachelor's Degree

YEAR OF STUDY

Year 4

Achievements: Subject distinctions, Academic awards

Study preferences

1 Software Engineering · University of Cape Town

2 Cyber Security · University of Pretoria

3 Information Systems · North-West University

Funding profile

HOUSEHOLD INCOME

Below R50,000

CURRENT FUNDING

I currently have no funding

NEEDS FUNDING FOR

Tuition fees, Accommodation, Books / study materials, Meals / living allowance, Full funding

CITIZENSHIP

South African citizen

FIRST-GENERATION STUDENT

Yes

Previous

Next

Review decision

The applicant is emailed when you shortlist, select, reject or request information.

Already shortlisted

Move to Selected

Request Information

Reject

Move back to review

Eligibility summary

Course supported

Course - worth 28%

Institution supported

Institution - worth 22%

Academic requirement met

Academic requirement - worth 18%

Qualification level matches

Qualification level - worth 8%

Open to all provinces

Location requirement - worth 8%

Financial requirement met

Financial requirement - worth 4%

Figure 35 Applicant review, with the eligibility verdict

 Bursary-Bridge

Dashboard

Programmes

Applications

Shortlists

Beneficiaries

Reports

Notifications1

Organisation

Settings

Help & Support

Log out

Shortlists

Applicants you're considering. Move them to Selected when you're ready to award funding.

All programmes

Best match

☐	APPLICANT	MATCH %	AVERAGE	PROGRAMME	DATE SHORTLISTED	STATUS
☐	FM Farhaan Mthembu Software Engineering · University of Cape Town	100% Match	86%	2026 Technology Bursary ...	24 Sept 2026	Shortlisted
☐	KN Kagiso Naidoo Computer Science · University of Pretoria	100% Match	93%	2026 Technology Bursary ...	24 Sept 2026	Shortlisted
☐	AZ Amahle Zulu Accounting · University of Pretoria	89% Match	—	Chartered Accountancy Sc...	24 Sept 2026	Shortlisted
☐	MR Megan Radebe Computer Science · University of Pretoria	89% Match	—	2026 Technology Bursary ...	24 Sept 2026	Shortlisted
☐	AP Amahle Petersen Nursing · University of KwaZulu-Natal	72% Match	75%	Chartered Accountancy Sc...	24 Sept 2026	Shortlisted

Showing 5 of 5

Figure 36 Shortlists

Bursary-Bridge

Dashboard

Programmes

Applications

Shortlists

Beneficiaries

Reports

Notifications1

Organisation

Settings

Help & Support

Log out

Beneficiaries

Applicants your organisation has selected for funding.

Q Search by name

All programmes

Best match

APPLICANT	MATCH %	AVERAGE	PROGRAMME	DATE SELECTED	STATUS
AM Anele Mabaso Accounting - University of Pretoria	82% Match	59%	Chartered Accountancy Scholars...	24 Sept 2026	Selected
TB Thabo Botha Accounting - University of Pretoria	72% Match	93%	2026 Technology Bursary Progra...	24 Sept 2026	Selected

Showing 2 of 2

Figure 37 Beneficiaries

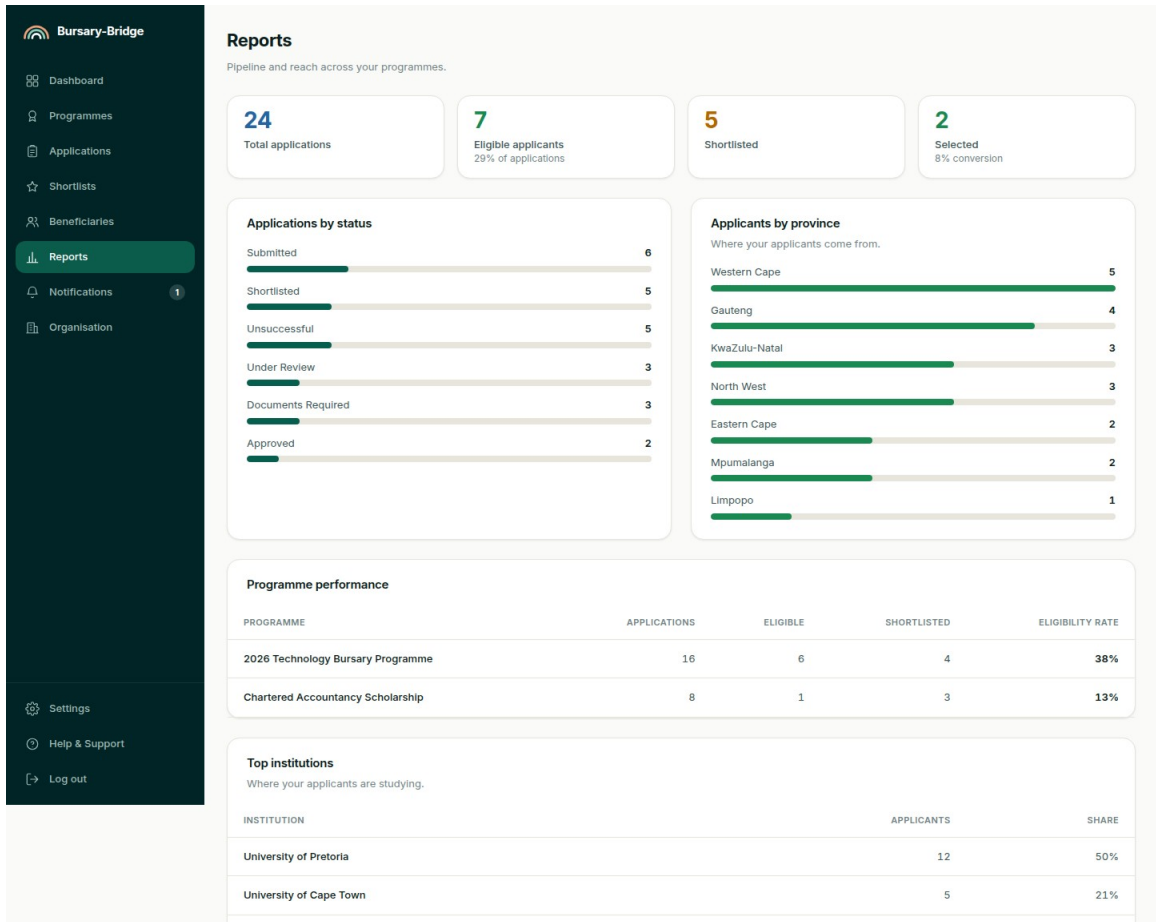


Figure 38 Corporate reports

 Bursary-Bridge

Dashboard

Programmes

Applications

Shortlists

Beneficiaries

Reports

Notifications1

Organisation

Settings

Help & Support

Log out

Kgotso Holdings

On Bursary-Bridge since 24 Sept 2026

Edit organisation

2

Active programmes

24

Applications

5

Shortlisted

2

Beneficiaries

Your details

Edit

Name

Sarah Dlamini

Work email

corporate@demo.bursarybridge.local

Mobile

0823456789

Organisation

Edit

Organisation name

Kgotso Holdings

Type

Corporation

Industry

Financial Services

Website

https://www.kgotsoholdings.example

Country

South Africa

Your role

Edit

Role

Corporate Social Investment Manager

Organisation size

251 – 1,000 employees

Department

Corporate Social Investment / CSI

Funding programmes

Edit

Currently offers funding

Yes

Programme types

Bursaries, Scholarships, Internships

Annual applications

1,001 – 5,000

Current process

Edit

Manages applications with

Email, Excel / Spreadsheets, Manual process

Biggest challenges

Too many applications, Manual screening, Document verification

Users & roles

People who can manage this organisation's programmes and applicants.

Sarah Dlamini

corporate@demo.bursarybridge.local

Figure 39 Organisation profile

Page 74 of 83

CONNECTING TALENT. FUNDING FUTURES.

Real opportunities. A brighter tomorrow.

Bursary-Bridge connects students with bursaries, scholarships and funding opportunities and helps organisations invest in tomorrow's leaders.

I'm a Student →

I'm an Organisation →

🕒 Free for students

✅ Verified against the source

🌐 Opportunities nationwide

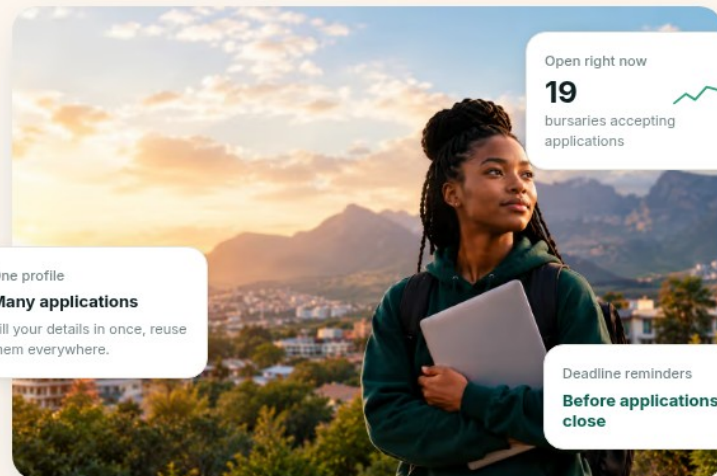


Figure 40 Landing page at a tablet width of 834 pixels

Opportunities

24 funding opportunities are open right now, ranked by how well they match your profile.

Best match


 Filters

Showing 8 of 24 opportunities

HP Inc Bursary South Africa 2027

Bursary

100%
Match

HP Inc

HP Inc. is an information technology company known for its personal computers (PCs), printers and related supplies, as well as 3D printing solutions. HP Inc. was formed in 2015, as the legal successor of the original Hewlett-Packard Company (which was formed in 1939), after the original Hewlett-Pack

SUPPORTED COURSES

All courses

SUPPORTED INSTITUTIONS

All institutions

 Closes 30 Sept 2026 · 6 days left

[View Opportunity](#)

Western Cape Department of Health and Wellness Bursary South Africa 2027

Bursary

100%
Match

Western Cape Department of Health and Wellness

The Western Cape Department of Health and Wellness, formed in 1994, is responsible for delivering public healthcare and health services to people who reside within the province. They are also in charge of various specialised facilities, which include rehabilitation centres; psychiatric hospitals and

SUPPORTED COURSES

All courses

SUPPORTED INSTITUTIONS

All institutions

 Closes 30 Sept 2026 · 6 days left

[View Opportunity](#)

South African National Space Agency (SANSA) Bursary South Africa 2027

Bursary

100%
Match

South African National Space Agency (SANSA)

The South African National Space Agency (SANSA), established in 2010, are a public entity and government agency operating under the National Department of Science and Technology (DST). SANSA are responsible developing and promoting aeronautics and aerospace space research, with the companies main fo

SUPPORTED COURSES

All courses

SUPPORTED INSTITUTIONS

All institutions

Figure 41 Opportunities at a tablet width of 834 pixels

Applications

Every application submitted to your organisation's programmes.

All (24) Submitted In Review (6) Shortlisted (5) Selected (2) Rejected (5)					
Search applicants by name or email		All programmes		Filters	
☐	APPLICANT	MATCH %	AVERAGE	ELIGIBILITY	STATUS
☐	FM Farhaan Mthembu Software Engineering · Bachelor's Degree University of Cape Town	100% Match	86%	Eligible	Shortlisted
☐	TB Thabo Botha Accounting · Bachelor's Degree University of Pretoria	100% Match	93%	Eligible	Submitted
☐	PM Palesa Mabaso Computer Science · Bachelor's Degree University of Pretoria	100% Match	90%	Eligible	Unsuccess
☐	AN Asanda Nsibande Computer Science · Bachelor's Degree University of Pretoria	100% Match	82%	Eligible	Documents Required
☐	PM Palesa Motaung Computer Science · Bachelor's Degree University of Pretoria	100% Match	86%	Eligible	Documents Required
☐	MD Mpho Dube Software Engineering · Bachelor's Degree University of Cape Town	100% Match	74%	Eligible	Submitted
☐	KN Kagiso Naldoo Computer Science · Bachelor's Degree University of Pretoria	100% Match	93%	Eligible	Shortlisted
☐	AZ Amahle Zulu Accounting · Bachelor's Degree University of Pretoria	89% Match	—	Pending verification	Shortlisted
☐	MM Megan Mabaso Software Engineering · Bachelor's Degree University of Cape Town	89% Match	—	Pending verification	Unsuccess
☐	TJ Tumelo Jacobs Computer Science · Bachelor's Degree University of Pretoria	89% Match	—	Pending verification	Submitted

Figure 42 Applications received at a tablet width of 834 pixels

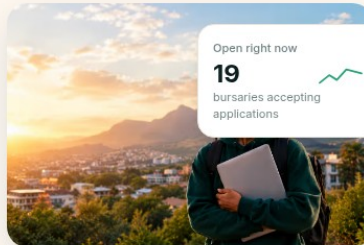
CONNECTING TALENT. FUNDING FUTURES.

Real opportunities. A brighter tomorrow.

Bursary-Bridge connects students with bursaries, scholarships and funding opportunities and helps organisations invest in tomorrow's leaders.

[I'm a Student →](#)[I'm an Organisation →](#)

- ✓ Free for students
- ✓ Verified against the source
- 🌐 Opportunities nationwide



Open right now

19bursaries accepting
applications

Figure 43 Landing page at a mobile width of 390 pixels

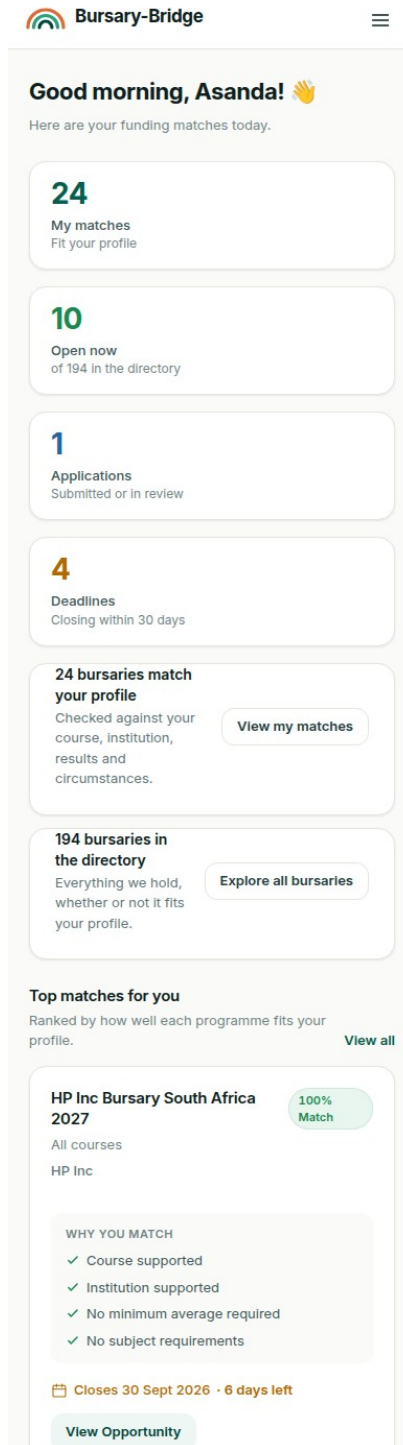


Figure 44 Student dashboard at a mobile width of 390 pixels

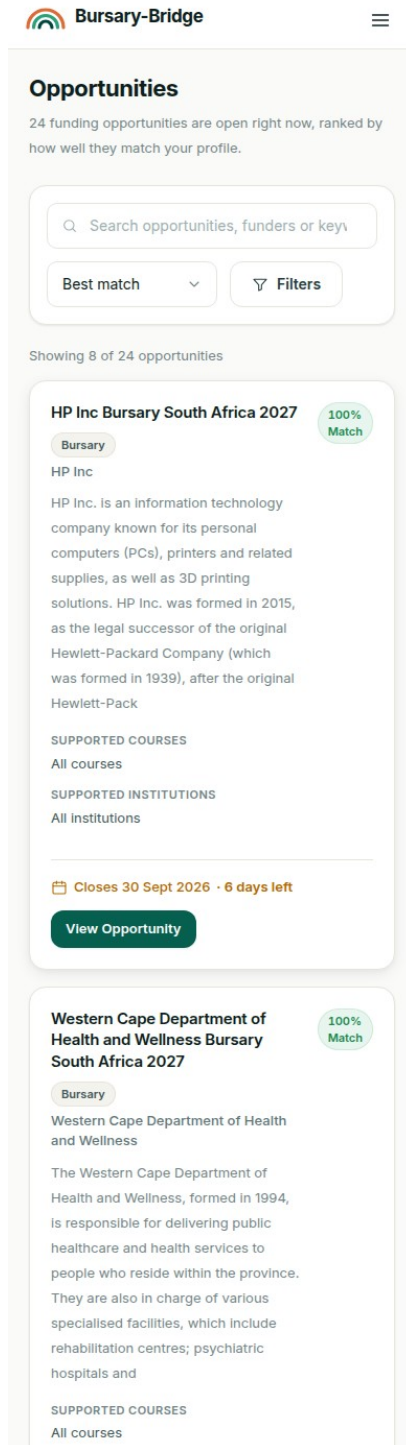


Figure 45 Opportunities at a mobile width of 390 pixels

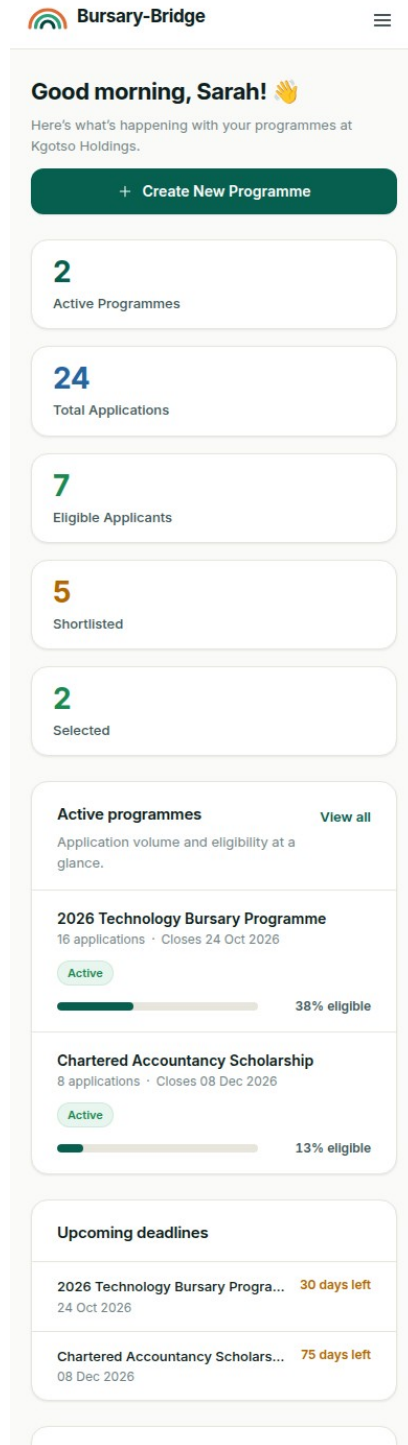


Figure 46 Corporate dashboard at a mobile width of 390 pixels

7.0. Help system design

The system carries contextual guidance rather than a separate help application. Wherever the system asks for something it says why it is being asked, each onboarding step carries a

short explanation of how the information affects matching, and every match is accompanied by the reasons behind its score, which is the explanation a Student most often needs.

A help page is provided at /help, together with the terms of service and the privacy notice, which states what is collected and why. No indexed help system, guided tour or offline documentation is planned for this release.

8.0. Index

Administration Portal.....	4–5, 13–14
Administrator.....	4, 14–15
App Router.....	4–5, 7
Applicant.....	5, 8, 12–13, 18–20, 30, 36, 45, 47–49, 70
Application.....	4–5, 8–10, 12–14, 17–18, 20–21, 23, 26–36, 40, 43–48, 60–62, 81
Application Import Pipeline.....	18
Application Programming Interface.....	23, 26
ApplicationImportBatch.....	35
Audit Trail.....	14
Authentication and Session Module.....	11–12, 14, 21, 27
Bursary Ingestion Pipeline.....	15
Bursary-Bridge Database.....	16, 18, 22, 29–31
Corporate Portal.....	4, 12, 50
Corporate User.....	4–5, 8, 12–13, 18, 37, 44–48
Design Entity.....	5–6
Document Extraction Engine.....	16–19, 43, 45
Document Storage Service.....	17–18, 27–28
Edge Middleware.....	10, 27
Eligibility Outcome.....	28
Eligibility Rule.....	5, 13, 24, 26, 31, 44
Eligibility Service.....	12–13, 18–19, 25, 28, 45, 47
Funding Programme.....	5–6, 12–14, 18–19, 21, 23–24, 28, 36, 44–45, 68
Guard.....	5, 10–14, 27–28, 32, 35–36, 47
Match Score.....	5, 12, 28, 31, 44
Matching Engine.....	5–6, 8, 10–11, 13, 18–19, 23, 25–26, 28, 31, 39, 45
Motivational Letter Composer.....	20, 42

Next.js.....	4–5, 7–8
Notification and Email Service.....	30
Organisation.....	4–6, 12–14, 17–20, 22, 28, 30, 34, 36, 45, 47–48, 50, 55, 67, 74
PostgreSQL.....	4, 8, 31, 33
Prisma.....	4, 6–7, 10, 24, 31–33
Programme.....	5–6, 11–14, 18–19, 21, 23–26, 28, 32, 34–36, 40, 42, 44–45, 47–50, 68
Route Handler.....	6, 8, 11, 28, 30
Server Component.....	6
Service Layer.....	6, 10–14, 26–27, 32, 36, 39
Session.....	6, 10–12, 14–15, 21–23, 27–29, 33, 36–37
Shortlist.....	32, 48–49
Student.....	4–6, 8, 10–12, 14, 17–29, 31, 33–34, 36–44, 46, 49–50, 54, 56–57, 61, 63–65, 79
Student Portal.....	4, 10, 50
Study Preference.....	6, 31, 36
Zod.....	6, 27

Bursary-Bridge System Test Plan

Test Plan Identifier

Bursary-Bridge TP_2.0

This test plan covers the Bursary-Bridge system developed by CPP Mchunu (202242486), A Nsibande (240079346), KN Bikile (202356797) and SP Tshazi (240103222) for the Department of Computer Science, University of Zululand, under the supervision of Ms Zulu.

References

The documents below support this test plan and are cited by bracketed number where they are relied on. Every source listed here is cited at least once in the text.

- [1] Department of Computer Science, University of Zululand, “Final Year Project Proposal: Bursary-Bridge,” 2026.
- [2] Department of Computer Science, University of Zululand, “Software Requirements Specification: Bursary-Bridge,” 2026. Section 3.2 defines the thirty-seven use cases from which the features in this plan are drawn.
- [3] Department of Computer Science, University of Zululand, “Software Design Description: Bursary-Bridge,” 2026. Section 5 traces twelve of those use cases to the design objects that carry them out.
- [4] Republic of South Africa, Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer, 2013. [Online]. Available: <https://www.justice.gov.za/legislation/acts/2013-004.pdf>.

Introduction

This test plan for the Bursary-Bridge system supports the following objectives:

1. To define the tools to be used throughout the testing process.
2. To communicate to the responsible parties the items to be tested, set expectations around schedule, and define environmental needs.
3. To define how the tests will be conducted.

Test Items

The systems to be tested are the Student portal, the Corporate portal, the Administration portal, the application programming interface that serves all three, and the server-side modules behind them: the matching engine, the eligibility service, the bursary ingestion pipeline, the document extraction engine, the application import pipeline and the motivational letter composer. Persistent state is held in PostgreSQL and reached through Prisma, and the stored schema is itself a test item because the eligibility model depends on its typed columns.

The automated interface checks run in Chromium through Playwright at a desktop width of 1440 pixels, a tablet width of 834 pixels and a mobile width of 390 pixels. Manual checks use Chromium as well, so one browser engine is exercised and only one.

Features To Be Tested

Features to be tested include the following:

- As a student, registering an account and signing in
- As a student, resetting a forgotten password through an issued token
- As a student, completing the profile through the onboarding steps
- As a student, adding study preferences that pair a course with an institution
- As a student, capturing subject and module results that the matching engine reads
- As a student, viewing matched opportunities ranked by match score
- As a student, viewing the per-criterion reasons behind a match score
- As a student, browsing the full directory of opportunities the system holds, whether or not they match the profile
- As a student, searching and filtering the directory, and seeing the source each opportunity was taken from
- As a student, viewing the eligibility requirements of an opportunity and the verdict against them
- As a student, submitting an application to a funding programme
- As a student, tracking the status of a submitted application
- As a student, uploading and managing supporting documents
- As a student, having a downloaded PDF application form completed from the stored profile, and reviewing every field before accepting it

- As a student, generating a motivational letter assembled from stored facts and the student's own answers
- As a student, responding to a funder's request for further information
- As a corporate user, registering an organisation account
- As a corporate user, completing the organisation profile
- As a corporate user, creating a funding programme and its structured eligibility criteria
- As a corporate user, publishing a funding programme
- As a corporate user, viewing applications received
- As a corporate user, reviewing an applicant and the eligibility verdict
- As a corporate user, requesting further information from an applicant
- As a corporate user, shortlisting an applicant and selecting a beneficiary
- As a corporate user, importing in bulk the applications the organisation received outside the system, and reviewing the extraction before anything is imported
- As a corporate user, viewing reports
- As an administrator, maintaining the catalogue of institutions, qualifications and subjects
- As an administrator, suspending and restoring an account
- The ingestion pipeline: obeying robots directives, recording the source of every imported opportunity, rejecting a listing it cannot fully resolve, and resolving a disagreement between two sources in favour of the more authoritative one
- The scoring rules of the matching engine, including the weight of each criterion, the treatment of a criterion that cannot be evaluated, and the classification thresholds
- The eligibility service, including that missing information yields `PENDING_VERIFICATION` rather than an automatic rejection
- Document extraction, including that a field read with insufficient confidence is left empty rather than guessed at
- Duplicate detection and identity resolution in the import pipeline, including that a suspected duplicate is flagged rather than merged or deleted
- Access control, that a student cannot read another student's data and a corporate user cannot read another organisation's applicants
- Server-side validation of every submitted form
- The responsive layout at desktop, tablet and mobile widths

Features Not To Be Tested

Fund disbursement will not be tested, because the prototype does not disburse funds. Verification of submitted documents against any external authority will not be tested, because no such integration exists. Integration with the student information systems of universities or with government systems will not be tested, because it lies outside the scope of this release.

Delivery of electronic mail will not be tested. Notifications are written to the database and shown in the application, and no outbound mail is dispatched, so there is no delivery to observe. Storage of documents in S3-compatible object storage will not be tested, because the prototype writes to local disk. Generation of prose by a language model will not be tested, because the motivational letter composer assembles text deterministically from stated facts and reaches no model.

Behaviour in browsers other than Chromium will not be tested. Both the automated suites and the manual checks use Chromium, so a defect specific to the Gecko or WebKit engine would not be found by this plan. This is a known limitation of the plan rather than a claim about the system.

Load, stress and performance testing are outside the scope of this plan. The system is tested for correctness, not for behaviour under concurrent load, and no performance target has been set for this release.

Approach

Testing is carried out in five layers, all of which run from the project repository with a single command each.

First, static checking. The command `npm run typecheck` runs the TypeScript compiler across the whole project. Because the database client is generated from the schema, a change that breaks a query fails this step rather than failing at run time.

Second, unit testing of the pure domain modules. These suites exercise the scoring rules, the eligibility rules and the field-label matching against constructed inputs for which the correct answer is known. They reach neither a browser nor a server.

Third, integration testing of the service layer against a real database. These suites create their own data, exercise the service functions directly, assert the resulting database state, and remove only what they created. Nothing is mocked and no suite may delete a record it did not write.

Fourth, end-to-end testing through the real interface. The command `npm run test:e2e` drives complete journeys against a running server using real HTTP requests and real session cookies, then asserts the database state that resulted. Two further browser suites drive Chromium through Playwright to check the bursary directory and the letter composer as a user sees them.

Fifth, accuracy measurement. The command `npm run test:pdf` fills prepared application forms whose correct values are known in advance, then reads the saved documents back and reports how many fields were populated correctly. This measures accuracy rather than returning a pass or a fail, because a field the system correctly declines to guess at is a success and not a failure.

The counted suites and the checks each contributed on the run of 24 September 2026 are as follows:

- `npm run test:matching` – the matching engine, the eligibility service and the subject requirements: 35 checks
- `npm run test:pdf:labels` – field-label matching and normalisation: 39 checks
- `npm run test:autofill` – the auto-fill service and its handling of unreadable documents: 32 checks
- `npm run test:catalogue` – the catalogue of institutions, qualifications and courses, and the service over it: 43 checks
- `npm run test:results` – subject and module results and their effect on matching: 32 checks
- `npm run test:requests` – requests for further information: 31 checks
- `npm run test:letters` – the letter composer and the service over it: 87 checks
- `npm run test:import` – bulk application intake, extraction, duplicate detection and identity resolution: 59 checks
- `npm run test:ingest` – robots handling, parsing, provenance, deduplication and conflict resolution: 78 checks
- `npm run test:directory` – the bursary directory in the browser: 25 checks
- `npm run test:letters:ui` – the letter composer in the browser: 27 checks
- `npm run test:e2e` – complete student and organisation journeys through the real interface: 109 checks

A separate audit, `npm run audit:production`, confirms that no funding opportunity visible to a user is invented rather than traceable to a recorded source. It is run before any demonstration or submission.

Manual exploratory testing covers what the automated suites do not reach, in particular the rendering of each screen at the three widths and the usability of the onboarding steps. Defects found by any layer are recorded as GitHub issues against the project repository, with the steps needed to reproduce them.

Pass/Fail Criteria

A test run passes when npm run typecheck reports no errors and every check in every automated suite passes. The command npm test runs all thirteen commands in sequence and stops at the first that fails. Twelve of them report a pass and fail count; the thirteenth is the accuracy harness, which is judged as described below. On the run of 24 September 2026 every command succeeded: 597 checks passed across the twelve counted suites, none failed, and the compiler reported no errors.

The accuracy harness is judged differently, because it measures rather than asserts. It passes when every field it populated carries a value the source document actually stated, when every populated field is present in the saved PDF on read-back, and when no field was filled on a low-confidence guess. On the run recorded above it checked 46 fields across 6 forms with 46 correct and none filled from anything the source did not say, populating 20 of the 46 fields; the remaining 26 were correctly left for the student because the source form did not answer them.

A defect is classified as high severity when it prevents a user from completing a journey, exposes data belonging to another user or organisation, causes the match score shown to a student to contradict the eligibility verdict shown to a funder, or causes the system to present information it cannot trace to a source. The system is not accepted for submission while any high-severity defect remains open.

Suspension Criteria

Testing is paused immediately if users cannot sign in, if the database is unavailable or its migrations fail to apply, or if the test environment cannot be prepared. Testing is also paused if a defect is found that exposes one user's data to another, because every later result would be unsafe to trust until that is corrected and because the system holds information the Protection of Personal Information Act [4] requires to be kept from anyone not entitled to see it.

Testing is paused if a suite is found to delete data it did not create. A suite that removes records belonging to the imported directory destroys the evidence that later checks depend on, and any result recorded after such

a run must be discarded and the run repeated. Testing resumes once the blocking defect is fixed and the affected checks pass again.

Test Deliverables

The deliverables of this testing project are this test plan, the recorded output of each test run showing the checks that passed and failed, the accuracy report produced by the document extraction harness, the report produced by the production audit, the list of defects recorded during testing, and a test summary report prepared at the end of each round.

Testing Tasks

The following activities must be completed:

- Test plan prepared.
- Software Requirements Specification [2] and Software Design Description [3] delivered to the testing team.
- Environment prepared: Node.js 20 or later installed, PostgreSQL 16 running, the database created and its migrations applied, and the bursary directory restored from the recorded snapshot.
- Static checking run and its output recorded.
- Automated suites run and their output recorded.
- Accuracy harness run and its figures recorded.
- Production audit run and its report recorded.
- Manual exploratory testing performed on what the automated suites do not reach.
- Defects recorded and assigned.
- Defects fixed and the affected checks re-run.
- Test summary report prepared.

Environmental Needs

Testing requires a machine with Node.js 20 or later and PostgreSQL 16. Node.js 20 is the floor because Playwright, which drives the browser suites, requires it. Chromium must be available to Playwright.

The repository must contain a .env file holding DATABASE_URL and AUTH_SECRET. The database must be created and its migrations applied with npm run db:deploy. The bursary directory is restored with

npm run import:snapshot, which loads the 189 opportunities the ingestion pipeline collected, each carrying the source page it was read from; the matching suites need that directory in place to have anything to score against.

The end-to-end and browser suites need the development server running on <http://localhost:3000>. Each suite creates the accounts it needs, uses them, and removes them again, so no standing test accounts are required. Where a screen can only be populated by an organisation that has received applications, demonstration data is created for the purpose, clearly identified as such, and removed with `npm run purge:mock` before delivery.

Responsibilities

The Test Manager, Salma, is responsible for facilitating the testing project, preparing the test environment, coordinating the availability of the testers, reviewing the results of each test run and reporting back to the project team and to the supervisor.

The testers are responsible for running the automated suites, performing the manual checks assigned to them, and recording each defect with the steps needed to reproduce it. The developers are responsible for fixing recorded defects and confirming that the affected checks pass again. No developer signs off a defect in their own area without a second member confirming the fix.

Staffing And Training Needs

Testing is carried out by the project team of four. Each tester should work through the student journey and the organisation journey, so that no journey is only ever seen by one person, and the areas of the system allocated in Section 10.1 of the Final Year Project Proposal [1] should not determine who tests them.

Testers need no specialist tools beyond the repository itself, but they must be able to run the project locally and read the output of the test scripts. Before testing begins they should read Section 3.2 of the Software Requirements Specification [2], so that the expected behaviour of each use case is clear; they should understand how the weighted criteria produce a match score, so that an incorrect score is recognised rather than accepted; and they should understand that a criterion the system cannot evaluate is reported as requiring verification rather than as a failure, so that correct behaviour is not recorded as a defect.

Schedule

Testing takes place between 21 and 24 September 2026. The submission date is 24 September 2026, which is the last day of that window.

One week is allowed for the first round of testing. That is longer than the window, so the round does not sit inside it. The automated part of the first round begins before 21 September and needs no scheduled window of its own, because the full sequence runs in minutes and is run on every change in any case. The four days from 21 to 24 September are reserved for the manual checks, which need a person rather than a command.

Because the submission date is the last day of the window, there is no time after it. Any defect found between 21 and 24 September must be fixed and its checks re-run within those four days, or accepted as an open defect and recorded as such in the test summary report.

The static check and the unit suites are quick enough to be run on every change. The integration and browser suites are run before each increment is shown to the supervisor. The full sequence, the accuracy harness and the production audit are run before any demonstration and before submission.

Risks And Contingencies

If the environment is not prepared correctly, testing cannot start. Three failures have been seen in practice: PostgreSQL not running, Node.js older than version 20, and a stale generated database client left behind when a dependency install was a no-op. The setup script in the repository addresses all three, and testers should use it rather than preparing the environment by hand.

If a suite deletes data it did not create, the imported directory can be lost during a run and every later result becomes unreliable. This has occurred: two suites removed every externally sourced organisation rather than only the ones they had written, and one run of the full sequence emptied the directory of all 189 imported bursaries. Both suites now record what exists before they write anything and remove only the difference. The contingency is the recorded snapshot: `npm run import:snapshot` restores the directory in full, and it should be run and the audit repeated before results are trusted again.

If a suite assumes it is the only writer, it can fail for reasons that are not defects. Checks that expected a fixture to appear on the first page of the directory began failing once the directory held real imported bursaries; the checks now search for their own fixture instead of assuming its position. A failure of this kind

must be diagnosed before it is reported, because recording it as a system defect wastes development time on correct behaviour.

If the testers do not understand how a match score is produced, an incorrect score may be accepted as correct, or correct handling of an unevaluable criterion may be reported as a defect. The training described above reduces this risk.

If the first round of testing is not completed on schedule, there is no slack to absorb it: the submission date of 24 September 2026 is the last day of the testing window, so a round that runs late does not delay the final round, it removes it. The contingency is to prioritise the checks that cover access control, the matching engine and the provenance of imported data, because a defect in any of the three is the most serious kind this system can have, and to record anything left unchecked in the test summary report rather than leaving it unstated.

Approvals

The Test Manager, Salma, and the Project Supervisor, Ms Zulu, must both agree that the testing project is complete and determine when the system is ready for submission.